

Lab Manual

CSC340 Computer Networks



**Department of Computer Science
Islamabad Campus**

CUI

Lab Contents:

The aim of this lab manual is to learn and practice concepts related to Data Communication and Computer Networks. The topics include Hyper Text Transfer Protocol(HTTP), DNS, Transmission Control Protocol (TCP) Using Wireshark ; Socket Programming; Router configuration; IP addresses assignment using static and Dynamic configurations; Variable Length Subnet Mask (VLSM); Dynamic Routing; Security using ACLs, Network Address Translation , configuration of Wireless Networks and Firewalls.

Student Outcomes (SO)

S.#	Description
2	Identify, explore several aspects of the HTTP, TCP protocol, better understand the working of DNS
3	IP addresses assignment using static and Dynamic configurations, VLSM, Dynamic routing, Security using ACLs,
4	Design a physical network architecture to execute network services and applications
5,9	Create, a network application using socket programming, design a physical network architecture to execute network services and applications

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -5	Create a network application using socket programming.	<i>Applying</i>	2,4
CLO -6	Design a physical network architecture to execute network services and Applications.	<i>Creating</i>	2-5

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab #	Main Topic	Page #
Lab 01	Getting started: Introduction to Wireshark	05
Lab 02	Introduction to Packet Tracer	15
Lab 03	Hyper Text Transfer Protocol (HTTP) using Wireshark	40
Lab 04	Domain Name System (DNS) using Wireshark	49
Lab 05	Transport Control Protocol (TCP) using Wireshark	62
Lab 06	User Datagram protocol (UDP) using Wireshark	7
Lab 07	Socket Programming	78
Lab 08	Basic Configurations	106
	Mid Term Exam	
Lab 09	Dynamic Host Configuration Protocol (DHCP)	125
Lab 10	Subnet an IPv4 Network and Static Routing	136
Lab 11	Network Address Translation (NAT) Configuration	163
Lab 12	RIP Protocol configuration	175
Lab 13	Open Shortest Path First (OSPF), Configuring OSPF in Single Area, Configuring OSPF in Multiple Area	184
Lab 14	EIGRP	201
Lab 15	Configure a Wireless Network	213
Lab 16	Configure IPv4 Access Control List (ACL), Firewall Implementation (Allow Good Traffic and Deny Bad Traffic)	222
	Final Term Exam	

Lab 01

Introduction to Wireshark

Objective:

The objective of this lab is that the students get acquainted with Wireshark and make some simple packet captures and observations.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Students will have gained the basic understanding of Wireshark Packet Sniffer to see “protocols in action”
- After this lab, students will have developed basic understanding of digging deep into the network protocols.

Instructor Note:

Wireshark is a network packet analyzer. A network packet analyzer presents captured packet data in as much detail as possible. You could think of a network packet analyzer as a measuring device for examining what’s happening inside a network cable, just like an electrician uses a voltmeter for examining what’s happening inside an electric cable (but at a higher level, of course). In the past, such tools were either very expensive, proprietary, or both. However, with the advent of Wireshark, that has changed. Wireshark is available for free, is open source, and is one of the best packet analyzers available today.

In this Wireshark lab, you’ll capture some http packets using Wireshark and make some observations on them. For more details, please check the below reference.

References

- <https://www.wireshark.org/>

1. Useful Concepts

One's understanding of network protocols can often be greatly deepened by "seeing protocols in action" and by "playing around with protocols" – observing the sequence of messages exchanged between two protocol entities, delving down into the details of protocol operation, and causing protocols to perform certain actions and then observing these actions and their consequences. This can be done in simulated scenarios or in a "real" network environment such as the Internet. In the Wireshark labs you'll be doing in this course, you'll be running various network applications in different scenarios using your own computer (or you can borrow a friend's; let me know if you don't have access to a computer where you can install/run Wireshark). You'll observe the network protocols in your computer "in action," interacting and exchanging messages with protocol entities executing elsewhere in the Internet. Thus, you and your computer will be an integral part of these "live" labs. You'll observe, and you'll learn, by doing. For more details, please check the below reference.

References:

1. Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
2. Wireshark Labs, https://gaia.cs.umass.edu/kurose_ross/wireshark.php
3. Wireshark User Guide: <https://www.wireshark.org/download/docs/user-guide.pdf>
4. Wireshark Walkthrough: <https://www.youtube.com/watch?v=TkCSr30UojM>
5. Wireshark Website: <https://www.wireshark.org>

2. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>20</i>	<i>Low</i>	<i>CLO-6</i>
<i>3</i>	<i>20</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1:

Getting Wireshark

In order to run Wireshark, you will need to have access to a computer that supports both Wireshark and the libpcap or WinPCap packet capture library. The libpcap software will be installed for you, if it is not installed within your operating system, when you install Wireshark. See <http://www.wireshark.org/download.html> for a list of supported operating systems and download sites

Download and install the Wireshark software:

- Go to <http://www.wireshark.org/download.html> and download and install the Wireshark binary for your computer. The Wireshark FAQ has a number of helpful hints and interesting tidbits of information, particularly if you have trouble installing or running Wireshark.

Running Wireshark

When you run the Wireshark program, you'll get a startup screen, as shown below:

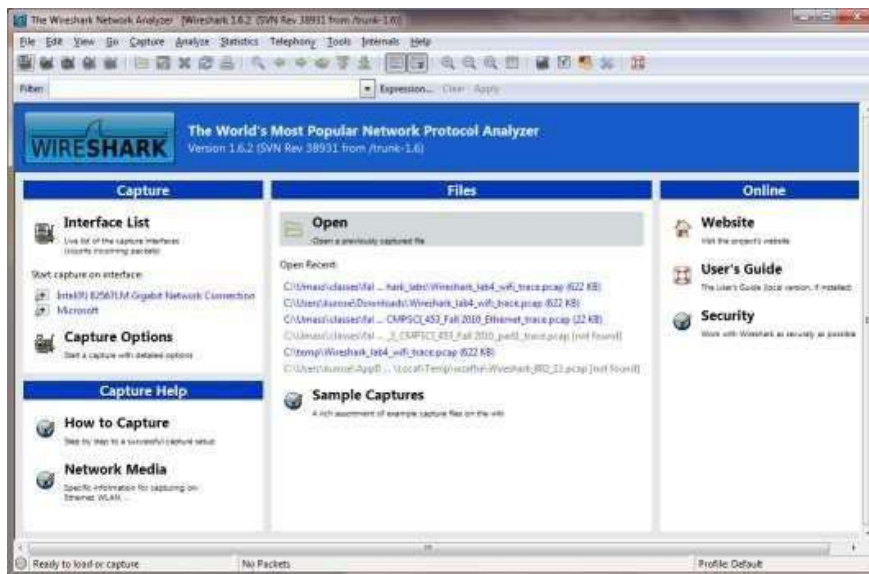


Figure 1: Initial Wireshark Screen

Take a look at the upper left hand side of the screen – you'll see an "Interface list". This is the list of network interfaces on your computer. Once you choose an interface, Wireshark will capture all packets on that interface. In the example above, there is an Ethernet interface (Gigabit network Connection) and a wireless interface ("Microsoft").

If you click on one of these interfaces to start packet capture (i.e., for Wireshark to begin capturing all packets being sent to/from that interface), a screen like the one below will be displayed, showing information about the packets being captured. Once you start packet capture, you can stop it by using the Capture pull down menu and selecting Stop.

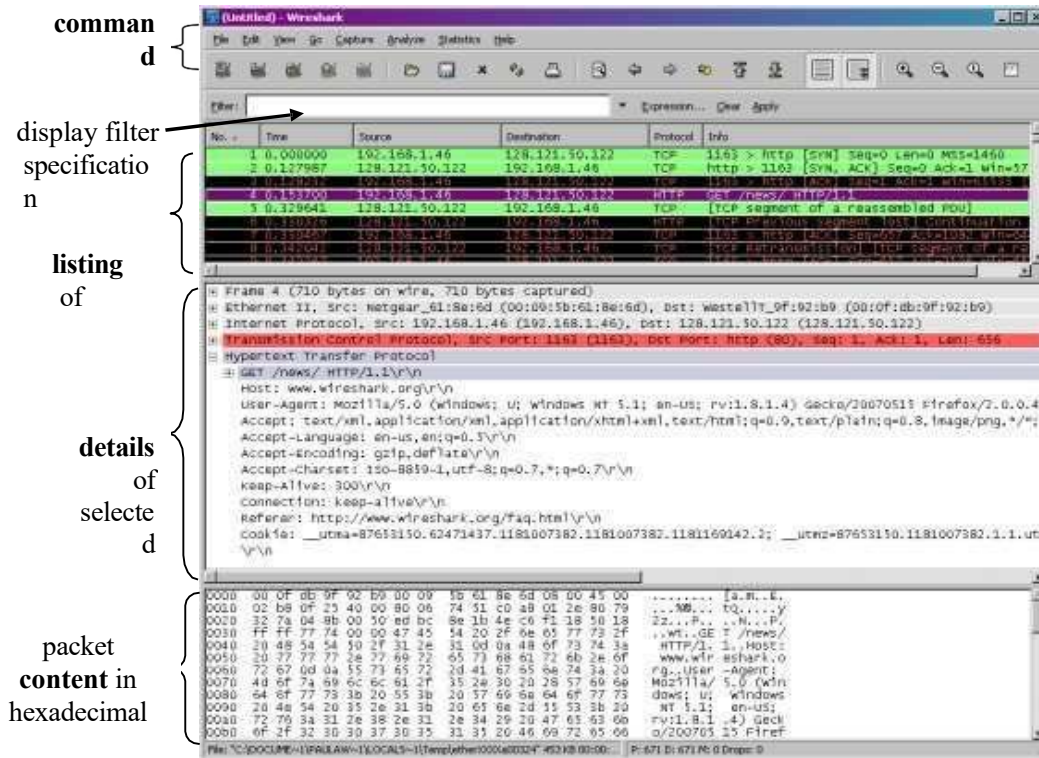


Figure 2: Wireshark Graphical User Interface, during packet capture and analysis

The Wireshark interface has five major components:

- The command menus are standard pulldown menus located at the top of the window. Of interest to us now are the File and Capture menus. The File menu allows you to save captured packet data or open a file containing previously captured packet data, and exit the Wireshark application. The Capture menu allows you to begin packet capture.
- The packet-listing window displays a one-line summary for each packet captured, including the packet number (assigned by Wireshark; this is not a packet number contained in any protocol's header), the time at which the packet was captured, the packet's source and destination addresses, the protocol type, and protocol-specific information

contained in the packet. The packet listing can be sorted according to any of these categories by clicking on a column name. The protocol type field lists the highest-level protocol that sent or received this packet, i.e., the protocol that is the source or ultimate sink for this packet.

- The packet-header details window provides details about the packet selected (highlighted) in the packet-listing window. (To select a packet in the packet-listing window, place the cursor over the packet's one-line summary in the packet-listing window and click with the left mouse button.). These details include information about the Ethernet frame (assuming the packet was sent/received over an Ethernet interface) and IP datagram that contains this packet. The amount of Ethernet and IP-layer detail displayed can be expanded or minimized by clicking on the plus minus boxes to the left of the Ethernet frame or IP datagram line in the packet details window. If the packet has been carried over TCP or UDP, TCP or UDP details will also be displayed, which can similarly be expanded or minimized. Finally, details about the highest-level protocol that sent or received this packet are also provided.

- The packet-contents window displays the entire contents of the captured frame, in both ASCII and hexadecimal format.

Towards the top of the Wireshark graphical user interface, is the packet display filter field, into which a protocol name or other information can be entered in order to filter the information displayed in the packet-listing window (and hence the packet-header and packet-contents windows). In the example below, we'll use the packet-display filter field to have Wireshark hide (not display) packets except those that correspond to HTTP messages.

Activity 2:

The best way to learn about any new piece of software is to try it out! We'll assume that your computer is connected to the Internet via a wired Ethernet interface. Indeed, I recommend that you do this first lab on a computer that has a wired Ethernet connection, rather than just a wireless connection. Do the following

1. Start up your favorite web browser, which will display your selected homepage.
2. Start up the Wireshark software. You will initially see a window similar to that shown in Figure 2. Wireshark has not yet begun capturing packets.
3. To begin packet capture, select the Capture pull down menu and select Interfaces. This will cause the "Wireshark: Capture Interfaces" window to be displayed, as shown in Figure 4.

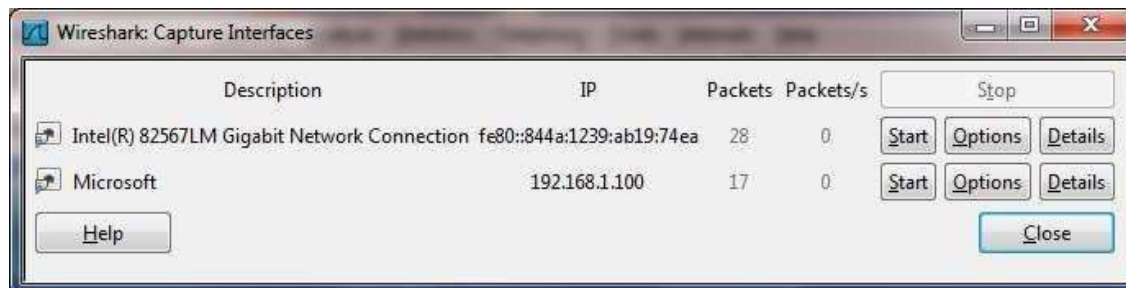


Figure 3: Wireshark Capture Interface Window

4. You'll see a list of the interfaces on your computer as well as a count of the packets that have been observed on that interface so far. Click on Start for the interface on which you want to begin packet capture (in the case, the Gigabit network Connection). Packet capture will now begin - Wireshark is now capturing all packets being sent/received from/by your computer!

5. Once you begin packet capture, a window similar to that shown in Figure 3 will appear. This window shows the packets being captured. By selecting Capture pulldown menu and selecting Stop, you can stop packet capture. But don't stop packet capture yet. Let's capture some interesting packets first. To do so, we'll need to generate some network traffic. Let's do so using a web browser, which will use the HTTP protocol that we will study in detail in class to download content from a website.

6. While Wireshark is running, enter the URL: <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>

and have that page displayed in your browser. In order to display this page, your browser will contact the HTTP server at gaia.cs.umass.edu and exchange HTTP messages with the server in order to download this page, as discussed in section 2.2 of the text. The Ethernet frames containing these HTTP messages (as well as all other frames passing through your Ethernet adapter) will be captured by Wireshark.

7. After your browser has displayed the [INTRO-wireshark-file1.html](http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html) page (it is a simple one line of congratulations), stop Wireshark packet capture by selecting stop in the Wireshark capture window. The main Wireshark window should now look similar to Figure 3. You now have live packet data that contains all protocol messages exchanged between your computer and other network entities! The HTTP message exchanges with the gaia.cs.umass.edu web server should appear somewhere in the listing of packets captured. But there will be many other types of packets displayed as well (see, e.g., the many different protocol types shown in the Protocol column in Figure 3). Even though the only action you took was to download a web page, there were evidently many other protocols running on your computer that are unseen by the user. We'll learn much more about these protocols as we progress through the text! For now, you should just be aware that there is often much more going on than "meet's the eye"!

8. Type in “http” (without the quotes, and in lower case – all protocol names are in lower case in Wireshark) into the display filter specification window at the top of the main Wireshark window. Then select Apply (to the right of where you entered “http”). This will cause only HTTP message to be displayed in the packet-listing window.

9. Find the HTTP GET message that was sent from your computer to the gaia.cs.umass.edu HTTP server. (Look for an HTTP GET message in the “listing of captured packets” portion of the Wireshark window (see Figure 3) that shows “GET” followed by the gaia.cs.umass.edu URL that you entered. When you select the HTTP GET message, the Ethernet frame, IP datagram, TCP segment, and HTTP message header information will be displayed in the packet-header window 1 . By clicking on ‘+’ and ‘-’ right-pointing and down-pointing arrowheads to the left side of the packet details window, minimize the amount of Frame, Ethernet, Internet Protocol, and Transmission Control Protocol information displayed. Maximize the amount information displayed about the HTTP protocol. Your Wireshark display should now look roughly (Note, in particular, the minimized amount of protocol information for all protocols except HTTP, and the maximized amount of protocol information for HTTP in the packet-header window).

The goal of this first lab was primarily to introduce you to Wireshark. The following questions will demonstrate that you’ve been able to get Wireshark up and running, and have explored some of its capabilities. Answer the following questions, based on your Wireshark experimentation:

1. List 3 different protocols that appear in the protocol column in the unfiltered packet- listing window in step 7 above.
2. How long did it take from when the HTTP GET message was sent until the HTTP OK reply was received? (By default, the value of the Time column in the packet-listing window is the amount of time, in seconds, since Wireshark tracing began. To display the Time field in time-of-day format, select the Wireshark View pull down menu, then select Time Display Format, then select Time-of-day.)
3. What is the Internet address of the gaia.cs.umass.edu (also known as www- net.cs.umass.edu)? What is the Internet address of your computer?
4. Print the two HTTP messages (GET and OK) referred to in question 2 above. To do so, select Print from the Wireshark File command menu, and select the “Selected Packet Only” and “Print as displayed” radial buttons, and then click OK.

Solution:

Ans1: The following protocols appeared in the protocol column in the unfiltered packet listing window after downloading a webpage: TCP, UDP, HTTP, DNS.

Ans2: If we look at the frame section of the GET request we see that the time the packet arrived is 11:43:13.422848000

Frame 109(492 bytes on wire, 492 bytes captured) Arrival Time: sep 17, 2004 11:43:13.42284800

Time delta from previous packet: 6.826032000 seconds Time since reference or first frame: 9.263432000 seconds Frame Number: 109

Packet Length: 492 bytes Capture Length: 492 bytes

The same section for the HTTP OK shows an arrival time of 11:43:13.43960400 Frame 110(444 bytes on wire, 444 bytes captured)

Arrival Time: sep 17, 2004 11:43:13.439604000

Time delta from previous packet: 0.016756000 seconds Time since reference or first frame: 9.280188000 seconds Frame Number: 110

Packet Length: 444 bytes Capture Length: 444 bytes

The difference of these 2 times gives $.43960400 - .426032000 = 0.013572$ seconds

Ans3: If we look at the IP section of the GET request, the source and destination Source: (128.238.244.28 (128.238.244.28)

Destination: 128.119.245.12 (128.119.245.12

The source is the local machine's address and the destination is the web server's public My (local machine's) address = 128.238.244.28

IP address 128.119.245.12 = www-net.cs.umass.edu.

```

HTTP GET:
Frame 4 (862 bytes on wire, 862 bytes captured)
Ethernet II, Src: Netgear_61:8e:6d (00:09:5b:61:8e:6d), Dst: WestellT_9f:92:b9
(00:0f:db:9f:92:b9)
Internet Protocol, Src: 192.168.1.46 (192.168.1.46), Dst: 128.119.245.12
(128.119.245.12)
Transmission Control Protocol, Src Port: 1474 (1474), Dst Port: http (80), Seq: 1,
Ack: 1, Len: 808
Hypertext Transfer Protocol
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
Host: gaia.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.8.1.4)
Gecko/20070515 Firefox/2.0.0.4\r\n
Accept:
text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,im
age/png,*/*;q=0.5\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
Keep-Alive: 300\r\n
Connection: keep-alive\r\n

HTTP OK:
Frame 6 (439 bytes on wire, 439 bytes captured)
Ethernet II, Src: WestellT_9f:92:b9 (00:0f:db:9f:92:b9), Dst: Netgear_61:8e:6d
(00:09:5b:61:8e:6d)
Internet Protocol, Src: 128.119.245.12 (128.119.245.12), Dst: 192.168.1.46
(192.168.1.46)
Transmission Control Protocol, Src Port: http (80), Dst Port: 1474 (1474), Seq: 1,
Ack: 809, Len: 385
Hypertext Transfer Protocol
HTTP/1.1 200 OK\r\n
Date: Thu, 07 Jun 2007 18:09:01 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Thu, 07 Jun 2007 18:08:01 GMT\r\n
ETag: "d6c69-50-cb94a240"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 80
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n

```

3. Graded Lab Tasks

Lab Task

Start a new capture, and do some arbitrary web browsing, such as visiting some Wikipedia articles, reading some news, etc. Note down all the protocols used at each layer. You have to show this task at the end of lab.

Lab 02

Introduction to Packet Tracer

Objective:

The objective of this lab is that the students get acquainted with Packet Tracer and make some simple Packet Tracer scenarios.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Students will have gained the basic understanding of Packet Tracer to see “protocols in action”
- After this lab, students will have developed basic understanding of digging deep into the network protocols.

Instructor Note:

Packet Tracer is a cross-platform visual simulation tool designed by Cisco systems that allows users to create network topologies and imitate modern computer networks. The software allows users to simulate the configuration of Cisco routers and switches using a simulated command line interface. Packet Tracer makes use of a drag and drop user interface, allowing users to add and remove simulated network devices as they see fit. In this first Packet Tracer lab, you'll get acquainted with Packet Tracer, and make some simple configurations. For more details, please check the below reference.

Reference:

- <https://www.netacad.com/courses/packet-tracer>
- <https://ipcisco.com/course/cisco-hands-on-course-with-packet-tracer/>

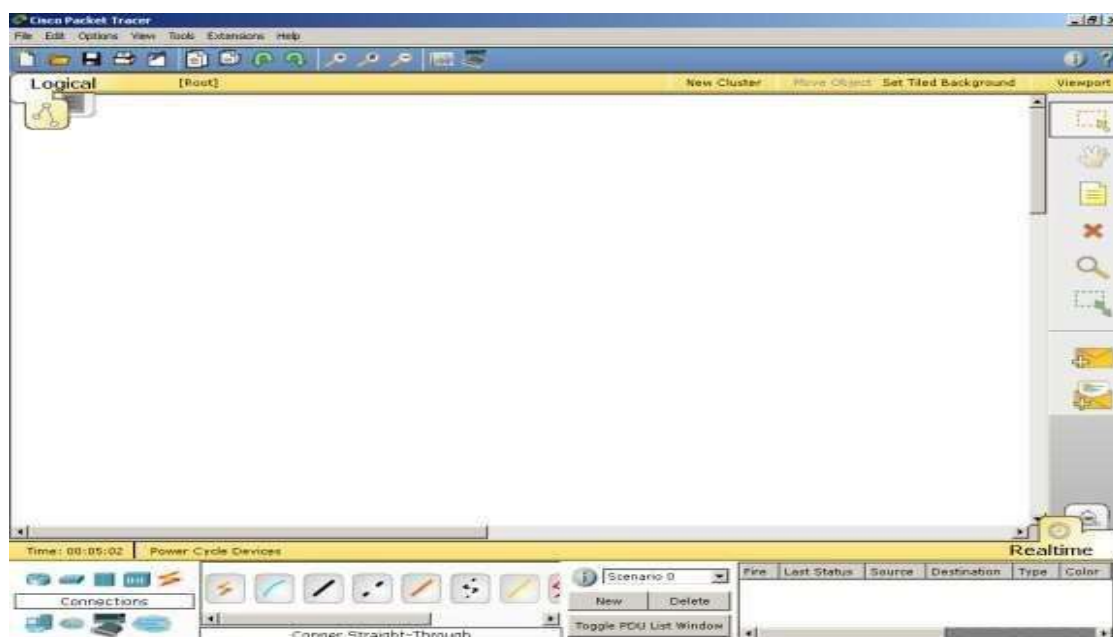
1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>25</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>30</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1

Introduction to the Packet Tracer Interface using a Hub Topology

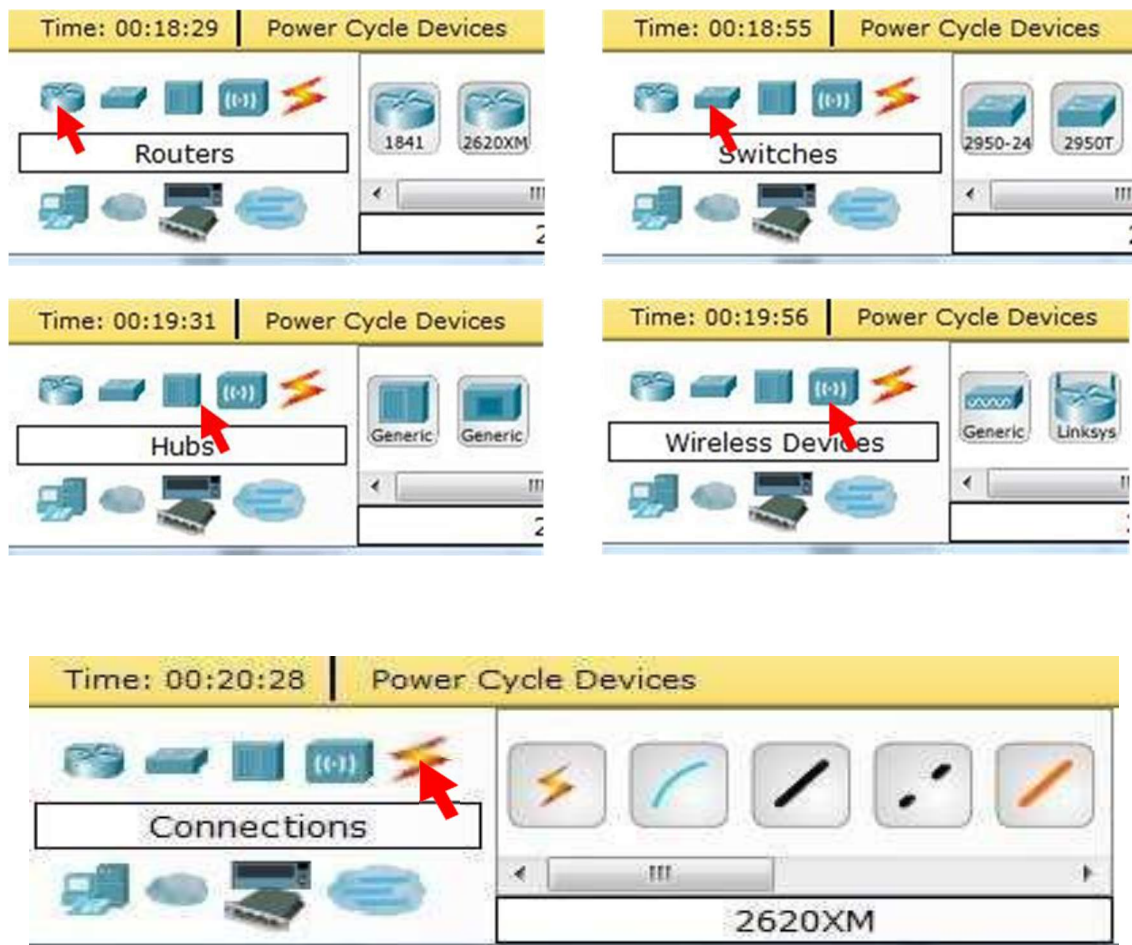
Step 1: Start Packet Tracer



Step 2: Choosing Devices and Connections

We will begin building our network topology by selecting devices and the media in which to connect them. Several types of devices and network connections can be used. For this lab we will keep it simple by using End Devices, Switches, Hubs, and Connections.

Single click on each group of devices and connections to display the various choices. The devices you see may differ slightly.





1. Other than generic routers, name 3 router models available on the simulation software.
2. What are the two types of serial cables available for WAN connectivity?
3. What are the two types of copper cable connectors?
4. Other than generic end devices, enumerate four end devices available.

Step 3: Building the Topology – Adding Hosts Single click on the End Devices.

Single click on the Generic host.

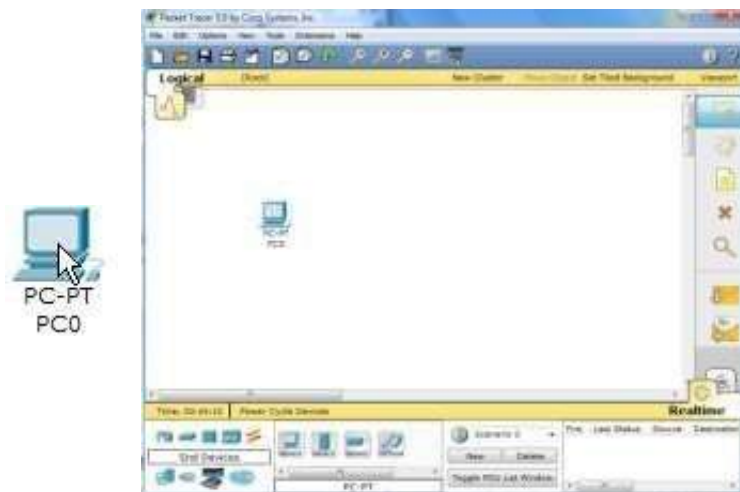




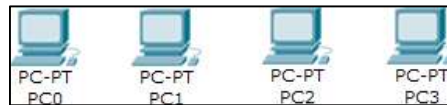
Move the cursor into topology area. You will notice it turns into a plus “+” sign.

+

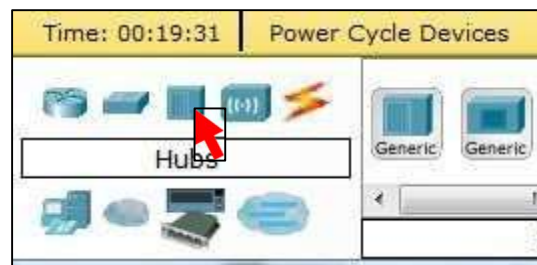
Single click in the topology area and it copies the device.



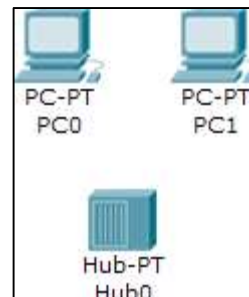
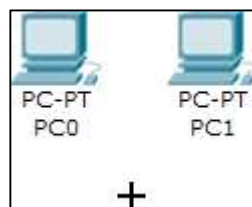
Add three more hosts.



Step 4: Building the Topology – Connecting the Hosts to Hubs and Switches Adding a Hub
Select a hub, by clicking once on Hubs and once on a Generic hub.



Add the hub by moving the plus sign “+” below PC0 and PC1 and click once.



Connect PC0 to Hub0 by first choosing Connections.

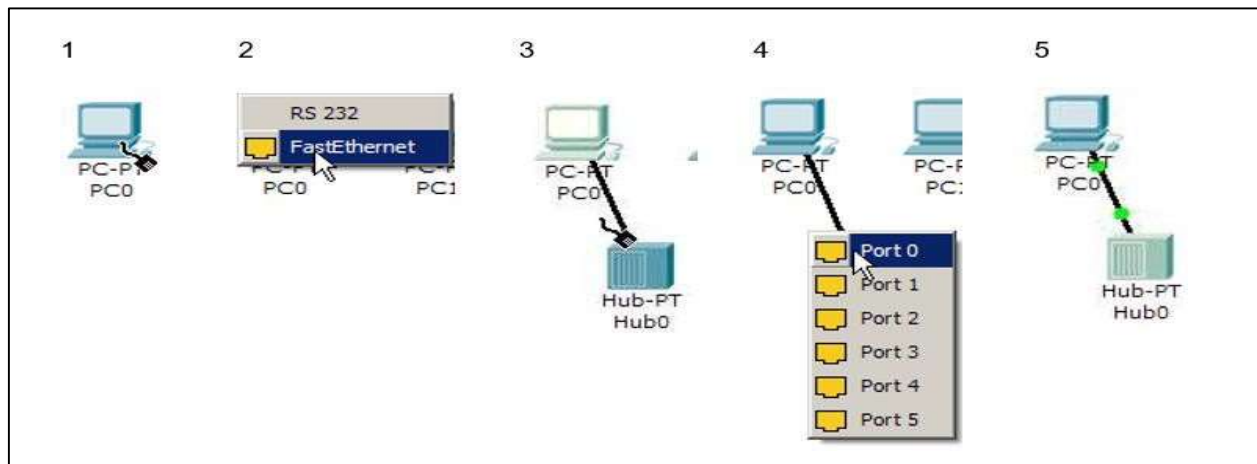


Click once on the Copper Straight-through cable.



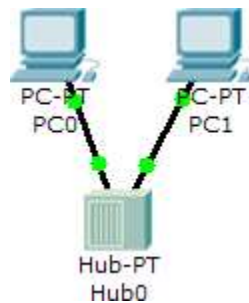
Perform the following steps to connect PC0 to Hub0:

1. Click once on PC0
2. Choose FastEthernet
3. Drag the cursor to Hub0
4. Click once on Hub0 and choose Port 0
5. Notice the green link lights on both the PC0 Ethernet NIC and the Hub0 Port 0 showing that the link is active.



Repeat the steps above for PC1 connecting it to Port 1 on Hub0. (The actual hub port you choose does not matter.)

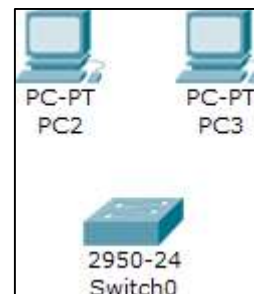
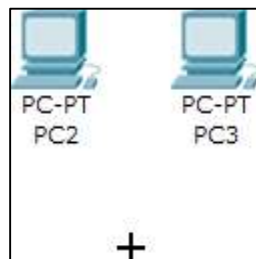
Adding a Switch



Select a switch, by clicking once on Switches and once on a 2950-24 switch.



Add the switch by moving the plus sign “+” below PC2 and PC3 and click once.



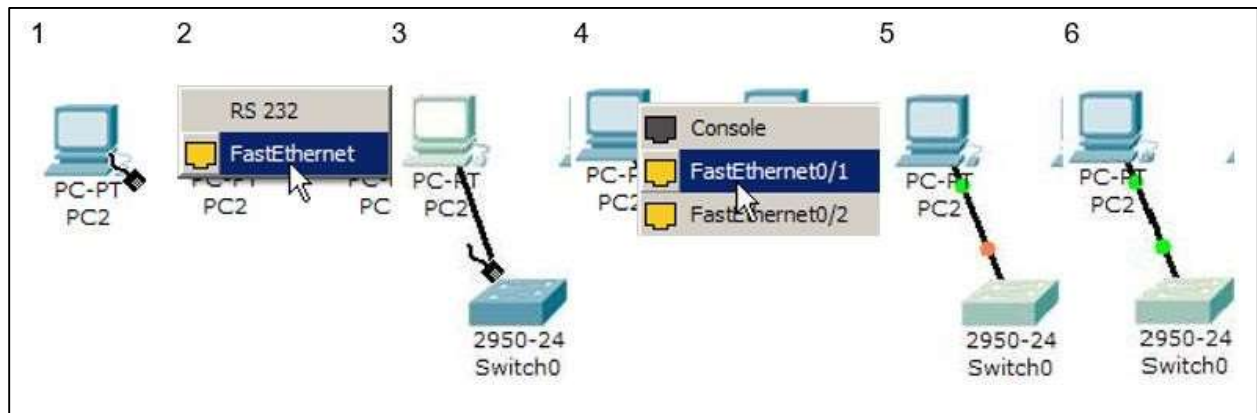
Connect PC2 to Hub0 by first choosing Connections.

Click once on the Copper Straight-through cable.

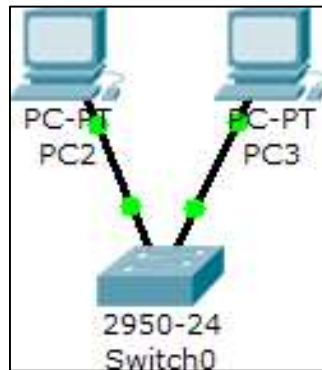
Perform the following steps to connect PC2 to Switch0

1. Click once on PC2
2. Choose FastEthernet
3. Drag the cursor to Switch0

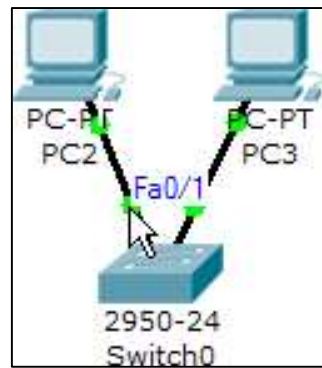
4. Click once on Switch0 and choose FastEthernet0/1
5. Notice the green link lights on PC2 Ethernet NIC and amber light Switch0 FastEthernet0/1 port. The switch port is temporarily not forwarding frames, while it goes through the stages for the Spanning Tree Protocol (STP) process.
6. After a about 30 seconds the amber light will change to green indicating that the port has entered the forwarding stage. Frames can now forwarded out the switch port.



Repeat the steps above for PC3 connecting it to Port 3 on Switch0 on port FastEthernet0/2. (The actual switch port you choose does not matter.)



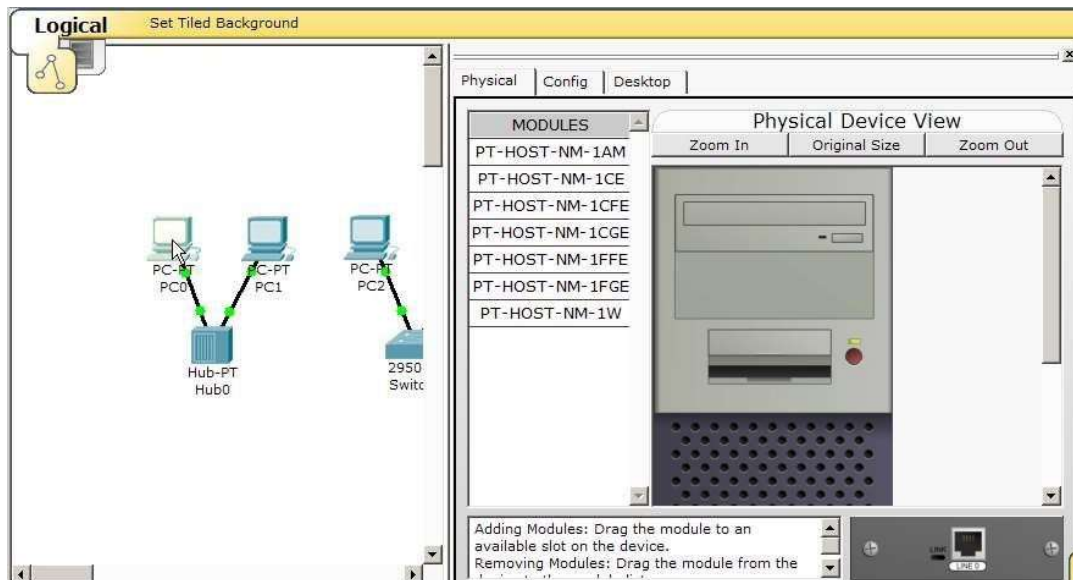
Move the cursor over the link light to view the port number. Fa means FastEthernet, 100 Mbps Ethernet.



Step 5: Configuring IP Addresses and Subnet Masks on the Hosts

Before we can communicate between the hosts we need to configure IP Addresses and Subnet Masks on the devices.

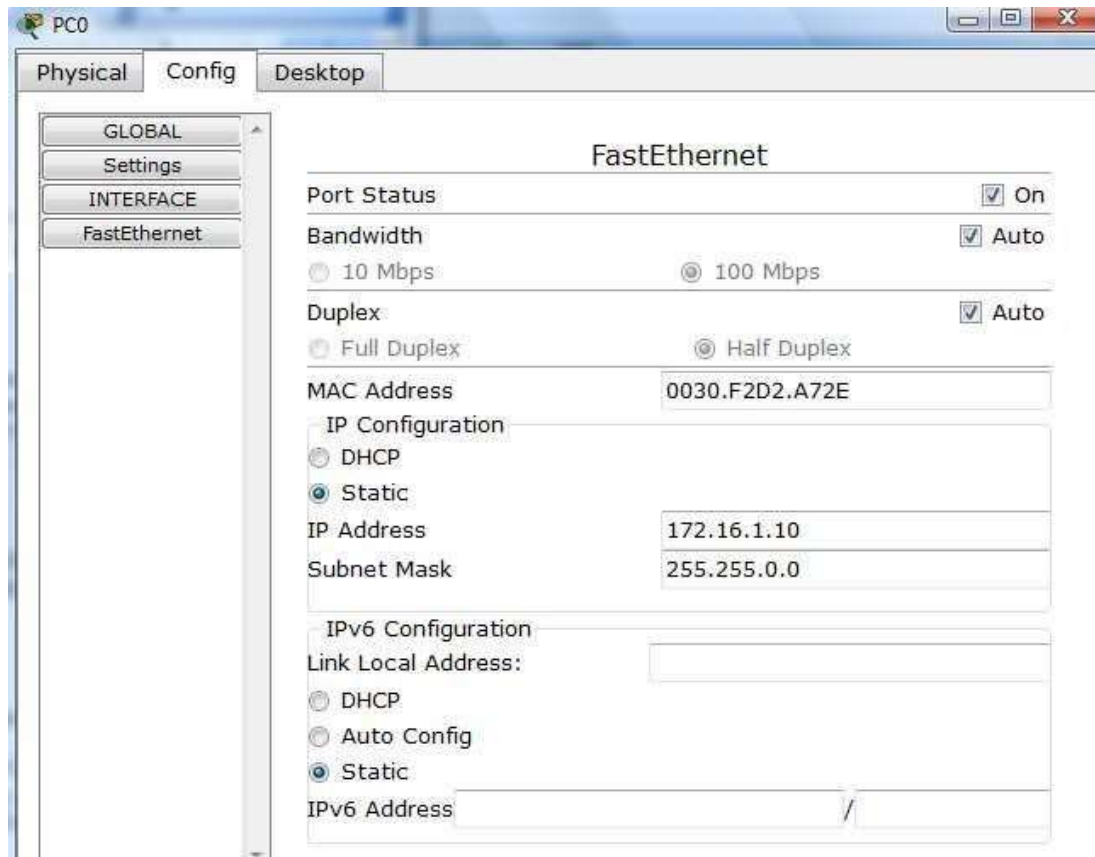
Click once on PC0.



Choose the Config tab and click on Settings. It is here that you can change the name of PC0. It is also here where you would enter a Gateway IP Address, also known as the default gateway and the DNS Server IP Address. We will discuss this later, but this would be the IP address of the local router. If you want, you can enter the Gateway IP Address 172.16.1.1 and DNS Server IP Address 172.16.1.100, although it will not be used in this lab.



Click on Interface and then FastEthernet. Although we have not yet discussed IP Addresses, add the IP Address to 172.16.1.10. Click once in the Subnet Mask field to enter the default Subnet Mask. You can leave this at 255.255.0.0.



Also, notice this is where you can change the Bandwidth (speed) and Duplex of the Ethernet NIC (Network Interface Card). The default is Auto (autonegotiation), which means the NIC will negotiate with the hub or switch. The bandwidth and/or duplex can be manually set by removing the check from the Auto box and choosing the specific option.

Bandwidth - Auto

If the host is connected to a hub or switch port which can do 100 Mbps, then the Ethernet NIC on the host will choose 100 Mbps (Fast Ethernet). Otherwise, if the hub or switch port can only do 10 Mbps, then the Ethernet NIC on the host will choose 10 Mbps (Ethernet).

Duplex - Auto

Hub: If the host is connected to a hub, then the Ethernet NIC on the host will choose Half Duplex.

Switch: If the host is connected to a switch, and the switch port is configured as Full Duplex (or Autonegotiation), then the Ethernet NIC on the host will choose Full Duplex. If the switch port is configured as Half Duplex, then the Ethernet NIC on the host will choose Half Duplex. (Full Duplex is a much more efficient option.)

The information is automatically saved when entered. To close this dialog box, click the “X” in the upper right.

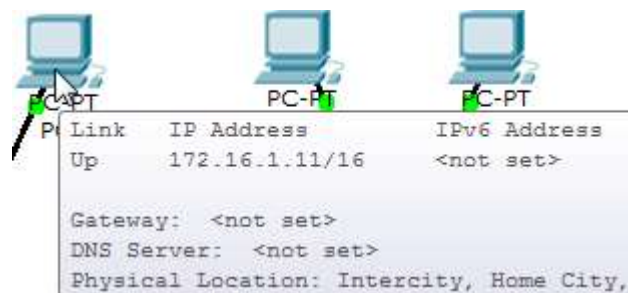


Repeat these steps for the other hosts. Use the information below for IP Addresses and Subnet Masks.

Host	IP Address	Subnet Mask
PC0	172.16.1.10	255.255.0.0
PC1	172.16.1.11	255.255.0.0
PC2	172.16.1.12	255.255.0.0
PC3	172.16.1.13	255.255.0.0

Verify the information

To verify the information that you entered, move the Select tool (arrow) over each host.



Deleting a Device or Link

To delete a device or link, choose the Delete tool and click on the item you wish to delete.

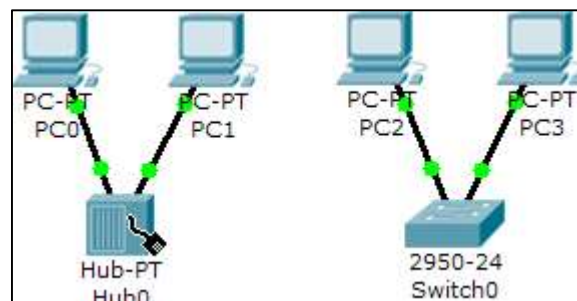


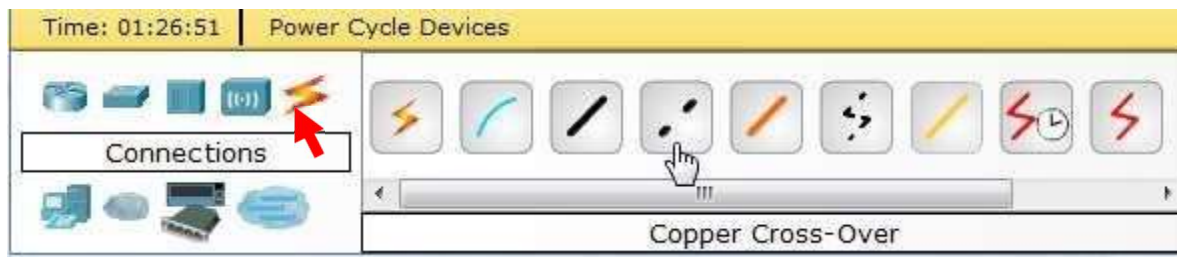
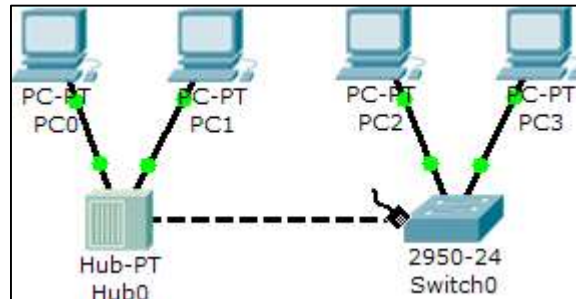
Step 6: Connecting Hub0 to Switch0

To connect like-devices, like a Hub and a Switch, we will use a Cross-over cable. Click once the Cross- over Cable from the Connections options.

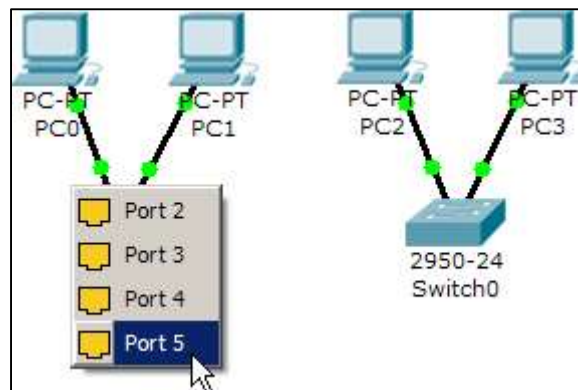


Move the Connections cursor over Hub0 and click once.



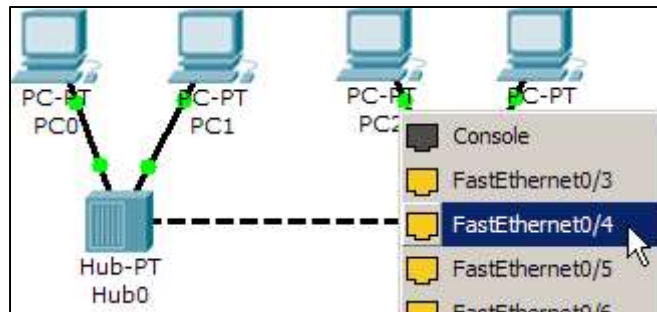


Select Port 5 (actual port does not matter).

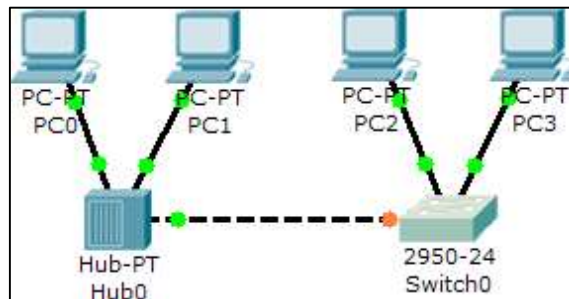


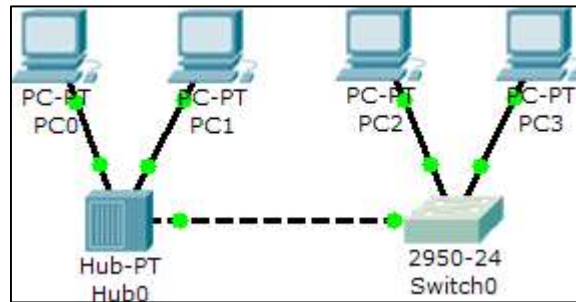
Move the Connections cursor to Switch0.

Click once on Switch0 and choose FastEthernet0/4 (actual port does not matter).



The link light for switch port FastEthernet0/4 will begin as amber and eventually change to green as the Spanning Tree Protocol transitions the port to forwarding.



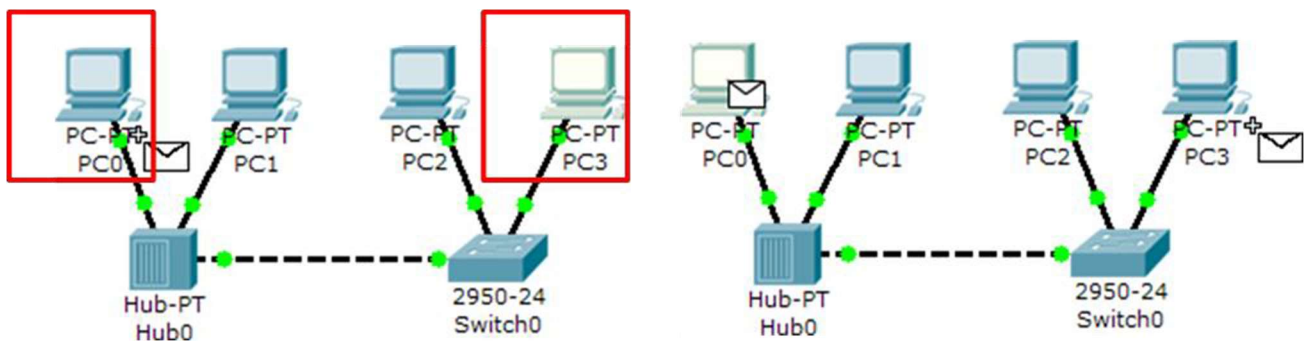


Step 7: Verifying Connectivity in Realtime Mode Be sure you are in Realtime mode.

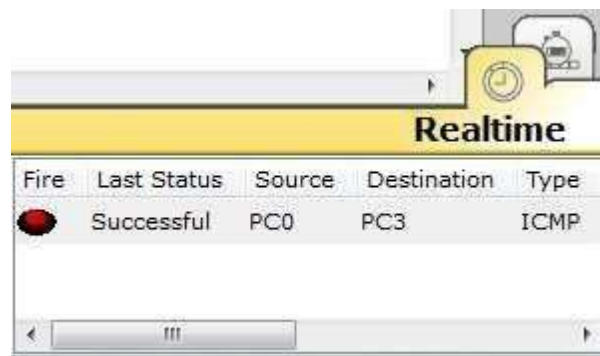
Select the Add Simple PDU tool used to ping devices.



Click once on PC0, then once on PC3.



The PDU Last Status should show as Successful.



The image shows a 'Realtime' window from a network simulation. It contains a table with the following data:

Fire	Last Status	Source	Destination	Type
	Successful	PC0	PC3	ICMP

Change the IP address of PC3 to 172.16.2.13. Perform a ping from PC0 to PC3. What is the ping result?

Return the IP address of PC3 to 172.16.1.13. Change the IP address of PC2 to 172.17.1.12. Perform a ping from PC0 to PC2. What is the ping result?

Resetting the Network

At this point we will want to reset the network, whenever you want to reset the network and begin the simulation again, perform the following tasks:

Click Delete in the PDU area.



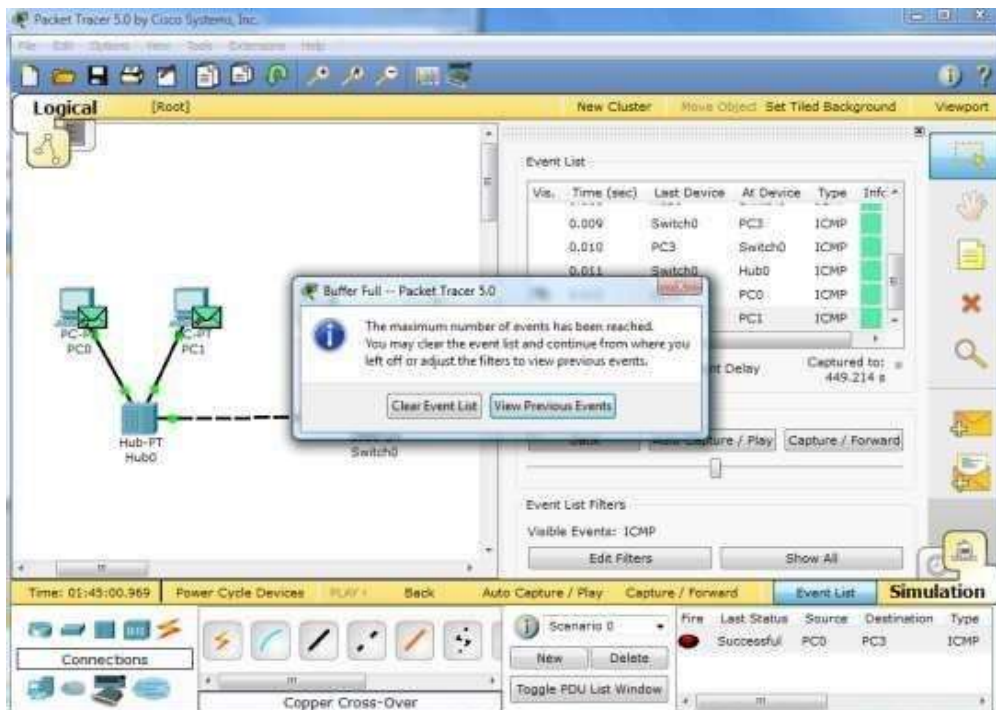
Now, Power Cycle Devices and confirm the action.

Waiting for Spanning Tree Protocol (STP)

Note: Because Packet Tracer also simulates the Spanning Tree Protocol, at times the switch may show amber lights on its interfaces. You will need to wait for the lights to turn green on the switches before they will forward any Ethernet frames.

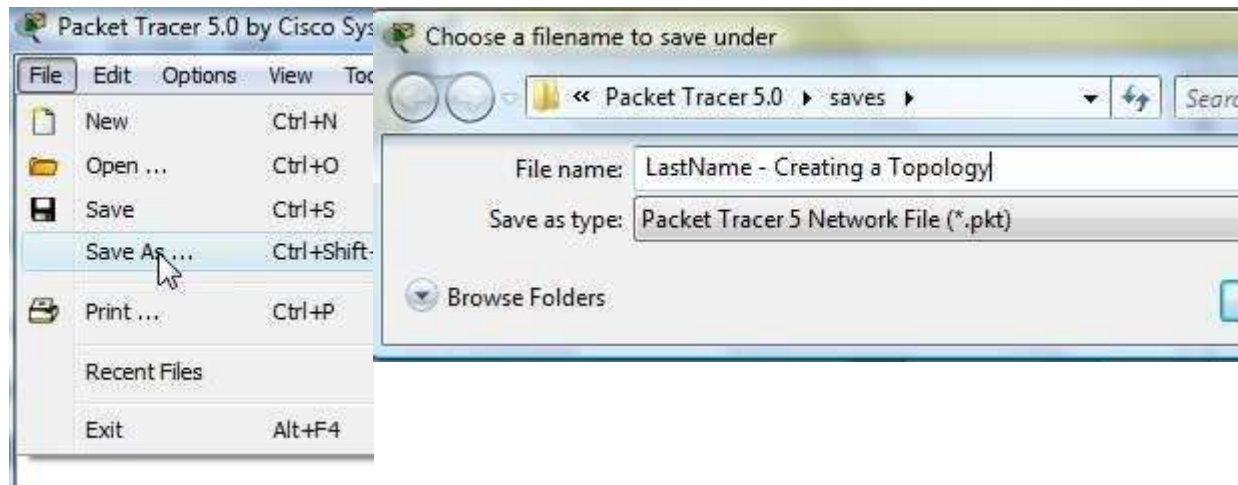
Step 8: Verifying Connectivity in Simulation Mode

- Be sure you are in Simulation mode.
- Deselect all filters (All/None) and select only ICMP.
- Select the Add Simple PDU tool used to ping devices..
- Click once on PC0, then once on PC3.
- Continue clicking Capture/Forward button until the ICMP ping is completed. You should see the ICMP messages move between the hosts, hub and switch. The PDU Last Status should show as Successful. Click on Clear Event List if you do not want to look at the events or click Preview Previous Events if you do. For this exercise it does not matter.



Step 9: Saving the Topology

- Perform the following steps to save the topology (uses .pkt file extension).



- Opening Existing Topologies
- Opening Existing PT Topologies

3. Graded Lab Tasks

Lab Tasks

Make topologies in packet tracer and provide connectivity:

- 1) Point-to-point
- 2) Bus
 - a. Linear bus
 - b. Distributed bus
- 3) Star
 - a. Extended star

-
- b. Distributed Star
 - 4) Ring
 - 5) Mesh
 - a. Fully connected network
 - b. Partially connected network

Lab 03

Hyper Text Transfer Protocol(HTTP) using WireShark

Objective:

The objective of this lab is that the students will explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, retrieving HTML files with embedded objects, and HTTP authentication and security.

Activity Outcomes:

Students will gain better understanding of the HTTP protocol.

Instructor Note:

The HyperText Transfer Protocol (HTTP), the Web's application-layer protocol, is at the heart of the Web. It is defined in [RFC 1945] and [RFC 2616]. HTTP is implemented in two programs: a client program and a server program. The client program and server program, executing on different end systems, talk to each other by exchanging HTTP messages. HTTP defines the structure of these messages and how the client and server exchange the messages.

In this Wireshark lab, you'll capture some http packets using wireshark and make some observations on them. For more details, please check the below reference.

References:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- Wireshark Labs, https://gaia.cs.umass.edu/kurose_ross/wireshark.php
- Wireshark User Guide: <https://www.wireshark.org/download/docs/user-guide.pdf>
- Wireshark Walkthrough: <https://www.youtube.com/watch?v=TkCSr30UojM>
- Wireshark Website: <https://www.wireshark.org>

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>20</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>20</i>	<i>Low</i>	<i>CLO-6</i>
<i>3</i>	<i>15</i>	<i>Medium</i>	<i>CLO-6</i>
<i>4</i>	<i>15</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1:

Having gotten our feet wet with the Wireshark packet sniffer in the introductory lab, we're now ready to use Wireshark to investigate protocols in operation. In this lab, we'll explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, retrieving HTML files with embedded objects, and HTTP authentication and security. Before beginning these labs, you might want to review Section 2.2 of the text.¹

1. The Basic HTTP GET/response interaction

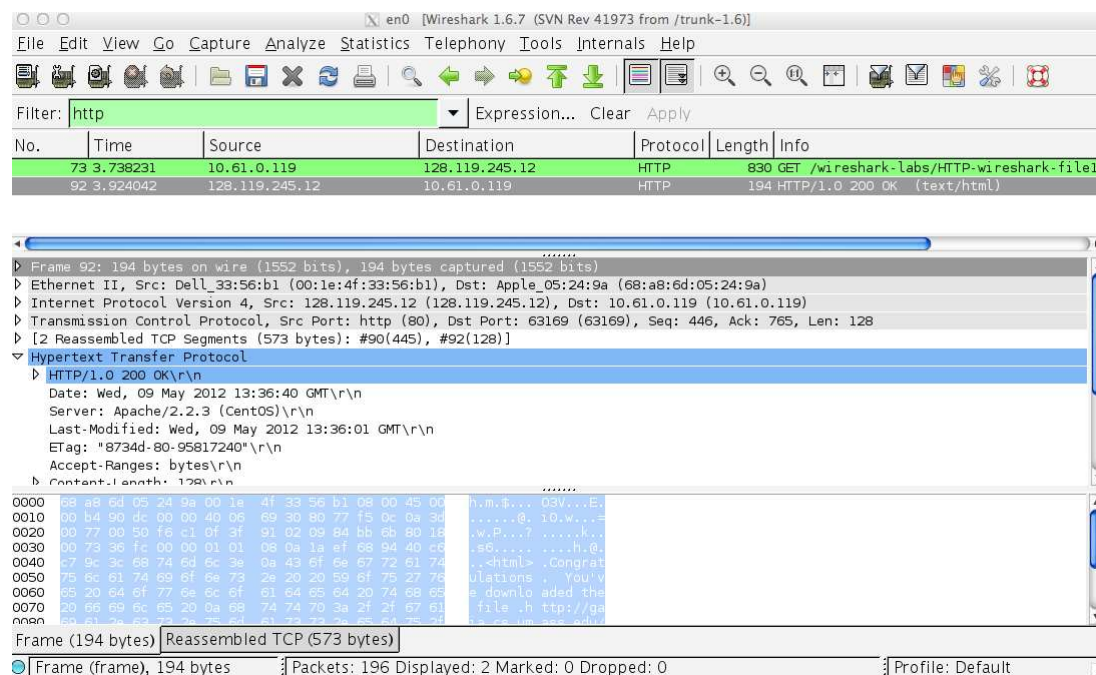
Let's begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

1. Start up your web browser.
2. Start up the Wireshark packet sniffer, as described in the Introductory lab (but don't yet begin packet capture). Enter "http" (just the letters, not the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We're only interested in the HTTP protocol here, and don't want to see the clutter of all captured packets).
3. Wait a bit more than one minute (we'll see why shortly), and then begin Wireshark packet capture.

¹ References to figures and sections are for the 7th edition of our text, *Computer Networks, A Top-down Approach*, 7th ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2016.

4. Enter the following to your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html>
Your browser should display the very simple, one-line HTML file.
5. Stop Wireshark packet capture.

Your Wireshark window should look similar to the window shown in Figure 1. If you are unable to run Wireshark on a live network connection, you can download a packet trace that was created when the steps above were followed.²



Download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip> and extract the file http-ethereal-trace-1. The traces in this zip file were collected by Wireshark running on one of the author's computers, while performing the steps indicated in the Wireshark lab. Once you have downloaded the trace, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the http-ethereal-trace-1 trace file. The resulting display should look similar to Figure 1. (The Wireshark user interface displays just a bit differently on different operating systems, and in different versions of Wireshark).

The example in Figure 1 shows in the packet-listing window that two HTTP messages were captured: the GET message (from your browser to the `gaia.cs.umass.edu` web server) and the response message from the server to your browser. The packet-contents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we're interested in HTTP here, and will be investigating these other protocols in later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, undisplayed information), and the HTTP line has a minus sign or a down-pointing triangle (which means that all information about the HTTP message is displayed).

(Note: You should ignore any HTTP GET and response for `favicon.ico`. If you see a reference to this file, it is your browser automatically asking the server if it (the server) has a small icon file that should be displayed next to the displayed URL in your browser. We'll ignore references to this pesky file in this lab.)

By looking at the information in the HTTP GET and response messages, answer the following questions. When answering the following questions, you should print out the GET and response messages (see the introductory Wireshark lab for an explanation of how to do this) and indicate where in the message you've found the information that answers the following questions. When you hand in your assignment, annotate the output so that it's clear where in the output you're getting the information for your answer (e.g., for our classes, we ask that students markup paper copies with a pen, or annotate electronic copies with text in a colored font).

1. Is your browser running HTTP version 1.0 or 1.1? What version of HTTP is the server running?
2. What languages (if any) does your browser indicate that it can accept to the server?
3. What is the IP address of your computer? Of the `gaia.cs.umass.edu` server?
4. What is the status code returned from the server to your browser?
5. When was the HTML file that you are retrieving last modified at the server?
6. How many bytes of content are being returned to your browser?

-
7. By inspecting the raw data in the packet content window, do you see any headers within the data that are not displayed in the packet-listing window? If so, name one.

In your answer to question 5 above, you might have been surprised to find that the document you just retrieved was last modified within a minute before you downloaded the document. That's because (for this particular file), the gaia.cs.umass.edu server is setting the file's last-modified time to be the current time, and is doing so once per minute. Thus, if you wait a minute between accesses, the file will appear to have been recently modified, and hence your browser will download a "new" copy of the document.

Solution:

Ans1:Both are running HTTP 1.1

Ans2:Accept-Language: en-us, en

Ans3: My IP address is 192.168.1.46 and the server's is 128.119.245.12

Ans4: HTTP/1.1 200 OK (text/html)

Ans5: Last-Modified: Thu, 07 Jun 2007 22:09:01 GMT

Ans6: Content-Length: 126

Ans7: No all of the headers can be found in the raw data.

Activity 2:

The HTTP CONDITIONAL GET/response interaction

Recall from Section 2.2.5 of the text, that most web browsers perform object caching and thus perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty. (To do this under Firefox, select *Tools->Clear Recent*

History and check the Cache box, or for Internet Explorer, select *Tools->Internet Options->Delete File*; these actions will remove cached files from your browser's cache.) Now do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>
Your browser should display a very simple five-line HTML file.
- Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.
- (*Note:* If you are unable to run Wireshark on a live network connection, you can use the http-ethereal-trace-2 packet trace to answer the questions below; see footnote 1. This trace file was gathered while performing the steps above on one of the author's computers.)

Answer the following questions:

8. Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE" line in the HTTP GET?
9. Inspect the contents of the server response. Did the server explicitly return the contents of the file? How can you tell?
10. Now inspect the contents of the second HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE:" line in the HTTP GET? If so, what information follows the "IF-MODIFIED-SINCE:" header?
11. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Explain.

Solution:

Ans8: No.

Ans9: Yes because we can see the contents in the Line-based text data field

Ans10: Yes. The information following is: Thu, 07 Jun 2007 16:29:01 GMT which is the date of the last modification of the file from the previous get request.

Ans11: The status code and phrase returned from the server is HTTP/1.1 304 Not Modified. The server didn't return the contents of the file since the browser loaded it from its cache.

Activity 3

Retrieving Long Documents

In our examples thus far, the documents retrieved have been simple and short HTML files. Let's next see what happens when we download a long HTML file. Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html>
Your browser should display the rather lengthy US Bill of Rights.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.
- (*Note:* If you are unable to run Wireshark on a live network connection, you can use the http-ethereal-trace-3 packet trace to answer the questions below; see footnote 1. This trace file was gathered while performing the steps above on one of the author's computers.)

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. This multiple-packet response deserves a bit of explanation. Recall from Section 2.2 (see Figure 2.9 in the text) that the HTTP response message consists of a status line, followed by header lines, followed by a blank line, followed by the entity body. In the case of our HTTP GET, the entity body in the response is the *entire* requested HTML file. In our case here, the HTML file is rather long, and at 4500 bytes is too large to fit in one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment (see Figure 1.24 in the text). In recent versions of Wireshark, Wireshark indicates each TCP segment as a separate packet, and the fact that the single HTTP response was fragmented across multiple TCP packets is indicated by the "TCP segment of a reassembled PDU" in the Info column of the Wireshark display. Earlier versions of Wireshark used the "Continuation" phrase to indicate that the entire content of an HTTP message was broken across multiple TCP segments.. We stress here that there is no "Continuation" message in HTTP!

Answer the following questions:

-
12. How many HTTP GET request messages did your browser send? Which packet number in the trace contains the GET message for the Bill of Rights?
 13. Which packet number in the trace contains the status code and phrase associated with the response to the HTTP GET request?
 14. What is the status code and phrase in the response?
 15. How many data-containing TCP segments were needed to carry the single HTTP response and the text of the Bill of Rights?

Solution:

Ans12: There was 1 HTTP GET request message sent by my browser

Ans13: There were 5 data containing TCP segments containing 309 ,1452 ,1452, 1452 and 144 bytes respectively for a total of 4500 bytes.

Ans14: 200 OK

Activity 4:

HTML Documents with Embedded Objects

Now that we've seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html>

Your browser should display a short HTML file with two images. These two images are referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file. As

discussed in the textbook, your browser will have to retrieve these logos from the indicated web sites. Our publisher's logo is retrieved from the `gaia.cs.umass.edu` web site. The image of the cover for our 5th edition (one of our favorite covers) is stored at the `caite.cs.umass.edu` server. (These are two different web servers inside `cs.umass.edu`).

- Stop Wireshark packet capture, and enter “http” in the display-filter-specification window, so that only captured HTTP messages will be displayed.
- (*Note:* If you are unable to run Wireshark on a live network connection, you can use the `http-ethereal-trace-4` packet trace to answer the questions below; see footnote 1. This trace file was gathered while performing the steps above on one of the author's computers.)

Answer the following questions:

16. How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent?
17. Can you tell whether your browser downloaded the two images serially, or whether they were downloaded from the two web sites in parallel? Explain.

Solution:

Ans15: There were 3 HTTP GET requests sent to the following Internet addresses: a. 128.119.245.12 b. 128.119.240.90 c. 165.193.123.218

Ans16: By checking the TCP ports we can see if our files were downloaded serially or in parallel. In this case the 2 images were transmitted over 2 TCP connections therefore they were downloaded serially.

3. Graded Lab Tasks

Finally, let's try visiting a web site that is password-protected and examine the sequence of HTTP message exchanged for such a site. The URL

`http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html` is password protected. The username is “wireshark-students” (without the quotes), and the password is “network” (again, without the quotes). So let's access this “secure” password-protected site. Do the following:

- Make sure your browser's cache is cleared, as discussed above, and close down your browser. Then, start up your browser

-
- Start up the Wireshark packet sniffer
 - Enter the following URL into your browser
http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html
Type the requested user name and password into the pop up box.
 - Stop Wireshark packet capture, and enter “http” in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.
 - (*Note:* If you are unable to run Wireshark on a live network connection, you can use the http-ethereal-trace-5 packet trace to answer the questions below; see footnote 2. This trace file was gathered while performing the steps above on one of the author’s computers.)

Now let’s examine the Wireshark output. You might want to first read up on HTTP authentication by reviewing the easy-to-read material on “HTTP Access Authentication Framework” at [http://frontier.userland.com/stories/storyReader\\$2159](http://frontier.userland.com/stories/storyReader$2159)

Answer the following questions:

18. What is the server’s response (status code and phrase) in response to the initial HTTP GET message from your browser?
19. When your browser’s sends the HTTP GET message for the second time, what new field is included in the HTTP GET message?

Lab 04

Domain Name System(DNS) using WireShark

Objective:

The objective of this lab is that the students will capture some DNS packets using Wireshark and make some observations on them.

Activity Outcomes:

The activities provide hands-on practice with the following topics

- Students will have gained the basic understanding of Wireshark Packet Sniffer to see “protocols in action”
- After this lab, students will have developed basic understanding of digging deep into the network protocols.

Instructor Note:

The DNS is a distributed database implemented in a hierarchy of DNS servers. Moreover, as an application-layer protocol, it allows hosts to query the distributed database. The DNS protocol runs over UDP and uses port 53. DNS is commonly employed by other application-layer protocols—including HTTP, SMTP, and FTP—to translate user-supplied hostnames to IP addresses. As an example, consider what happens when a browser (that is, an HTTP client), running on some user's host, requests the URL `www.someschool.edu/index.html`. In order for the user's host to be able to send an HTTP request message to the Web server `www.someschool.edu`, the user's host must first obtain the IP address of `www.someschool.edu`.

For more details, please check the below reference.

References:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.

-
- Wireshark Labs, https://gaia.cs.umass.edu/kurose_ross/wireshark.php
 - [https://eclass.teicrete.gr/modules/document/file.php/TP326/%CE%98%CE%B5%CF%89%CF%81%CE%AF%CE%B1%20\(Lectures\)/Computer_Networking_A_Top-Down_Approach.pdf](https://eclass.teicrete.gr/modules/document/file.php/TP326/%CE%98%CE%B5%CF%89%CF%81%CE%AF%CE%B1%20(Lectures)/Computer_Networking_A_Top-Down_Approach.pdf)
 - Wireshark User Guide: <https://www.wireshark.org/download/docs/user-guide.pdf>
 - Wireshark Walkthrough: <https://www.youtube.com/watch?v=TkCSr30UojM>
 - Wireshark Website: <https://www.wireshark.org>

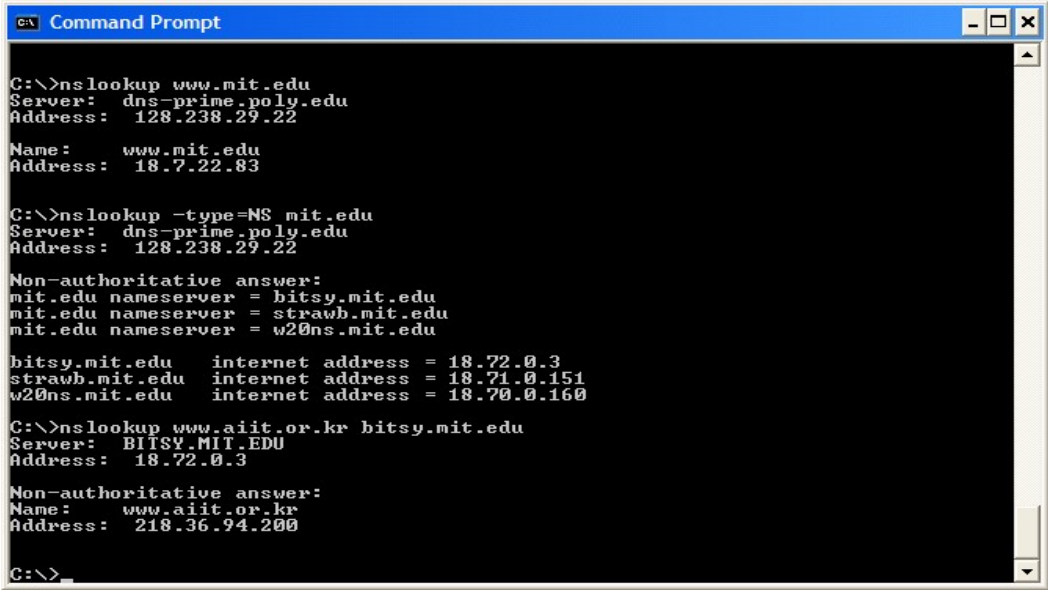
1. Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>10</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>
<i>3</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>
<i>4</i>	<i>15</i>	<i>Medium</i>	<i>CLO-6</i>
<i>5</i>	<i>15</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1:

nslookup

In this lab, we'll make extensive use of the nslookup tool, which is available in most Linux/Unix and Microsoft platforms today. To run nslookup in Linux/Unix, you just type the nslookup



```
C:\>nslookup www.mit.edu
Server: dns-prime.poly.edu
Address: 128.238.29.22

Name: www.mit.edu
Address: 18.7.22.83

C:\>nslookup -type=NS mit.edu
Server: dns-prime.poly.edu
Address: 128.238.29.22

Non-authoritative answer:
mit.edu nameserver = bitsy.mit.edu
mit.edu nameserver = strawb.mit.edu
mit.edu nameserver = w20ns.mit.edu

bitsy.mit.edu internet address = 18.72.0.3
strawb.mit.edu internet address = 18.71.0.151
w20ns.mit.edu internet address = 18.70.0.160

C:\>nslookup www.aiit.or.kr bitsy.mit.edu
Server: BITSY.MIT.EDU
Address: 18.72.0.3

Non-authoritative answer:
Name: www.aiit.or.kr
Address: 218.36.94.200

C:\>
```

command on the command line. To run it in Windows, open the Command Prompt and run nslookup on the command line.

In its most basic operation, nslookup tool allows the host running the tool to query any specified DNS server for a DNS record. The queried DNS server can be a root DNS server, a top-level-domain DNS server, an authoritative DNS server, or an intermediate DNS server (see the textbook for definitions of these terms). To accomplish this task, nslookup sends a DNS query to the specified DNS server, receives a DNS reply from that same DNS server, and displays the result.

The above screenshot shows the results of three independent nslookup commands (displayed in the Windows Command Prompt). In this example, the client host is located on the campus of Polytechnic University in Brooklyn, where the default local DNS server is dns-prime.poly.edu. When running nslookup, if no DNS server is specified, then nslookup sends the query to the default DNS server, which in this case is dns-prime.poly.edu. Consider the first command:

nslookup www.mit.edu

In words, this command is saying “please send me the IP address for the host `www.mit.edu`”. As shown in the screenshot, the response from this command provides two pieces of information: (1) the name and IP address of the DNS server that provides the answer; and (2) the answer itself, which is the host name and IP address of `www.mit.edu`. Although the response came from the local DNS server at Polytechnic University, it is quite possible that this local DNS server iteratively contacted several other DNS servers to get the answer, as described in Section 2.5 of the textbook.

Now consider the second command:

nslookup -type=NS mit.edu

In this example, we have provided the option “-type=NS” and the domain “mit.edu”. This causes nslookup to send a query for a type-NS record to the default local DNS server. In words, the query is saying, “please send me the host names of the authoritative DNS for mit.edu”. (When the -type option is not used, nslookup uses the default, which is to query for type A records.) The answer, displayed in the above screenshot, first indicates the DNS server that is providing the answer (which is the default local DNS server) along with three MIT nameservers. Each of these servers is indeed an authoritative DNS server for the hosts on the MIT campus. However, nslookup also indicates that the answer is “non-authoritative,” meaning that this answer came from the cache of some server rather than from an authoritative MIT DNS server. Finally, the answer also includes the IP addresses of the authoritative DNS servers at MIT. (Even though the type-NS query generated by nslookup did not explicitly ask for the IP addresses, the local DNS server returned these “for free” and nslookup displays the result.)

Now finally consider the third command:

nslookup www.aiit.or.kr bitsy.mit.edu

In this example, we indicate that we want to the query sent to the DNS server `bitsy.mit.edu` rather than to the default DNS server (`dns-prime.poly.edu`). Thus, the query and reply transaction takes place directly between our querying host and `bitsy.mit.edu`. In this example, the DNS server `bitsy.mit.edu` provides the IP address of the host `www.aiit.or.kr`, which is a web server at the Advanced Institute of Information Technology (in Korea).

Now that we have gone through a few illustrative examples, you are perhaps wondering about the general syntax of nslookup commands. The syntax is:

nslookup -option1 -option2 host-to-find dns-server

In general, nslookup can be run with zero, one, two or more options. And as we have seen in the above examples, the dns-server is optional as well; if it is not supplied, the query is sent to the default local DNS server.

Now that we have provided an overview of nslookup, it is time for you to test drive it yourself. Do the following (and write down the results):

6. Run nslookup to obtain the IP address of a Web server in Asia. What is the IP address of that server?
7. Run nslookup to determine the authoritative DNS servers for a university in Europe.
8. Run nslookup so that one of the DNS servers obtained in Question 2 is queried for the mail servers for Yahoo! mail. What is its IP address?

Solution:

Ans1: I performed nslookup for www.rediff.com. Its IP address is 208.184.138.70

Ans2: I performed nslookup for a European University in Ioannina Greece. Its IP address is 128.238.29.22

Ans3: the IP address of the mail server(s) is 18.72.0.3.

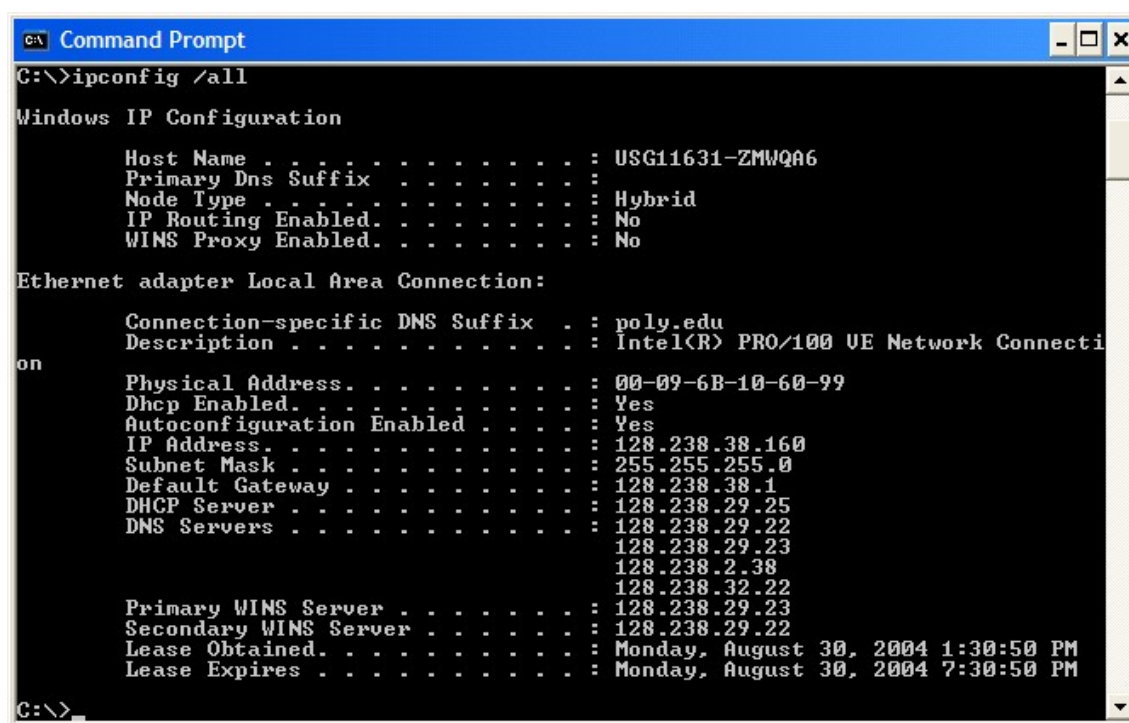
ipconfig

ipconfig (for Windows) and ifconfig (for Linux/Unix) are among the most useful little utilities in your host, especially for debugging network issues. Here we'll only describe ipconfig, although the Linux/Unix ifconfig is very similar. ipconfig can be used to show your current TCP/IP

information, including your address, DNS server addresses, adapter type and so on. For example, if you all this information about your host simply by entering

ipconfig /all

into the Command Prompt, as shown in the following screenshot.



```
C:\>ipconfig /all

Windows IP Configuration

    Host Name . . . . . : USG11631-ZMWQA6
    Primary Dns Suffix . . . . . :
    Node Type . . . . . : Hybrid
    IP Routing Enabled. . . . . : No
    WINS Proxy Enabled. . . . . : No

Ethernet adapter Local Area Connection:

    Connection-specific DNS Suffix  . : poly.edu
    Description . . . . . : Intel(R) PRO/100 VE Network Connection
    Physical Address. . . . . : 00-09-6B-10-60-99
    Dhcp Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    IP Address. . . . . : 128.238.38.160
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 128.238.38.1
    DHCP Server . . . . . : 128.238.29.25
    DNS Servers . . . . . : 128.238.29.22
                           128.238.29.23
                           128.238.2.38
                           128.238.32.22
    Primary WINS Server . . . . . : 128.238.29.23
    Secondary WINS Server . . . . . : 128.238.29.22
    Lease Obtained. . . . . : Monday, August 30, 2004 1:30:50 PM
    Lease Expires . . . . . : Monday, August 30, 2004 7:30:50 PM

C:\>
```

ipconfig is also very useful for managing the DNS information stored in your host. In Section 2.5 we learned that a host can cache DNS records it recently obtained. To see these cached records, after the prompt C:\> provide the following command:

ipconfig /displaydns

Each entry shows the remaining Time to Live (TTL) in seconds. To clear the cache, enter *ipconfig /flushdns*

Flushing the DNS cache clears all entries and reloads the entries from the hosts file

Activity 2:

Now that we are familiar with nslookup and ipconfig, we're ready to get down to some serious business. Let's first capture the DNS packets that are generated by ordinary Web-surfing activity.

- Use ipconfig to empty the DNS cache in your host.
- Open your browser and empty your browser cache. (With Internet Explorer, go to Tools menu and select Internet Options; then in the General tab select Delete Files.)
- Open Wireshark and enter "ip.addr == your_IP_address" into the filter, where you obtain your_IP_address with ipconfig. This filter removes all packets that neither originate nor are destined to your host.
- Start packet capture in Wireshark.
- With your browser, visit the Web page: <http://www.ietf.org>
- Stop packet capture.

If you are unable to run Wireshark on a live network connection, you can download a packet trace file that was captured while following the steps above on one of the author's computers³. Answer the following questions. Whenever possible, when answering a question below, you should hand in a printout of the packet(s) within the trace that you used to answer the question asked. Annotate the printout⁴ to explain your answer. To print a packet, use File->Print, choose Selected packet only, choose Packet summary line, and select the minimum amount of packet detail that you need to answer the question.

1. Locate the DNS query and response messages. Are then sent over UDP or TCP?
2. What is the destination port for the DNS query message? What is the source port of DNS response message?

-
3. To what IP address is the DNS query message sent? Use ipconfig to determine the IP address of your local DNS server. Are these two IP addresses the same?
 4. Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?
 5. Examine the DNS response message. How many “answers” are provided? What do each of these answers contain?
 6. Consider the subsequent TCP SYN packet sent by your host. Does the destination IP address of the SYN packet correspond to any of the IP addresses provided in the DNS response message?
 7. This web page contains images. Before retrieving each image, does your host issue new DNS queries?

Solution:

Ans1: They are sent over UDP

Ans2: The destination port for the DNS query is 53 and the source port of the DNS response is 53.

Ans3: It's sent to 192.168.1.1, which is the IP address of one of my local DNS servers.

Ans4: It's a type A Standard Query and it doesn't contain any answers.

Ans5: : There were 2 answers containing information about the name of the host, the type of address, class, the TTL, the data length and the IP address.

Answers www.ietf.org: type A, class IN, addr 209.173.57.180 Name:

www.ietf.org

Type: A (Host address) Class: IN (0x0001)

Time to live: 30 minutes Data length: 4

Addr: 209.173.57.180

www.ietf.org: type A, class IN, addr 209.173.53.180 Name:

www.ietf.org

Type: A (Host address) Class: IN (0x0001) Time to live: 30 minutes

Data length: 4

Addr: 209.173.53.180

Ans6: The first SYN packet was sent to 209.173.57.180 which corresponds to the first IP address provided in the DNS response message.

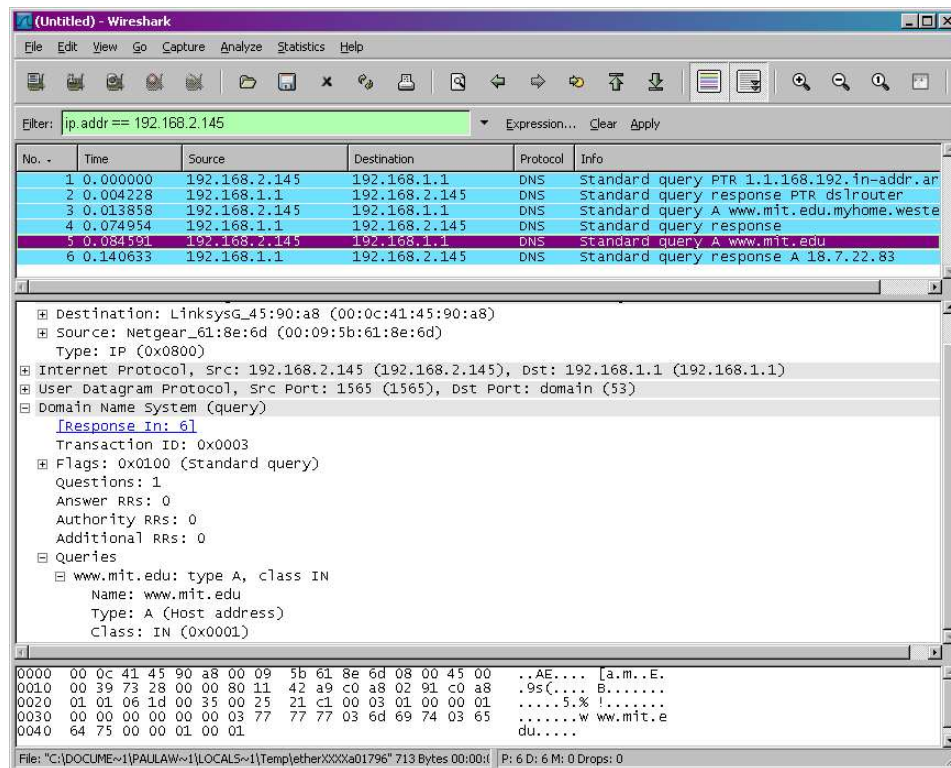
Ans7: No

Activity 3:

Now let's play with nslookup5.

- Start packet capture.
- Do an nslookup on www.mit.edu
- Stop packet capture.

You should get a trace that looks something like the following:



We see from the above screenshot that nslookup actually sent three DNS queries and received three DNS responses. For the purpose of this assignment, in answering the following questions, ignore the first two sets of queries/responses, as they are specific to nslookup and are not normally generated by standard Internet applications. You should instead focus on the last query and response messages.

1. What is the destination port for the DNS query message? What is the source port of DNS response message?
2. To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?

3. Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?

4. Examine the DNS response message. How many “answers” are provided? What do each of these answers contain?

5. Provide a screenshot.

Solution:

Ans1: The destination port of the DNS query is 53 and the source port of the DNS response is 53. Ans2: It’s sent to 192.168.1.1 which as we can see from the ipconfig –all screenshot, is the default local DNS server.

Ans3: The query is of type A and it doesn’t contain any answers.

Ans4: The response DNS message contains one answer containing the name of the host, the type of address, the class, and the IP address.

Answers

www.mit.edu:

type A,

class IN, addr 18.7.22.83 Name:

www.mit.edu Type: A (Host address) Class: IN (0x0001) Time to live: 1 minute Data length: 4
Addr: 18.7.22.83

Ans5: Screen shots provided

Activity 4:

Now repeat the previous experiment, but instead issue the command:

nslookup –type=NS mit.edu Answer the following questions6 :

-
1. To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?
 2. Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?
 3. Examine the DNS response message. What MIT nameservers does the response message provide? Does this response message also provide the IP addresses of the MIT nameservers?
 4. Provide a screenshot.

Solution:

Ans1: It was sent to 128.238.29.22 which is my default DNS server.

Ans2: It’s a type NS DNS query that doesn’t contain any answers.

Ans3: The nameservers are bitsy, strawb and w20ns. We can find their IP addresses if we expand the Additional records field in Wireshark as seen below. Answers mit.edu: type NS, class inet, ns bitsy.mit.edu mit.edu: type NS, class inet, ns strawb.mit.edu mit.edu: type NS, class inet, ns w20ns.mit.edu Additional records bitsy.mit.edu: type A, class inet, addr 18.72.0.3 strawb.mit.edu: type A, class inet, addr 18.71.0.151 w20ns.mit.edu: type A, class inet, addr 18.70.0.160.

4. Graded Lab Tasks

Lab Task

Answer the following questions:

- To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server? If not, what does the IP address correspond to?
- Examine the DNS query message. What “Type” of DNS query is it? Does the query message contain any “answers”?
- Examine the DNS response message. How many “answers” are provided? What does each of these answers contain?

Lab 05

Transport Control Protocol (TCP) using WireShark

Objective:

The objective of this lab is that the students will investigate the behavior of the celebrated TCP protocol in detail

Activity Outcomes:

Students will gain better understanding of the TCP protocol.

- Analyze a trace of the TCP segments sent and received in transferring a 150KB file from your computer to a remote server.
- Study TCP's use of sequence and acknowledgement numbers for providing reliable data transfer
- Study TCP's congestion control algorithm – slow start and congestion avoidance – in action; and we'll look at TCP's receiver-advertised flow control mechanism.
- Study TCP connection setup and investigate the performance (throughput and round-trip time) of the TCP connection between student's computer and the server.

Instructor Note:

Transport protocols are used to deliver information from one port to another and thereby enable communication between application programs. The TCP connection-oriented protocol establishes a communication link between a source port/IP address and a destination port/IP address. The ports are bound together via this link until the connection is terminated and the link is broken. An example of a connection-oriented protocol is a telephone conversation. A telephone connection is established, communication takes place, and then the connection is terminated. The reliability of the communication between the source and destination programs is ensured through error-detection and error-correction mechanisms that are implemented within TCP. TCP also ensures flow control and congestion control. In this lab, you'll capture some TCP packets using Wireshark and make some observations on them.

References:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- [https://eclass.teicrete.gr/modules/document/file.php/TP326/%CE%98%CE%B5%CF%89%CF%81%CE%AF%CE%B1%20\(Lectures\)/Computer_Networking_A_Top-Down_Approach.pdf](https://eclass.teicrete.gr/modules/document/file.php/TP326/%CE%98%CE%B5%CF%89%CF%81%CE%AF%CE%B1%20(Lectures)/Computer_Networking_A_Top-Down_Approach.pdf)
- Wireshark Labs, https://gaia.cs.umass.edu/kurose_ross/wireshark.php
- Wireshark User Guide: <https://www.wireshark.org/download/docs/user-guide.pdf>
- Wireshark Walkthrough: <https://www.youtube.com/watch?v=TkCSr30UojM>
- Wireshark Website: <https://www.wireshark.org>

Useful Concepts

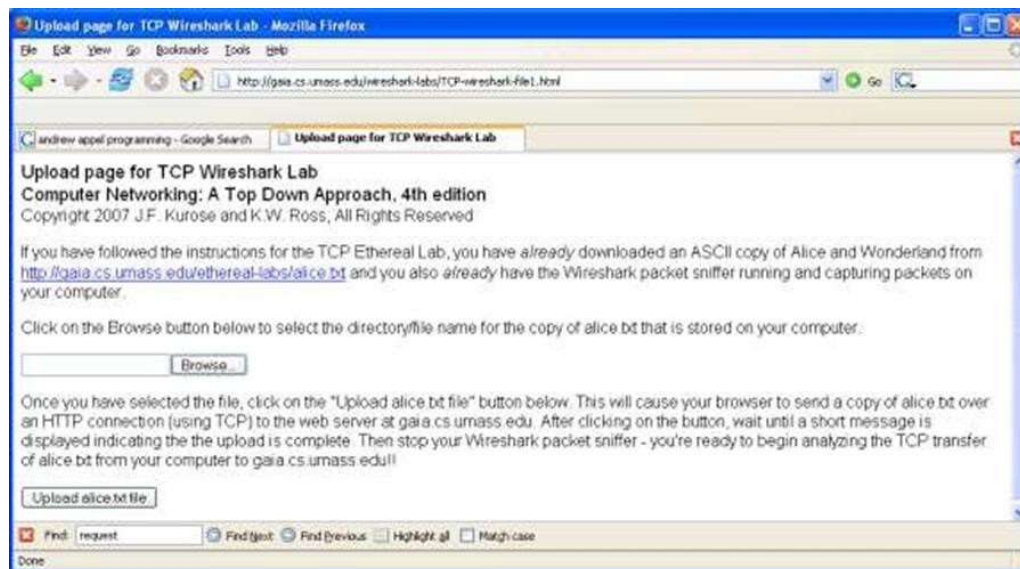
In this lab, we'll investigate the behavior of the celebrated TCP protocol in detail. We'll do so by analyzing a trace of the TCP segments sent and received in transferring a 150KB file (containing the text of Lewis Carroll's Alice's Adventures in Wonderland) from your computer to a remote server. We'll study TCP's use of sequence and acknowledgement numbers for providing reliable data transfer; we'll see TCP's congestion control algorithm – slow start and congestion avoidance – in action; and we'll look at TCP's receiver-advertised flow control mechanism. We'll also briefly consider TCP connection setup and we'll investigate the performance (throughput and round-trip time) of the TCP connection between your computer and the server. For more details, please check the below reference.

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>25</i>	<i>Medium</i>	<i>CLO-6</i>
<i>2</i>	<i>20</i>	<i>High</i>	<i>CLO-6</i>
<i>3</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>

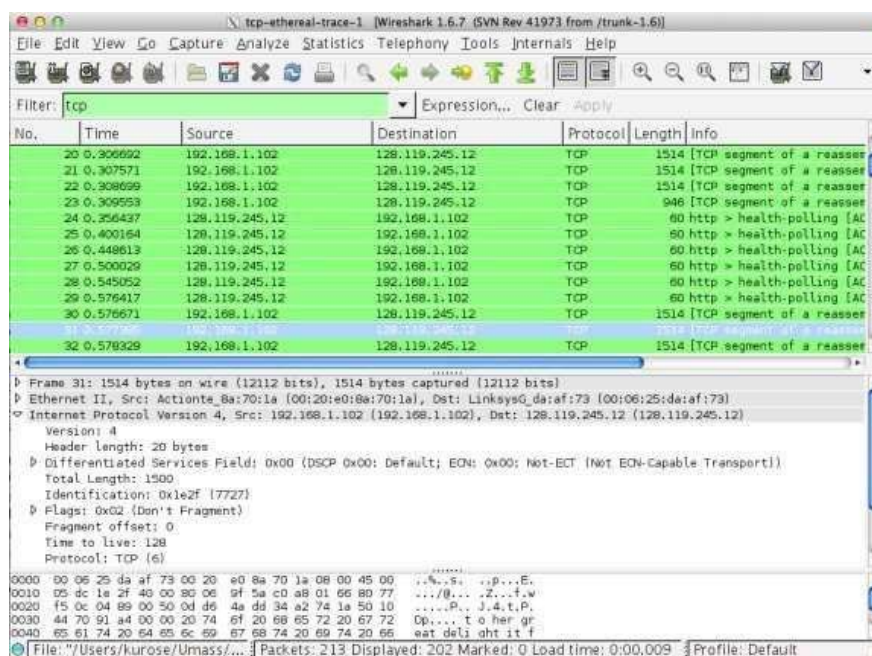
Activity 1:

- Start up your web browser. Go the <http://gaia.cs.umass.edu/wireshark-labs/alice.txt> and retrieve an ASCII copy of Alice in Wonderland. Store this file somewhere on your computer.
- Next go to <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>.
- You should see a screen that looks like:



- Use the Browse button in this form to enter the name of the file (full path name) on your computer containing Alice in Wonderland (or do so manually). Don't yet press the "Upload alice.txt file" button.
- Now start up Wireshark and begin packet capture (Capture->Start) and then press OK on the Wireshark Packet Capture Options screen (we'll not need to select any options here).

- Returning to your browser, press the “Upload alice.txt file” button to upload the file to the gaia.cs.umass.edu server. Once the file has been uploaded, a short congratulations message will be displayed in your browser window.
- Stop Wireshark packet capture. Your Wireshark window should look similar to the window shown below.



If you are unable to run Wireshark on a live network connection, you can download a packet trace file that was captured while following the steps above on one of the author’s computers⁸. You may well find it valuable to download this trace even if you’ve captured your own trace and use it, as well as your own trace, when you explore the questions below.

A first look at the captured trace

Before analyzing the behavior of the TCP connection in detail, let’s take a high level view of the trace.

-
- First, filter the packets displayed in the Wireshark window by entering “tcp” (lowercase, no quotes, and don’t forget to press return after entering!) into the display filter specification window towards the top of the Wireshark window.

What you should see is series of TCP and HTTP messages between your computer and gaia.cs.umass.edu. You should see the initial three-way handshake containing a SYN message. You should see an HTTP POST message. Depending on the version of Wireshark you are using, you might see a series of “HTTP Continuation” messages being sent from your computer to gaia.cs.umass.edu. Recall from our discussion in the earlier HTTP Wireshark lab, that is no such thing as an HTTP Continuation message – this is Wireshark’s way of indicating that there are multiple TCP segments being used to carry a single HTTP message. In more recent versions of Wireshark, you’ll see “[TCP

Download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip> and extract the file tcp-ethereal-trace-1. The traces in this zip file were collected by Wireshark running on one of the author’s computers, while performing the steps indicated in the Wireshark lab. Once you have downloaded the trace, you can load it into Wireshark and view the trace using the File pull down menu, choosing Open, and then selecting the tcp-ethereal-trace-1 trace file.

segment of a reassembled PDU]” in the Info column of the Wireshark display to indicate that this TCP segment contained data that belonged to an upper layer protocol message (in our case here, HTTP). You should also see TCP ACK segments being returned from gaia.cs.umass.edu to your computer.

Answer the following questions, by opening the Wireshark captured packet file tcp-ethereal-trace-1 in <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip> (that is download the trace and open that trace in Wireshark; see footnote 2). Whenever possible, when answering a question you should hand in a printout of the packet(s) within the trace that you used to answer the question asked. Annotate the printout⁹ to explain your answer. To print a packet, use File->Print, choose Selected packet only, choose Packet summary line, and select the minimum amount of packet detail that you need to answer the question.

1. What is the IP address and TCP port number used by the client computer (source) that is transferring the file to gaia.cs.umass.edu? To answer this question, it’s probably easiest to select an HTTP message and explore the details of the TCP packet used to carry this HTTP message,

2. What is the IP address of `gaia.cs.umass.edu`? On what port number is it sending and receiving TCP segments for this connection?

3. What is the IP address and TCP port number used by your client computer (source) to transfer the file to gaia.cs.umass.edu?

tcp-ether-trace-1 [Wireshark 1.6.7 (SVN Rev 41973 from /trunk-1.6)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: tcp Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Length	Info
2	0.023172	128.119.245.12	192.168.1.102	TCP	62	http > health-polling [SYN]
3	0.023265	192.168.1.102	128.119.245.12	TCP	54	health-polling > http [ACK]
4	0.026477	192.168.1.102	128.119.245.12	TCP	619	health-polling > http [PSH]
5	0.041737	192.168.1.102	128.119.245.12	TCP	1514	health-polling > http [PSH]
6	0.053637	128.119.245.12	192.168.1.102	TCP	60	http > health-polling [ACK]
7	0.054026	192.168.1.102	128.119.245.12	TCP	1514	health-polling > http [ACK]
8	0.054690	192.168.1.102	128.119.245.12	TCP	1514	health-polling > http [ACK]
9	0.077294	128.119.245.12	192.168.1.102	TCP	60	http > health-polling [ACK]
10	0.077405	192.168.1.102	128.119.245.12	TCP	1514	health-polling > http [ACK]
11	0.078157	192.168.1.102	128.119.245.12	TCP	1514	health-polling > http [ACK]
12	0.124065	128.119.245.12	192.168.1.102	TCP	60	http > health-polling [ACK]
13	0.124185	192.168.1.102	128.119.245.12	TCP	1201	health-polling > http [PSH]

Frame 1: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)

Ethernet II, Src: Actionte_Ba:70:1a (00:20:e0:8a:70:1a), Dst: LinksysG_da:af:73 (00:06:25:da:af:73)

Destination: LinksysG_da:af:73 (00:06:25:da:af:73)

Address: LinksysG_da:af:73 (00:06:25:da:af:73)

.....0..... = IG bit: Individual address (unicast)

.....0..... = LG bit: Globally unique address (factory default)

Source: Actionte_Ba:70:1a (00:20:e0:8a:70:1a)

Address: Actionte_Ba:70:1a (00:20:e0:8a:70:1a)

.....0..... = IG bit: Individual address (unicast)

.....0..... = LG bit: Globally unique address (factory default)

0000 00 06 25 da af 73 00 20 e0 8a 70 1a 08 00 45 00 ...S...E

2010 00 30 1e 1d 40 00 03 06 45 18 c0 a8 01 66 80 70 ...f...W

0020 f5 0c f4 89 00 50 d6 01 74 00 00 00 00 70 02 ...P...P

3030 40 00 f6 a9 00 00 02 04 05 b4 01 01 04 020.....

File: ~/Users/kurpse/Urmass/... Packets: 213 Displayed: 202 Marked: 0 Load time: 0:00:011 Profile: Default

What do we mean by “annotate”? If you hand in a paper copy, please highlight where in the printout you’ve found the answer and add some text (preferably with a colored pen) noting what you found in what you’ve highlight. If you hand in an electronic copy, it would be great if you could also highlight and annotate.

This is what we’re looking for - a series of TCP segments sent between your computer and gaia.cs.umass.edu. We will use the packet trace that you have captured (and/or the packet trace tcp-ethereal-trace-1 in <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip>; see earlier footnote) to study TCP behavior in the rest of this lab.

Solution:

Ans1: Client computer (source) IP address: 192.168.1.102 TCP port number: 1161

Ans2: Destination computer: gaia.cs.umass.edu IP address: 128.119.245.12 TCP port number: 80

Ans3: If you did this problem on your own computer, you’ll have your own solution

Activity 2:

TCP Basics

Answer the following questions for the TCP segments:

1. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? What is it in the segment that identifies the segment as a SYN segment?
2. What is the sequence number of the SYNACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN? What is the value of the Acknowledgement field in the SYNACK segment? How did gaia.cs.umass.edu determine that value? What is it in the segment that identifies the segment as a SYNACK segment?

3. What is the sequence number of the TCP segment containing the HTTP POST command? Note that in order to find the POST command, you'll need to dig into the packet content field at the bottom of the Wireshark window, looking for a segment with a "POST" within its DATA field.

4. Consider the TCP segment containing the HTTP POST as the first segment in the TCP connection. What are the sequence numbers of the first six segments in the TCP connection (including the segment containing the HTTP POST)? At what time was each segment sent? When was the ACK for each segment received? Given the difference between when each

TCP segment was sent, and when its acknowledgement was received, what is the RTT value for each of the six segments? What is the EstimatedRTT value (see Section 3.5.3, page 239 in text) after the receipt of each ACK? Assume that the value of the EstimatedRTT is equal to the measured RTT for the first segment, and then is computed using the EstimatedRTT equation on page 239 for all subsequent segments.

Note: Wireshark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the "listing of captured packets" window that is being sent from the client to the gaia.cs.umass.edu server. Then select: Statistics->TCP Stream Graph->Round Trip Time Graph.

5. What is the length of each of the first six TCP segments?10

6. What is the minimum amount of available buffer space advertised at the receiver for the entire trace? Does the lack of receiver buffer space ever throttle the sender?

7. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

8. What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

Solution:

Ans1: Sequence number of the TCP SYN segment is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu. The value is 0 in this trace. The SYN flag is set to 1 and it indicates that this segment is a SYN segment.

Ans2: Sequence number of the SYNACK segment from gaia.cs.umass.edu to the client computer in reply to the SYN has the value of 0 in this trace. The value of the ACKnowledgement field in the SYNACK segment is 1. The value of the ACKnowledgement field in the SYNACK segment is determined by gaia.cs.umass.edu by adding 1 to the initial sequence number of SYN segment from the client computer (i.e. the sequence number of the SYN segment initiated by the client computer is 0.). The SYN flag and Acknowledgement flag in the segment are set to 1 and they indicate that this segment is a SYNACK segment

Ans3: No. 4 segment is the TCP segment containing the HTTP POST command. The sequence number of this segment has the value of 1.

Ans4: The HTTP POST segment is considered as the first segment. Segments 1 – 6 are No. 4, 5, 7, 8, 10, and 11 in this trace respectively. The ACKs of segments 1 – 6 are No. 6, 9, 12, 14, 15, and 16 in this trace.

Segment 1 sequence number: 1

Segment 2 sequence number: 566

Segment 3 sequence number: 2026

Segment 4 sequence number: 3486

Segment 5 sequence number: 4946

Segment 6 sequence number: 6406

The sending time and the received time of ACKs are tabulated in the following table
 $\text{EstimatedRTT} = 0.875 * \text{EstimatedRTT} + 0.125 * \text{SampleRTT}$

EstimatedRTT after the receipt of the ACK of segment 1 $\text{EstimatedRTT} = \text{RTT for Segment 1} = 0.02746$ second
EstimatedRTT after the receipt of the ACK of segment 2: $\text{EstimatedRTT} = 0.875 * 0.02746 + 0.125 * 0.035557 = 0.0285$

EstimatedRTT after the receipt of the ACK of segment 3: $\text{EstimatedRTT} = 0.875 * 0.0285 + 0.125 * 0.070059 = 0.0337$

EstimatedRTT after the receipt of the ACK of segment 4:

$$\text{EstimatedRTT} = 0.875 * 0.0337 + 0.125 * 0.11443 = 0.0438$$

EstimatedRTT after the receipt of the ACK of segment 5:

$$\text{EstimatedRTT} = 0.875 * 0.0438 + 0.125 * 0.13989 = 0.0558$$

EstimatedRTT after the receipt of the ACK of segment 6:

$$\text{EstimatedRTT} = 0.875 * 0.0558 + 0.125 * 0.18964 = 0.0725$$

Ans5: Length of the first TCP segment (containing the HTTP POST): 565 bytes Length of each of the other five TCP segments: 1460 bytes (MSS)

Ans6: The minimum amount of buffer space (receiver window) advertised at gaia.cs.umass.edu for the entire trace is 5840 bytes, which shows in the first acknowledgement from the server. This receiver window grows steadily until a maximum receiver buffer size of 62780 bytes. The sender is never throttled due to lacking of receiver buffer space by inspecting this trace.

Ans7: There are no retransmitted segments in the trace file. We can verify this by checking the sequence numbers of the TCP segments in the trace file. In the TimeSequence-Graph (Stevens) of this trace, all sequence numbers from the source (192.168.1.102) to the destination (128.119.245.12) are increasing monotonically with respect to time. If there is a retransmitted segment, the sequence number of this retransmitted segment should be smaller than those of its neighboring segments.

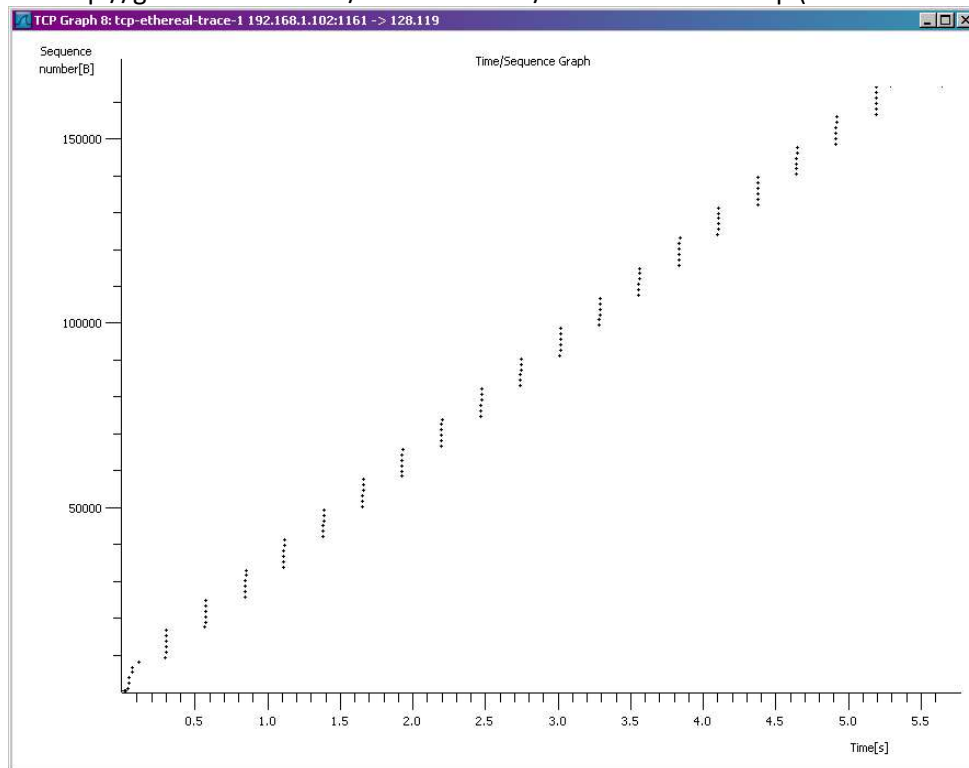
Ans8: The computation of TCP throughput largely depends on the selection of averaging time period. As a common throughput computation, in this question, we select the average time period as the whole connection time. Then, the average throughput for this TCP connection is computed as the ratio between the total amount data and the total transmission time. The total amount data transmitted can be computed by the difference between the sequence number of the first TCP segment (i.e. 1 byte for No. 4 segment) and the acknowledged sequence number of the last ACK (164091 bytes for No. 202 segment). Therefore, the total data are $164091 - 1 = 164090$ bytes. The whole transmission time is the difference of the time instant of the first TCP segment (i.e., 0.026477 second for No.4 segment) and the time instant of the last ACK (i.e., 5.455830 second for No. 202 segment). Therefore, the total transmission time is $5.455830 - 0.026477 = 5.4294$ seconds. Hence, the throughput for the TCP connection is computed as $164090/5.4294 = 30.222$ KByte/sec.

3. Graded Lab Tasks

TCP congestion control in action

Let's now examine the amount of data sent per unit time from the client to the server. Rather than (tediously!) calculating this from the raw data in the Wireshark window, we'll use one of Wireshark's TCP graphing utilities - Time-Sequence-Graph(Stevens) - to plot out data.

- Select a TCP segment in the Wireshark's "listing of captured-packets" window. Then select the menu :Statistics->TCP Stream Graph-> Time-Sequence-Graph(Stevens). You should see a plot that looks similar to the following plot, which was created from the captured packets in the packet trace tcp-ethereal-trace-1 in <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip> (see earlier footnote):



Here, each dot represents a TCP segment sent, plotting the sequence number of the segment versus the time at which it was sent. Note that a set of dots stacked above each other represents a series of packets that were sent back-to-back by the sender.

Answer the following questions for the TCP segments the packet trace tcp-ethereal-trace-1 in <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip>

Use the Time-Sequence-Graph(Stevens) plotting tool to view the sequence number versus time plot of segments being sent from the client to the gaia.cs.umass.edu server. Can you identify where TCP's slowstart phase begins and ends, and where congestion avoidance takes over? Comment on ways in which the measured data differs from the idealized behavior of TCP that we've studied in the text.

Answer each of two questions above for the trace that you have gathered when you transferred a file from your computer to gaia.cs.umass.edu

Lab 06

User Datagram protocol(UDP) using WireShark

Objective:

The objective of this lab will that the students will explore several aspects of UDP protocol.

Activity Outcomes:

After this lab, the students will get better understanding of the UDP protocol.

Instructor Note:

UDP, defined in [RFC 768], does just about as little as a transport protocol can do. Aside from the multiplexing/demultiplexing function and some light error checking, it adds nothing to IP. In fact, if the application developer chooses UDP instead of TCP, then the application is almost directly talking with IP. UDP takes messages from the application process, attaches source and destination port number fields for the multiplexing/demultiplexing service, adds two other small fields, and passes the resulting segment to the network layer. The network layer encapsulates the transport-layer segment into an IP datagram and then makes a best-effort attempt to deliver the segment to the receiving host. If the segment arrives at the receiving host, UDP uses the destination port number to deliver the segment's data to the correct application process. Note that with UDP there is no handshaking between sending and receiving transport-layer entities before sending a segment. For this reason, UDP is said to be connectionless

In this lab, students will capture some UDP packets using Wireshark and make some observations on them. For more details, please check the below reference.

References:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- [https://eclass.teicrete.gr/modules/document/file.php/TP326/%CE%98%CE%B5%CF%89%CF%81%CE%AF%CE%B1%20\(Lectures\)/Computer_Networking_A_Top-Down_Approach.pdf](https://eclass.teicrete.gr/modules/document/file.php/TP326/%CE%98%CE%B5%CF%89%CF%81%CE%AF%CE%B1%20(Lectures)/Computer_Networking_A_Top-Down_Approach.pdf)
- Wireshark Labs, https://gaia.cs.umass.edu/kurose_ross/wireshark.php
- Wireshark User Guide: <https://www.wireshark.org/download/docs/user-guide.pdf>
- Wireshark Walkthrough: <https://www.youtube.com/watch?v=TkCSr30UojM>
- Wireshark Website: <https://www.wireshark.org>

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>40</i>	<i>Medium</i>	<i>CLO-6</i>
<i>2</i>	<i>20</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1:

In this lab, we'll take a quick look at the UDP transport protocol. As we saw in Chapter 3 of the text, UDP is a streamlined, no-frills protocol. You may want to re-read section 3.3 in the text before doing this lab. Because UDP is simple and sweet, we'll be able to cover it pretty quickly in this lab. So if you've another appointment to run off to in 30 minutes, no need to worry, as you should be able to finish this lab with ample time to spare.

Start capturing packets in Wireshark and then do something that will cause your host to send and receive several UDP packets. It's also likely that just by doing nothing (except capturing packets via Wireshark) that some UDP packets sent by others will appear in your trace. In particular, the Simple Network Management Protocol (SNMP - chapter 9 in the text) sends SNMP messages inside of UDP, so it's likely that you'll find some SNMP messages (and therefore UDP packets) in your trace.

After stopping packet capture, set your packet filter so that Wireshark only displays the UDP packets sent and received at your host. Pick one of these UDP packets and expand the UDP fields in the details window. If you are unable to find UDP packets or are unable to run Wireshark on a live network connection, you can download a packet trace containing some UDP packets.¹¹

Whenever possible, when answering a question below, you should hand in a printout of the packet(s) within the trace that you used to answer the question asked. Annotate the printout¹² to explain your answer. To print a packet, use File->Print, choose Selected packet only, choose Packet

Download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip> and extract the file `http-ethereal-trace-5`, which contains some UDP packets carrying SNMP messages. The traces in this zip file were collected by Wireshark running on one of the author's computers. Once you have downloaded the trace, you can load it into Wireshark and view the trace using the File pull down menu, choosing Open, and then selecting the `http-ethereal-trace-5` trace file.

summary line, and select the minimum amount of packet detail that you need to answer the question.

1. Select one UDP packet from your trace. From this packet, determine how many fields there are in the UDP header. (You shouldn't look in the textbook! Answer these questions directly from what you observe in the packet trace.) Name these fields.
2. By consulting the displayed information in Wireshark's packet content field for this packet, determine the length (in bytes) of each of the UDP header fields.
3. The value in the Length field is the length of what? (You can consult the text for this answer). Verify your claim with your captured UDP packet.
4. What is the maximum number of bytes that can be included in a UDP payload? (Hint: the answer to this question can be determined by your answer to 2. above)
5. What is the largest possible source port number? (Hint: see the hint in 4.)
6. What is the protocol number for UDP? Give your answer in both hexadecimal and decimal notation. To answer this question, you'll need to look into the Protocol field of the IP datagram containing this UDP segment

7. Examine a pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet. (Hint: for a second packet to be sent in response to a first packet, the sender of the first packet should be the destination of the second packet). Describe the relationship between the port numbers in the two packets.

Solution:

1. Select one packet. From this packet, determine how many fields there are in the UDP header. (Do not look in the textbook! Answer these questions directly from what you observe in the packet trace.) Name these fields

Ans: The UDP header contains 4 fields: source port, destination port, length, and checksum.

2. From the packet content field, determine the length (in bytes) of each of the UDP header fields.

Ans: Each of the UDP header fields is 2 bytes long.

3. The value in the Length field is the length of what? Verify your claim with your captured UDP packet.

Ans: The value in the length field is the sum of the 8 header bytes, plus the 42 encapsulated data bytes.

4. What is the maximum number of bytes that can be included in a UDP payload.

Ans: The maximum number of bytes that can be included in a UDP payload is $2^{16} - 1$ less the header bytes. This gives $65535 - 8 = 65527$ bytes.

5. What is the largest possible source port number?

Ans: The largest possible source port number is $2^{16} - 1 = 65535$.

4. What is the protocol number for UDP? Give your answer in both hexadecimal and decimal notation. (To answer this question, you'll need to look into the IP header.)

Ans: The IP protocol number for UDP is 0x11 hex, which is 17 in decimal value

-
5. Examine a pair of UDP packets in which the first packet is sent by your host and the second packet is a reply to the first packet. Describe the relationship between the port numbers in the two packets.

Ans: The source port of the UDP packet sent by the host is the same as the destination port of the reply packet, and conversely the destination port of the UDP packet sent by the host is the same as the source port of the reply packet.

2. Graded Lab Tasks

Lab Task

Capture a small UDP packet. Manually verify the checksum in this packet. Show all work and explain all steps.

Lab 07

Socket Programming

Objective:

The objective of this lab is to familiarize the students with the socket programming.

Activity Outcomes:

After this lab the students will

- Have basic knowledge of socket programming. The students will understand the basic concepts of connection-oriented (TCP) and connection-less (UDP) communication using Java.
- Be familiar with the Socket and ServerSocket classes.
- Be familiar with the Datagram and DatagramPacket classes.
- Be able to write Java programs to write network applications, such as client-server application for chat.
- To use threads in network programming. The reason is simple, we don't want only a single client to connect to server at a particular time but many clients simultaneously. We want our architecture to support multiple clients at the same time.

Instructor Note:

Transport protocols are used to deliver information from one port to another and thereby enable communication between application programs. They use either a connection-oriented or connectionless method of communication. TCP is a connection-oriented protocol, and UDP is a connectionless transport protocol. The TCP connection-oriented protocol establishes a communication link between a source port/IP address and a destination port/IP address. The ports

are bound together via this link until the connection is terminated and the link is broken. An example of a connection-oriented protocol is a telephone conversation. A telephone connection is established, communication takes place, and then the connection is terminated. The reliability of the communication between the source and destination programs is ensured through error-detection and error-correction mechanisms that are implemented within TCP. TCP implements the connection as a stream of bytes from source to destination. This feature allows the use of the stream I/O classes provided by java.io. The UDP connectionless protocol differs from the TCP connection-oriented protocol in that it does not establish a link for the duration of the connection. An example of a connectionless protocol is postal mail. To mail something, you just write down a destination address (and an optional return address) on the envelope of the item you're sending and drop it into a mailbox. When using UDP, an application program writes the destination port and IP address on a datagram and then sends the datagram to its destination. UDP is less reliable than TCP because there are no delivery-assurance or error-detection-and-correction mechanisms built into the protocol. Application protocols such as FTP, SMTP, and HTTP use TCP to provide reliable, stream-based communication between client and server programs. Other protocols, such as the Time Protocol, use UDP because speed of delivery is more important than end-to-end reliability. In this lab, you will learn how to code client/server applications based on UDP and TCP protocols. For more details, please check the below reference.

Reference:

- *Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.*
- *Java Network Programming, by Elliotte Rusty Harold (O'Reilly)*
- *Java Network Programming, by Merlin and Conrad Hughes, Michael Shoffner, and Maria Winslow (Manning, an imprint of Prentice-Hall)*
- *Advanced Java Networking, by Prashant Sridharan (Prentice-Hall)*
- <https://www.ibm.com/docs/en/i/7.3?topic=design-creating-connectionless-socket>
- <https://www.javatpoint.com/DatagramSocket-and-DatagramPacket>

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>15</i>	<i>Low</i>	<i>CLO-5</i>
<i>2</i>	<i>10</i>	<i>Medium</i>	<i>CLO-5</i>
<i>3</i>	<i>15</i>	<i>Medium</i>	<i>CLO-5</i>
<i>4</i>	<i>20</i>	<i>High</i>	<i>CLO-5</i>
<i>5</i>	<i>15</i>	<i>Low</i>	<i>CLO-5</i>

Activity 1: Connectionless Communication using UDP

The `java.net` package provides several classes that support socket-based client/server communication. The `InetAddress` class encapsulates Internet IP addresses and supports conversion between dotted decimal addresses and host names. The `Socket`, `ServerSocket`, `DatagramSocket`, and `MulticastSocket` classes implement client and server sockets for connection-oriented and connectionless communication. The `DatagramPacket` class is used to construct UDP datagram packets. Two `java.net` classes define the heart of datagram-based messaging in Java, the `DatagramSocket` and the `DatagramPacket`. The `DatagramSocket` is the interface through which `DatagramPacket` are transmitted. A `DatagramPacket` is simply an IP-specific wrapper for a block of data. The `DatagramSocket` class provides a good interface to the UDP protocol. This class is responsible for sending and receiving `DatagramPacket` via the UDP protocol. The most commonly used `DatagramSocket` methods are listed below:

- `DatagramSocket()`. Constructor comes in two formats: one is used to specify the local port used and the other the system picks an ephemeral local port for you.
- `receive()`. Receive a `DatagramPacket` from any remote server.
- `send()`. Send a `DatagramPacket` to the remote server specified in the `DatagramPacket`.

•close(). Tear down local communication resources. After this method has been called, release this object.

•getLocalPost(). Returns the local port this DatagramSocket is using.

The following program demonstrates how to implement a simple server that converts the text transmitted by client to uppercase, and sends it back to the client. The client reads line from standard input (inFromUserstream) , sends to server via socket (outToServer stream). The server reads line from socket. Server converts line to uppercase, sends back to client. Client reads, prints modified line from socket (inFromServer stream)

Here's the code for the server:

```
import java.io.*;
import java.net.*;
public class UdpServer
{
    public static void main(String args[]) throws Exception
    {
        // open datagram socket on port 9876
        DatagramSocket sock = new DatagramSocket(9876);
        // create two packets sharing a common buffer
        byte[] buf = new byte[1000];
        DatagramPacket inPkt = new DatagramPacket(buf, buf.length);
        DatagramPacket outPkt = new DatagramPacket(buf, buf.length);
        while (true)
        {
            // wait for incoming packet
            sock.receive(inPkt);
            // set address, port and length fields of outPkt
            // so as to return contents of inPkt to sender
            outPkt.setAddress(inPkt.getAddress());
            outPkt.setPort(inPkt.getPort());
            outPkt.setLength(inPkt.getLength());
            // and send it back
            sock.send(outPkt);
        }
    }
}
```

And the following code is for a client program that simply connects to the server, takes input from the user, sends it to server. Upon receiving the text from server prints it.

```
import java.io.*; import java.net.*;
public class UdpClient {
    public static void main(String args[]) throws Exception {
        // get server address and open socket
        InetAddress serverAdr = InetAddress.getByName(args[0]);
        DatagramSocket sock = new DatagramSocket();
        // build packet addressed to server, then send it
        byte[] outBuf = args[1].getBytes("US-ASCII");
        DatagramPacket outPkt = new
            DatagramPacket(outBuf,outBuf.length,serverAdr,9876);
        sock.send(outPkt);
        // create buffer and packet for reply, then receive it
        byte[] inBuf = new byte[1000];
        DatagramPacket inPkt = new DatagramPacket(inBuf,inBuf.length);
        sock.receive(inPkt);
        // print buffer contents and close socket
        String reply = new String(inBuf,0,inPkt.getLength(),"US-ASCII");
        System.out.println(reply);
        sock.close();
    }
}
```

Activity 2: Socket Programming using TCP

The Socket class implements client connection-based sockets. These sockets are used to develop applications that utilize services provided by connection-oriented server applications. The access methods of the Socket class are used to access the I/O streams and connection parameters associated with a connected socket. The `getInetAddress()` and `getPort()` methods get the IP address of the destination host and the destination host port number to which the socket is connected. The `ServerSocket` class implements a TCP server socket. It provides three constructors that specify the port to which the server socket is to listen for incoming connection requests, an optional maximum connection request queue length, and an optional Internet address.

The `accept()` method is used to cause the server socket to listen and wait until an incoming connection is established. It returns an object of class `Socket` once a connection is made. This `Socket` object is then used to carry out a service for a single client. The `getInetAddress()` method

returns the address of the host to which the socket is connected. The `getLocalPort()` method returns the port on which the server socket listens for an incoming connection.

The following program demonstrates how to implement a simple server that returns the current date time for every new client. Here's the code:

```
import java.io.*;
import java.net.*;
import java.util.Date;

/**
 * This program demonstrates a simple TCP/IP socket server.
 */
public class TimeServer {

    public static void main(String[] args) {
        if (args.length < 1) return;

        int port = Integer.parseInt(args[0]);

        try (ServerSocket serverSocket = new ServerSocket(port)) {

            System.out.println("Server is listening on port " + port);

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");
                OutputStream output = socket.getOutputStream();
                PrintWriter writer = new PrintWriter(output, true);
                writer.println(new Date().toString());
            }

        } catch (IOException ex) {
            System.out.println("Server exception: " + ex.getMessage());
            ex.printStackTrace();
        }
    }
}
```

```
}
```

You need to specify a port number when running this server program, for example:

```
java TimeServer 6868
```

This makes the server listens for client requests on the port number 6868. You would see the server's output:

```
Server is listening on port 6868
```

And the following code is for a client program that simply connects to the server and prints the data received, and then terminates:

```
import java.net.*;
import java.io.*;

/**
 * This program demonstrates a simple TCP/IP socket client.
 */
public class TimeClient {

    public static void main(String[] args) {
        if (args.length < 2) return;

        String hostname = args[0];
        int port = Integer.parseInt(args[1]);

        try (Socket socket = new Socket(hostname, port)) {

            InputStream input = socket.getInputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));
            String time = reader.readLine();
            System.out.println(time);

        } catch (UnknownHostException ex) {
            System.out.println("Server not found: " + ex.getMessage());
        } catch (IOException ex) {
            System.out.println("I/O error: " + ex.getMessage());
        }
    }
}
```

To run this client program, you have to specify the hostname/IP address and port number of the server. If the client is on the same computer with the server, type the following command to run it:

```
java TimeClient localhost 6868
```

Then you see a new output in the server program indicating that the client is connected:

```
New client connected
```

And you should see the client's output:

```
Mon May 13 11:00:31 ICT 2022
```

This is the date time information returned from the server. Then the client terminates and the server is still running, waiting for new connections. It's that simple.

Multithreading

Learning Objectives:

After reading this chapter, you should:

- understand what is meant by a thread (in a programming context);
- appreciate the need for multithreaded programming;
- be aware of typical circumstances under which multithreading might be appropriate;
- know how to implement threads in Java;
- know how to implement variable locking in Java;
- be aware of the danger posed by deadlock;
- know what Java methods to use in order to improve thread efficiency and reduce the likelihood of deadlock;
- know how to implement a multithreaded server;
- know how to implement a non-blocking server via multiplexing.

Thread Basics

A **thread** is a flow of control through a program. Unlike a process, a thread does not have a separate allocation of memory but shares memory with other threads created by the same application. This means that servers using threads do not exhaust their supply of available memory and collapse under the weight of excessive demand from clients, as they were prone to do when creating many

separate processes. In addition, the threads created by an application can share global variables, which is often highly desirable. This does not prevent each thread from having its own local variables, of course, since it will still have its own stack for such variables.

Though it has been entirely transparent to us and we have had to make no explicit programming allowance for it, we have already been making use of threads in our Java programming. In fact, we cannot avoid using threads in Java, since each program will have at least one thread that is launched automatically by our machine's JVM when that program is executed. Such a thread is created when *main* is started and 'killed' when *main* terminates. If we wish to make use of further threads, in order to 'offload' processing tasks onto them, then we have to program such threads explicitly. Using more than one thread in this way is called **multithreading**.

Using Threads in Java

Java is unique amongst popular programming languages in making multithreading directly accessible to the programmer, without him/her having to go through an operating system API. Unfortunately, writing multithreaded programs can be rather tricky and there are certain pitfalls that need to be avoided. These pitfalls are caused principally by the need to coordinate the activities of the various threads, as will be seen in Section 3.4.

In Java, an object can be run as a thread if it implements the inbuilt interface *Runnable*, which has just one method: *run*. Thus, in order to implement the interface, we simply have to provide a definition for method *run*. Since the inbuilt class *Thread* implements this interface, there are two fundamental methods for creating a thread class:

- create a class that extends *Thread*;
- create a class that does not extend *Thread* and specify explicitly that it implements *Runnable*.

Extending the *Thread* Class

The *run* method specifies the actions that a thread is to execute and serves the same purpose for the process running on the thread as method *main* does for a full application program. Like *main*, *run* may not be called directly. The containing program calls the *start* method (inherited from class *Thread*), which then automatically calls *run*.

Class *Thread* has seven constructors, the two most common of which are:

- *Thread()*

- *Thread(String<name>)*

The second of these provides a name for the thread via its argument. If the first is used, the system generates a name of the form *Thread-n*, where n is an integer starting at zero and increasing in value for further threads. Thus, if three threads are created via the first constructor, they will have names *Thread-0*, *Thread-1* and *Thread-2* respectively. Whichever constructor is used, method *getName* may be used to retrieve the name.

Example

```
Thread firstThread = new Thread();
Thread secondThread = new Thread("namedThread");
System.out.println(firstThread.getName()); System.out.println(secondThread.getName());
```

The output from the above lines would be:

Thread-0 namedThread

Note that the name of the variable holding the address of a thread is **not** the same as the name of the thread! More often than not, however, we do not need to know the latter.

Method *sleep* is used to make a thread pause for a specified number of milliseconds. For example:

```
myThread.sleep(1500); //Pause for 1.5 seconds.
```

This suspends execution of the thread and allows other threads to be executed. When the sleeping time expires, the sleeping thread returns to a *ready* state, waiting for the processor.

Method *interrupt* may be used to interrupt an individual thread. In particular, this method may be used by other threads to ‘awaken’ a sleeping thread before that thread’s sleeping time has expired. Since method *sleep* will throw a checked exception (an *InterruptedException*) if another thread invokes the *interrupt* method, it must be called from within a try block that catches this exception. In the next example, static method *random* from core class *Math* is used to generate a random sleeping time for each of two threads that simply display their own names ten times. If we were to run the program without using a randomising element, then it would simply display alternating names, which would be pretty tedious and would give no indication that threads were being used. Method *random* returns a random decimal value in the range 0-0.999..., which is then multiplied

by a scaling factor of 3000 and typecast into an *int*, producing a final integer value in the range 0-2999. This randomising technique is also used in later thread examples, again in order to avoid producing the same pattern of output from a given program.

Activity 3:

Note the use of *extends Thread* in the opening line of the class. Though this class already implements the *Runnable* interface (and so has a definition of method *run*), the default implementation of *run* does nothing and must be overridden by a definition that we supply.

Example

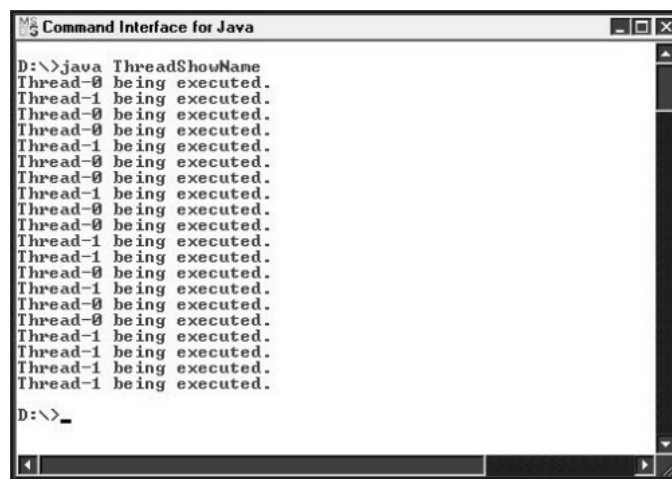
```
public class ThreadShowName extends Thread
{
    public static void main (String[] args)
    {
        ThreadShowName thread1, thread2; thread1 = new ThreadShowName();
        thread2 = new ThreadShowName();
        thread1.start(); //Will call run.
        thread2.start(); //Will call run.
    }

    public void run()
    {
        int pause; for (int i=0; i<10; i++)
        {
            try
            {
                System.out.println( getName()+" being executed.");
                pause = (int)(Math.random()*3000);
                sleep(pause);
                //0-3 seconds.
            }
            catch (InterruptedException interruptEx)
```

```

{
System.out.println(interruptEx); } }
}
}

```



Activity No 4:

In this example, we shall again create two threads, but we shall have one thread display the message 'Hello' five times and the other thread output integers 1-5. For the first thread, we shall create a class called *HelloThread*; for the second, we shall create class *CountThread*. Note that it is **not** the main application class (*ThreadHelloCount*, here) that extends class *Thread* this time, but each of the two subordinate classes, *HelloThread* and *CountThread*. Each has its own version of the *run* method.

```

public class ThreadHelloCount
{

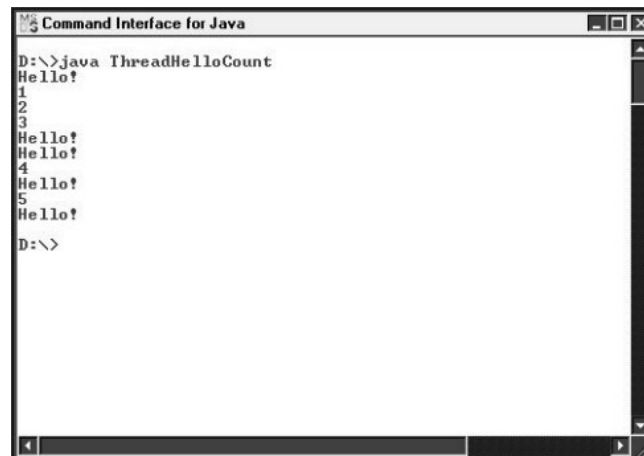
```

```
public static void main(String[] args)
{
    HelloThread hello = new HelloThread();
    CountThread count = new CountThread();
    hello.start();
    count.start();
}

class HelloThread extends Thread
{
    public void run()
    {
        int pause; for (int i=0; i<5; i++)
        {
            try
            {
                System.out.println("Hello!");
                //Again, introduce an element //of randomness...
                pause = (int)(Math.random()*3000);
                sleep(pause);
            }
            catch (InterruptedException interruptEx)
            {
                System.out.println(interruptEx);
            }
        }
    }
}

class CountThread extends Thread
{
    int pause;
    public void run()
```

```
{  
for (int i=0; i<5; i++)  
{  
try  
{  
System.out.println(i); pause=(int)(Math.random()*3000); sleep (pause);  
  
}  
  
catch (InterruptedException interruptEx)  
{  
System.out.println(interruptEx);  
}  
}  
}  
}
```



The screenshot shows a window titled "Command Interface for Java". The command prompt displays the command `D:\>java ThreadHelloCount`. The output shows the numbers 1 through 5, each followed by "Hello!" on a new line. The prompt `D:\>` is visible at the bottom.

Multithreaded Servers

There is a fundamental and important limitation associated with all the server programs encountered so far:

- they can handle only one connection at a time.

This restriction is simply not feasible for most real-world applications and would render the software useless. There are two possible solutions:

- use a non-blocking server;
- use a multithreaded server.

Though inferior to the non-blocking approach, the multithreaded technique has a couple of significant benefits:

- it offers a 'clean' implementation, by separating the task of allocating connections from that of processing each connection;
- it is robust, since a problem with one connection will not affect other connections.

The basic technique involves a two-stage process:

1. the main thread (the one running automatically in method *main*) allocates individual threads to incoming clients;
2. the thread allocated to each individual client then handles all subsequent interaction between that client and the server (via the thread's *run* method).

Since each thread is responsible for handling all further dialogue with its particular client, the main thread can 'forget' about the client once a thread has been allocated to it. It can then concentrate on its simple tasks of waiting for clients to make connection and allocating threads to them as they do so. For each client-handling thread that is created, of course, the main thread must ensure that the client-handling thread is passed a reference to the socket that was opened for the associated client. The separation of responsibilities means that, if a problem occurs with the connection to a particular client, it has no effect on the connections to other clients and there is no general loss of service. This is a major benefit, of course.

Activity No 5:

This is another echo server implementation, but one that uses multithreading to return messages to multiple clients. It makes use of a support class called *ClientHandler* that extends class *Thread*. Whenever a new client makes connection, a *ClientHandler* thread is created to handle all subsequent communication with that particular client. When the *ClientHandler* thread is created, its constructor is supplied with a reference to the relevant socket.

Here's the code for the server...

```
import java.io.*;
import java.net.*;

public class MultiEchoServer {
    private static ServerSocket serverSocket; private static final int PORT = 1234;
    public static void main(String[] args)
        throws IOException
    {
        try
        {
            serverSocket = new ServerSocket(PORT);
        }
        catch (IOException ioEx)
        {
            System.out.println("\nUnable to set up port!");
            System.exit(1);
        }
    }
    do
    {
        //Wait for client...
        Socket client = serverSocket.accept();
        System.out.println("\nNew client accepted.\n");
        //Create a thread to handle communication with //this client and pass the constructor for this
        //thread a reference to the relevant socket...
```

```
ClientHandler handler =  
new ClientHandler(client); handler.start();//As usual, method calls run.  
}  
while (true);  
}  
}
```

```
class ClientHandler extends Thread  
{  
private Socket client;  
private Scanner input;  
private PrintWriter output;  
public ClientHandler(Socket socket)  
{  
//Set up reference to associated socket...  
client = socket;  
try  
{  
input = new Scanner(client.getInputStream());  
output = new PrintWriter( client.getOutputStream(),true);  
}  
catch(IOException ioEx)  
{ ioEx.printStackTrace();  
}  
}  
public void run()  
{  
String received;  
do  
{  
  
//Accept message from client on //the socket's input stream...  
received = input.nextLine();
```

```
//Echo message back to client on //the socket's output stream...
output.println("ECHO: " + received);
//Repeat above until 'QUIT' sent by client...
}
while (!received.equals("QUIT"));
try
{
if (client!=null)
{
System.out.println( "Closing down connection...");
client.close();
}
}
catch(IOException ioEx)
{
System.out.println("Unable to disconnect!");
}
}
}
```

The code required for the client program is exactly that which was employed in the *TCPEchoClient* program from the last chapter. However, since (i) there was only a modest amount of code in the *run* method for that program, (ii) we should avoid confusion with the *run* method of the *Thread* class and (iii) it'll make a change (!) without being harmful, all the executable code has been placed inside *main* in the *MultiEchoClient* program below.

Activity No 6:

```
import java.io.*;
import java.net.*;
import java.util.*;
public class MultiEchoClient {
private static InetAddress host; private static final int PORT = 1234;
public static void main(String[] args)
```

```
{
try
{

host = InetAddress.getLocalHost();
}
catch(UnknownHostException uhEx)
{
System.out.println("\nHost ID not found!\n");
System.exit(1);
}
sendMessage();
}
private static void sendMessage()
{
Socket socket = null;
try
{
socket = new Socket(host,PORT);
Scanner networkInput = new Scanner(socket.getInputStream());
PrintWriter networkOutput = new PrintWriter( socket.getOutputStream(),true);
//Set up stream for keyboard entry...

Scanner userEntry = new Scanner(System.in);
String message, response;
do
{
System.out.print( "Enter message ('QUIT' to exit): ");
message = userEntry.nextLine();
//Send message to server on the //socket's output stream...
//Accept response from server on the //socket's input stream...
networkOutput.println(message);
response = networkInput.nextLine();
//Display server's response to user...
```

```
System.out.println( "\nSERVER> " + response);
}
while (!message.equals("QUIT"));
}
catch(IOException ioEx)

{
ioEx.printStackTrace();
}
finally
{
try
{
System.out.println( "\nClosing connection...");
socket.close();
}

catch(IOException ioEx)
{
System.out.println( "Unable to disconnect!");
System.exit(1);
}
}
}
```

Activity 7: TCP-based client/server program where the Server (single-threaded) is responsible for reversing the text sent by client.

The following TCP-based server program echoes anything sent from the client in reversed form (hence the name `ReverseServer`). Here's the code:

```
import java.io.*;
import java.net.*;

/**
 * The server program echoes anything sent from the client in reversed form
 * This server is single-threaded.
 */
public class ReverseServer {

    public static void main(String[] args) {
        if (args.length < 1) return;

        int port = Integer.parseInt(args[0]);

        try (ServerSocket serverSocket = new ServerSocket(port)) {

            System.out.println("Server is listening on port " + port);

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");

                InputStream input = socket.getInputStream();
                BufferedReader reader = new BufferedReader(new InputStreamReader(input));

                OutputStream output = socket.getOutputStream();
                PrintWriter writer = new PrintWriter(output, true);
                String text;

                do {
                    text = reader.readLine();
```

```

        String reverseText = new StringBuilder(text).reverse().toString();
        writer.println("Server: " + reverseText);

    } while (!text.equals("bye"));

    socket.close();
}

} catch (IOException ex) {
    System.out.println("Server exception: " + ex.getMessage());
    ex.printStackTrace();
}
}
}

```

you can see, the server continues serving the client until it says ‘bye’. Run this server program using the following command:

```
java ReverseServer 9090
```

The server is up and running, waiting for incoming requests from clients:

```
Server is listening on port 9090
```

Now, let’s create a client program. The following program connects to the server, reads input from the user and prints the response from the server. Here’s the code:

```

import java.net.*;
import java.io.*;

/**
 * This program demonstrates a simple TCP/IP socket client that reads input
 * from the user and prints echoed message from the server.
 */
public class ReverseClient {

    public static void main(String[] args) {
        if (args.length < 2) return;
    }
}

```

```

String hostname = args[0];
int port = Integer.parseInt(args[1]);
try (Socket socket = new Socket(hostname, port)) {

    OutputStream output = socket.getOutputStream();
    PrintWriter writer = new PrintWriter(output, true);
    Console console = System.console();
    String text;

    do {
        text = console.readLine("Enter text: ");

        writer.println(text);

        InputStream input = socket.getInputStream();
        BufferedReader reader = new BufferedReader(new InputStreamReader(input));

        String time = reader.readLine();

        System.out.println(time);

    } while (!text.equals("bye"));

    socket.close();

} catch (UnknownHostException ex) {

    System.out.println("Server not found: " + ex.getMessage());

} catch (IOException ex) {

    System.out.println("I/O error: " + ex.getMessage());

}
}

```

As you can see, this client program is running until the user types ‘bye’. Run it using the following command:

```
java ReverseClient localhost 9090
```

Then it asks you to enter some text:

```
Enter text: _
```

Type something, say ‘Hello’ and you should see the server’s response like this:

```
Enter text: Hello
Server: olleH
Enter text: _
```

Keep this first client program running, and start a new one. In the second client program, you will see it asks for input and then hangs forever. Why?

It’s because the server is single-threaded, and while it is busily serving the first client, subsequent clients are block.

Let’s see how to solve this problem in the next example.

Activity 8: TCP-based multi-threaded client/server application where the server is able to handle multiple clients

Modify the server’s code to handle each socket client in a new thread like this:

```
import java.io.*;
import java.net.*;

/**
 * This program demonstrates a simple TCP/IP socket server that echoes every
 * message from the client in reversed form.
 * This server is multi-threaded.
 *
 * @author www.codejava.net
 */
public class ReverseServer {

    public static void main(String[] args) {
        if (args.length < 1) return;

        int port = Integer.parseInt(args[0]);
```

```

        try (ServerSocket serverSocket = new ServerSocket(port)) {

            System.out.println("Server is listening on port " + port);

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");

                new ServerThread(socket).start();
            }

        } catch (IOException ex) {
            System.out.println("Server exception: " + ex.getMessage());
            ex.printStackTrace();
        }
    }

import java.io.*;
import java.net.*;

/**
 * This thread is responsible to handle client connection.
 *
 * @author www.codejava.net
 */
public class ServerThread extends Thread {
    private Socket socket;

    public ServerThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            InputStream input = socket.getInputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));

            OutputStream output = socket.getOutputStream();

```

```
PrintWriter writer = new PrintWriter(output, true);

String text;

do {
    text = reader.readLine();
    String reverseText = new StringBuilder(text).reverse().toString();
    writer.println("Server: " + reverseText);

    } while (!text.equals("bye"));

    socket.close();
} catch (IOException ex) {
    System.out.println("Server exception: " + ex.getMessage());
    ex.printStackTrace();
}
}
```

As you can see, we just move the processing code to be executed into a separate thread, implemented in the `run()` method.

Now let run this new server program and run several client programs, you will see the problem above has solved. All clients are running smoothly.

1. Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficult and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab.

In this lab task, you will build a simple client-server system, where you use the client to chat with a dummy "math" server. The protocol between the client and server is as follows.

- The server is first started on a known port.
- The client program is started (server IP and port is provided on the command line).

-
- The client connects to the server, and then asks the user for input. The user enters a simple arithmetic expression string (e.g., "1 + 2", "5 - 6", "3 * 4"). The user's input is sent to the server via the connected socket.
 - The server reads the user's input from the client socket, evaluates the expression, and sends the result back to the client.
 - The client should display the server's reply to the user, and prompt the user for the next input, until the user terminates the client program with Ctrl+C.

Home Task 1

Write a chat room application that allows multiple users to connect to the chat server. Before the user is able to see the chat window, each user needs to enter his or her name. Once a user is connected, the server window has to display a connected message with the user's address. The user's chat window should display a welcome message from the server. The chat window should include a message panel to display chat history. Below the chat panel, there should be a profile picture, a text field for user to type messages and a send button for sending messages to other connected users. In addition, there should be a setting button, an image button and an upload button. Setting button allows the user to change the background colour. Image button allows a user to select files from the computer. The upload button allows the user to upload the selected image file to the server. The server saves the image at a file named "image.jpg". In the user's chat panel, each message should display the user's name followed by the typed message. Please show screen shot of the server window, and three user chat windows. Three users are "Emily", "Andrew" and "Mark." Please see the result sample in the attached image.

Home Task 2

This exercise converts the above files into a simple email server and email client respectively. The server conversion has been done for you and is contained in file EmailServer.java, a printed version of which appears on the following pages for ease of reference. Some of the code for the client has also been provided for you and is held in file EmailClient.java, a printed version of which is also provided. You are to complete the coding for the client and then run the server program in one command window and the client program in each of two further command windows (so that there are two clients communicating with the server at the same time). The details of this simplified client-server application are given below.

-
- The server recognises only two users, called 'Dave' and 'Karen'.
 - Each of the above users has a message box on the server that can accept a maximum of 10 messages.
 - Each user may either send a one-line message to the other or read his/her own messages.
 - A count is kept of the number of messages in each mailbox. As another message is received, the appropriate count is incremented (if the maximum has not been reached). When messages are read, the appropriate count is reduced to zero.
 - When sending a message, the client sends three things: the user's name, the word 'send' and the message itself.
 - When requesting reading of mail, the client sends two things: the user's name and the word 'read'.
 - As each message is received by the server, it is added to the appropriate mailbox (if there is room). If the mailbox is full, the message is ignored.
 - When a read request is received, the server first sends an integer indicating the number of messages (possibly 0) that will be sent and then transmits the messages themselves (after which it reduces the appropriate message count to 0).
 - Each user is to be allowed to 'send' and/or 'read' as many times as he/she wishes, until he/she decides to quit.
 - When the user selects the 'quit' option, the client sends two things: the user's name and the word 'quit'.

Lab 08

Basic Configurations

Objectives

The objective of this lab is that the students get acquainted with basic configurations of packet tracer tool, computer, and switch.

Activity Outcomes:

-
- Provide hands - on practice related to basic configurations of packet tracer tool
 - How to establish basic connections, access the CLI, and explore Help
 - Explore EXEC Modes
 - Set the Clock
 - Provide hands - on practice related to basic configurations of switch
 - Verify the Default Switch Configuration
 - Configure a Basic Switch Configuration
 - Configure a MOTD Banner
 - Save Configuration Files to NVRAM
 - Configure S2
 - Provide hands - on practice related to basic configurations of personal computer

Useful Concepts

In this activity on Packet Tracer, you will practice skills necessary for navigating the Cisco IOS, such as different user access modes, various configuration modes, and common commands used on a regular basis. In this lab, you will also perform basic switch configuration tasks. You will secure access to the command-line interface (CLI) and console ports using encrypted and plain text passwords. You will also learn how to configure messages for users logging into the switch. These message banners are also used to warn unauthorized users that access is prohibited.

Note: In Packet Tracer, the Catalyst 2960 switch uses IOS version 12.2 by default. If required, the IOS version can be updated from a file server in the Packet Tracer topology. The switch can then be configured to boot to IOS version 15.0, if that version is required. For more details, please check the below reference.

References

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.

- <https://www.cisco.com/c/en/us/support/docs/smb/switches/cisco-350-series-managed-switches/smb5557-configure-the-internet-protocol-ip-address-settings-on-a-swi.html>
- <https://www.ciscopress.com/articles/article.asp?p=2181836&seqNum=4>

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>10</i>	<i>Low</i>	<i>CLO-6</i>
<i>3</i>	<i>20</i>	<i>Medium</i>	<i>CLO-6</i>
<i>4</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>
<i>5</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>

Activity 1: Basic configurations of packet tracer tool

Part 1: Establish Basic Connections, Access the CLI, and Explore Help

Step 1: Connect PC1 to S1 using a console cable.

- Click the **Connections** icon (the one that looks like a lightning bolt) in the lower left corner of the Packet Tracer window.
- Select the light blue Console cable by clicking it. The mouse pointer will change to what appears to be a connector with a cable dangling from it.
- Click **PC1**. A window displays an option for an RS-232 connection. Connect the cable to the RS-232 port.
- Drag the other end of the console connection to the S1 switch and click the switch to access the connection list.
- Select the **Console** port to complete the connection.

Step 2: Establish a terminal session with S1.

- Click **PC1** and then select the **Desktop** tab.
- Click the **Terminal** application icon. Verify that the Port Configuration default settings are correct.

Question:
What is the setting for bits per second?

Type your answers here.

9600

- c. Click **OK**.
- d. The screen that appears may have several messages displayed. Somewhere on the screen there should be a **Press RETURN to get started!** message. Press ENTER.

Question:
What is the prompt displayed on the screen?

Type your answers here.

S1>

Step 3: Explore the IOS Help.

- a. The IOS can provide help for commands depending on the level accessed. The prompt currently displayed is called **User EXEC**, and the device is waiting for a command. The most basic form of help is to type a question mark (?) at the prompt to display a list of commands.

Open Configuration Window

S1>?

Question:
Which command begins with the letter 'C'?

Type your answers here.

connect

- b. At the prompt, type t and then a question mark (?).

S1> **t?**

Question:
Which commands are displayed?

Type your answers here.

telnet terminal traceroute

At the prompt, type te and then a question mark (?).

S1> **te?**

Question:
Which commands are displayed?

Type your answers here.

telnet terminal

This type of help is known as context-sensitive help. It provides more information as the commands are expanded.

Part 2: Explore EXEC Modes

In Part 2 of this activity, you will switch to privileged EXEC mode and issue additional commands

Step 1: Enter privileged EXEC mode.

- a. At the prompt, type the question mark (?).

S1>?

Question:

What information is displayed for the **enable** command?

Type your answers here.

Turn on privileged commands

- b. Type **en** and press the **Tab** key.

S1> **en<Tab>**

Question:

What displays after pressing the **Tab** key?

Type your answers here.

enable

This is called command completion (or tab completion). When part of a command is typed, the **Tab** key can be used to complete the partial command. If the characters typed are enough to make the command unique, as in the case of the **enable** command, the remaining portion of the command is displayed.

Question:

What would happen if you typed **te<Tab>** at the prompt?

Type your answers here.

There is more than one command that begins with the letters 'te' because 'te' does not provide enough characters to make the command unique. The characters will continue to display, prompting the user for additional characters to make the command unique.

- c. Enter the **enable** command and press ENTER.

Question:

How does the prompt change?

Type your answers here.

It changes from S1> to S1#, which indicates privileged EXEC mode.

-
- d. When prompted, type the question mark (?).

S1# ?

One command starts with the letter 'C' in user EXEC mode.

Question:

How many commands are displayed now that privileged EXEC mode is active? (**Hint:** you could type c? to list just the commands beginning with 'C'.)

Type your answers here.

5 - clear, clock, configure, connect, and copy

Step 2: Enter Global Configuration mode

- a. When in privileged EXEC mode, one of the commands starting with the letter 'C' is **configure**. Type either the full command or enough of the command to make it unique. Press the <Tab> key to issue the command and press ENTER.

S1# **configure**

Question:

What is the message that is displayed?

Type your answers here.

Configuring from terminal, memory, or network [terminal]?

- b. Press Enter to accept the default parameter that is enclosed in brackets **[terminal]**.

Question:

How does the prompt change?

Type your answers here.

S1 (config) #

- c. This is called global configuration mode. This mode will be explored further in upcoming activities and labs. For now, return to privileged EXEC mode by typing **end**, **exit**, or **Ctrl-Z**.

S1 (config) # **exit**

S1#

Part 3: Set the Clock

Step 1: Use the clock command.

- a. Use the **clock** command to further explore Help and command syntax. Type **show clock** at the privileged EXEC prompt.

```
S1# show clock
```

Question:

What information is displayed? What is the year that is displayed?

Type your answers here.

UTC Mon Mar 1 1993 preceded by the hours, minutes, and seconds since the device started. The year is 1993.

- b. Use the context-sensitive help and the **clock** command to set the time on the switch to the current time. Enter the command **clock** and press ENTER.

```
S1# clock<ENTER>
```

Question:

What information is displayed?

Type your answers here.

% Incomplete command

- c. The “% Incomplete command” message is returned by the IOS. This indicates that the **clock** command needs more parameters. Any time more information is needed, help can be provided by typing a space after the command and the question mark (?).

```
S1# clock ?
```

Question:

What information is displayed?

Type your answers here.

set Set the time and date

- d. Set the clock using the **clock set** command. Proceed through the command one step at a time.

```
S1# clock set ?
```

Question:

What information is being requested?

Type your answers here.

hh:mm:ss Current Time

What would have been displayed if only the **clock set** command had been entered, and no request for help was made by using the question mark?

Type your answers here.

% Incomplete command

- e. Based on the information requested by issuing the **clock set ?** command, enter a time of 3:00 p.m. by using the 24-hour format of 15:00:00. Check to see if more parameters are needed.

```
S1# clock set 15:00:00 ?
```

The output returns a request for more information:

```
<1-31> Day of the month
```

```
MONTH Month of the year
```

- f. Attempt to set the date to 01/31/2035 using the format requested. It may be necessary to request additional help using context-sensitive help to complete the process. When finished, issue the **show clock** command to display the clock setting. The resulting command output should display as:

```
S1# show clock
```

```
*15:0:4.869 UTC Tue Jan 31 2035
```

- g. If you were not successful, try the following command to obtain the output above:

```
S1# clock set 15:00:00 31 Jan 2035
```

Activity 2: Basic configurations of Switch

Verify the Default Switch Configuration

Enter privileged EXEC mode.

You can access all switch commands from privileged EXEC mode. However, because many of the privileged commands configure operating parameters, privileged access should be password-protected to prevent unauthorized use.

The privileged EXEC command set includes the commands available in user EXEC mode, many additional commands, and the **configure** command through which access to the configuration modes is gained.

Click **S1** and then the CLI tab. Press Enter.

Enter privileged EXEC mode by entering the enable command:

Open Configuration Window for S1

```
Switch> enable
```

```
Switch#
```

Notice that the prompt changed to reflect privileged EXEC mode.

Create a Basic Switch Configuration

Assign a name to a switch.

To configure parameters on a switch, you may be required to move between various configuration modes. Notice how the prompt changes as you navigate through the switch.

```
Switch# configure terminal  
Switch(config)# hostname S1  
S1(config)# exit  
S1#
```

Secure access to the console line.

To secure access to the console line, access config-line mode and set the console password to **letmein**.

```
S1# configure terminal  
Enter configuration commands, one per line. End with  
CNTL/Z.  
S1(config)# line console 0  
S1(config-line)# password letmein  
S1(config-line)# login  
S1(config-line)# exit  
S1(config)# username cisco1 password cisco2  
%SYS-5-CONFIG_I: Configured from console by console  
S1#
```

Question:

Verify that console access is secured.

Exit privileged mode to verify that the console port password is in effect.

```
S1# exit
```

```
Switch con0 is now available
Press RETURN to get started.
```

```
User Access Verification
Password:
S1>
```

Note: If the switch did not prompt you for a password, then you did not configure the **login** parameter in Step 2.

Secure privileged mode access.

Set the **enable** password to **c1\$c0**. This password protects access to privileged mode.

Note: The **0** in **c1\$c0** is a zero, not a capital O. This password will not grade as correct until after you encrypt it in Step 8.

```
S1> enable
S1# configure terminal
S1(config)# enable password c1$c0
S1(config)# exit
%SYS-5-CONFIG_I: Configured from console by console
S1#
```

Verify that privileged mode access is secure.

Enter the **exit** command again to log out of the switch.

Press <**Enter**> and you will now be asked for a password:

```
User Access Verification
Password:
```

The first password is the console password you configured for **line con 0**. Enter this password to return to user EXEC mode.

Enter the command to access privileged mode.

Enter the second password you configured to protect privileged EXEC mode.

Verify your configuration by examining the contents of the running-configuration file:

```
S1# show running-config
```

Notice that the console and enable passwords are both in plain text. This could pose a security risk if someone is looking over your shoulder or obtains access to config files stored in a backup location.

Configure an encrypted password to secure access to privileged mode.

The **enable password** should be replaced with the newer encrypted secret password using the **enable secret** command. Set the enable secret password to **itsasecret**.

```
S1# config t
S1(config)# enable secret itsasecret
S1(config)# exit
S1#
```

Note: The **enable secret** password overrides the **enable** password. If both are configured on the switch, you must enter the **enable secret** password to enter privileged EXEC mode.

Verify that the enable secret password is added to the configuration file.

Enter the show running-config command again to verify the new enable secret password is configured.

Note: You can abbreviate **show running-config** as

```
S1# show run
```

Question:
What is displayed for the enable secret password?

Type your answers here.

```
$1$mERr$ILwq/b7kc.7X/ejA4Aosn0
```

Why is the enable secret password displayed differently from what we configured?

Type your answers here.

The enable secret is shown in encrypted form, whereas the enable password is in plain text.

Encrypt the enable and console passwords.

As you noticed in Step 7, the **enable secret** password was encrypted, but the **enable** and **console** passwords were still in plain text. We will now encrypt these plain text passwords using the **service password-encryption** command.

```
S1# config t
S1(config)# service password-encryption
```

```
S1(config)# exit
```

Question:

If you configure any more passwords on the switch, will they be displayed in the configuration file as plain text or in encrypted form? Explain.

Type your answers here.

The service password-encryption command encrypts all current and future passwords.

Configure a message of the day (MOTD) banner.

Configure a message of the day (MOTD) banner.

The Cisco IOS command set includes a feature that allows you to configure messages that anyone logging onto the switch sees. These messages are called message of the day, or MOTD banners. Enclose the banner text in quotations or use a delimiter different from any character appearing in the MOTD string.

```
S1# config t
S1(config)# banner motd "This is a secure system.
Authorized Access Only!"
S1(config)# exit
%SYS-5-CONFIG_I: Configured from console by console
S1#
```

Question:

When will this banner be displayed?

Type your answers here.

The message will be displayed when someone accesses the switch through the console port.

Why should every switch have a MOTD banner?

Type your answers here.

Every switch should have a banner to warn unauthorized users that access is prohibited. Banners can also be used for sending messages to network personnel/technicians (such as impending system shutdowns or who to contact for access).

Save and Verify Configuration Files to NVRAM

Verify that the configuration is accurate using the show run command.

Save the configuration file. You have completed the basic configuration of the switch. Now back up the running configuration file to NVRAM to ensure that the changes made are not lost if the system is rebooted or loses power.

```
S1# copy running-config startup-config
Destination filename [startup-config]? [Enter]
Building configuration...

[OK]
```

Close Configuration Window for S1
Question:

What is the shortest, abbreviated version of the **copy running-config startup-config** command?

Type your answers here.

cop r s

Examine the startup configuration file.

Which command will display the contents of NVRAM?

Type your answers here.

show startup-config

Are all the changes that were entered recorded in the file?

Type your answers here.

Yes, it is the same as the running configuration.

Configure S2

You have completed the configuration on S1. You will now configure S2. If you cannot remember the commands, refer to Parts 1 to 4 for assistance.

Configure S2 with the following parameters:

Open Configuration Window for S2

Device name: **S2**

Protect access to the console using the **letmein** password.

Configure an enable password of **c1\$c0** and an enable secret password of **itsasecret**.

Configure an appropriate message to those logging into the switch.

Encrypt all plain text passwords.

Ensure that the configuration is correct.

Save the configuration file to avoid loss if the switch is powered down.

Close Configuration Window for S2

Script

Switch S1

```
enable
config t
hostname S1
line console 0
  password letmein
  login
enable password c1$c0
enable secret itsasecret
banner motd $ This is a secure system. Authorized Access Only!$
service password-encryption
exit
copy running-config startup-config
```

Switch S2

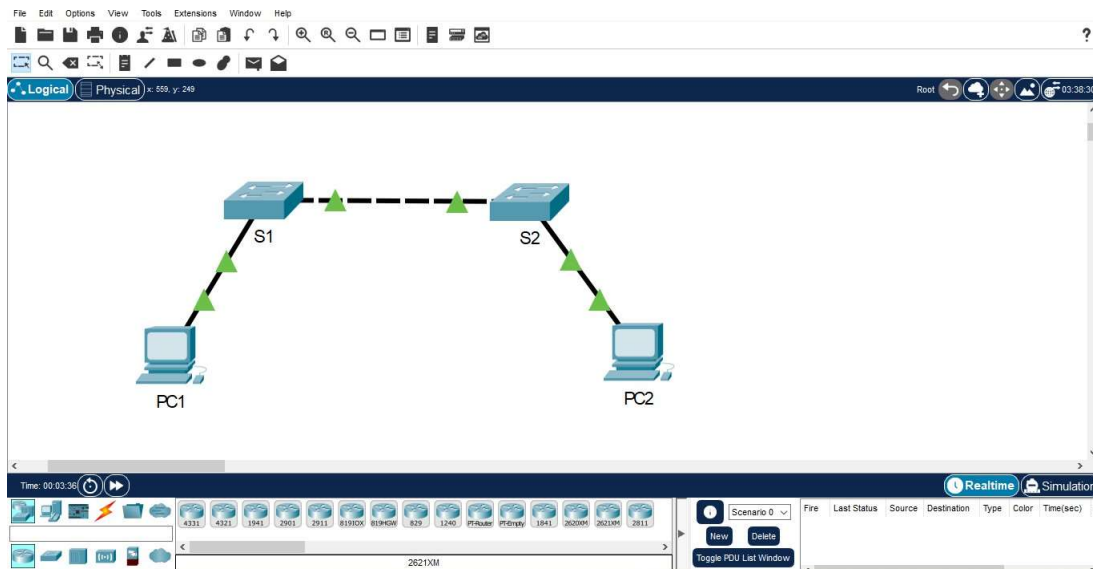
```
enable
config t
hostname S2
line console 0
  password letmein
  login
enable password c1$c0
enable secret itsasecret
banner motd $any text here$
service password-encryption
exit
copy running-config startup-config
```

The objective of this lab is that the students get acquainted with connectivity challenges and learn to perform configurations on PCs and switches.

Activity 3: Configure the PCs

In this activity, you will first create a basic switch configuration. Then, you will implement basic connectivity by configuring IP addressing on switches and PCs. When the IP addressing configuration is complete, you will use various **show** commands to verify the configuration and use the **ping** command to verify basic connectivity between devices.

Topology



Addressing Table

Device	Interface	IP Address	Subnet Mask
S1	VLAN 1	192.168.1.253	255.255.255.0
S2	VLAN 1	192.168.1.254	255.255.255.0
PC1	NIC	192.168.1.1	255.255.255.0
PC2	NIC	192.168.1.2	255.255.255.0

Step 1: Configure both PCs with IP addresses.

- Click PC1 and then click the Desktop tab.
- Click IP Configuration. In the Addressing Table above, you can see that the IP address for PC1 is 192.168.1.1 and the subnet mask is 255.255.255.0. Enter this information for PC1 in the IP Configuration window.
- Repeat steps 1a and 1b for PC2.

Step 2: Test connectivity to switches.

- Click PC1. Close the IP Configuration window if it is still open. In the Desktop tab, click Command Prompt.
- Type the **ping** command and the IP address for S1 and press Enter.

Packet Tracer PC Command Line 1.0

PC> **ping 192.168.1.253**

Question:

Were you successful? Explain.

Activity 4: Configure the Switch Management Interface

Configure S1 and S2 with an IP address.

Configure S1 with an IP address.

Switches can be used as plug-and-play devices. This means that they do not need to be configured for them to work. Switches forward information from one port to another based on MAC addresses.

Question:

If this is the case, why would we configure it with an IP address?

Use the following commands to configure S1 with an IP address.

```
S1# configure terminal
```

Enter configuration commands, one per line. End with CNTL/Z.

```
S1(config)# interface vlan 1
```

```
S1(config-if)# ip address 192.168.1.253 255.255.255.0
```

```
S1(config-if)# no shutdown
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to  
up  
S1(config-if)#  
S1(config-if)# exit
```

Question:

Why do you enter the **no shutdown** command?

Configure S2 with an IP address.

Use the information in the Addressing Table to configure S2 with an IP address.

Verify the IP address configuration on S1 and S2.

Use the **show ip interface brief** command to display the IP address and status of all the switch ports and interfaces. You can also use the **show running-config** command.

Save configurations for S1 and S2 to NVRAM.

Question:

Which command is used to save the configuration file in RAM to NVRAM?

Verify network connectivity.

Network connectivity can be verified using the **ping** command. It is very important that connectivity exists throughout the network. Corrective action must be taken if there is a failure. Ping S1 and S2 from PC1 and PC2.

- Click PC1 and then click the Desktop tab.
- Click Command Prompt.
- Ping the IP address for PC2.
- Ping the IP address for S1.
- Ping the IP address for S2.

Note: You can also use the **ping** command on the switch CLI and on PC2.

All pings should be successful. If your first ping result is 80%, try again. It should now be 100%. You will learn why a ping may sometimes fail the first time later in your studies. If you are unable to ping any of the devices, recheck your configuration for errors.

End of Document

Script

Switch S1

```
enable
configure terminal
hostname S1
line con 0
  pass cisco
  login
username cisco1 password cisco2

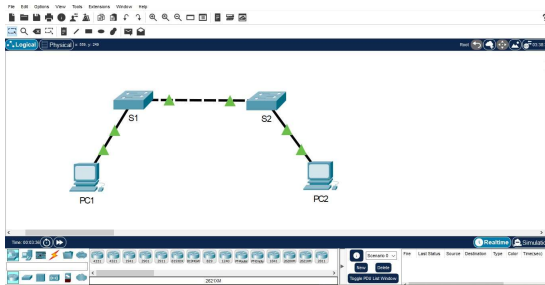
interface vlan 1
  ip address 192.168.1.253 255.255.255.0
  no shutdown
enable secret class
banner motd $ Authorized access only. Violators will be prosecuted to the
full extent of the law.$
exit
copy running-config startup-config
```

Switch S2

```
enable
configure terminal
hostname S2
line con 0
  pass cisco
  login
interface vlan 1
  ip address 192.168.1.254 255.255.255.0
  no shutdown
enable secret class
banner motd $ Authorized access only. Violators will be prosecuted to the
full extent of the law.$
exit
copy running-config startup-config
```

6. Graded Lab Tasks

Perform a Basic Configuration on S1 and S2.



Addressing Table

Device	Interface	IP Address	Subnet Mask
S1	VLAN 1	192.168.1.253	255.255.255.0
S2	VLAN 1	192.168.1.254	255.255.255.0
PC1	NIC	192.168.1.1	255.255.255.0
PC2	NIC	192.168.1.2	255.255.255.0

Perform a Basic Configuration on S1 and S2

1. Configure S1 with a hostname.
2. Configure the console and encrypted privileged EXEC mode passwords.
3. Verify the password configurations for S1.
4. How can you verify that both passwords were configured correctly?
5. Configure an MOTD banner.
6. Save the configuration file to NVRAM.
7. Repeat Steps 1 to 5 for S2

LAB 09

Dynamic Host Configuration Protocol (DHCP) Configuration

Objective:

The objective of this lab is that the students would learn how to Configure DHCP. This lab explains how to configure and verify a DHCP server on routers in detail through a packet tracer example. Learn how to use a router as a DHCP server.

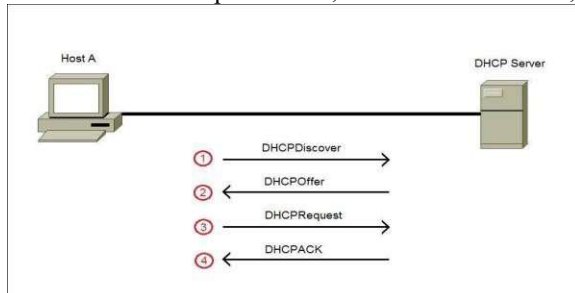
Activity Outcomes:

The activities provide hands - on practice with the following topics

- Configure a Router as a DHCP Server
- Verify DHCP and Connectivity

Useful Concepts

Dynamic Host Configuration Protocol (DHCP) is an application layer protocol used to distribute network configuration parameters, such as IP addresses, subnet masks, default gateways, etc. to hosts on a TCP/IP network. Assigning network parameters using DHCP reduces the amount of work of a network administrator, since there is no need to statically configure parameters on each device. DHCP employs a client-server architecture; a DHCP client is configured to request network parameters from a DHCP server. A DHCP server is configured with a pool of available IP addresses and assigns one of them to the DHCP client. Besides IP addresses, a DHCP server can provide some additional network parameters, such as: subnet mask, default gateway, domain name, and DNS server.



As you can see from the picture above, a DHCP client goes through the four-step process:

1. A DHCP client sends a broadcast packet (DHCPDiscover) to discover DHCP servers on the network.
2. The DHCP server receives the DHCPDiscover packet and respond with a DHCPOffer packet, offering IP addressing information to the DHCP client.
3. If there is more than one DHCP server on the network segment and the DHCP client receives more than one DHCPOffer packets, the client will accept the first DHCPOffer packet. The DHCP client responds by broadcasting a DHCPRequest packet, requesting network parameters from the DHCP server.
4. The DHCP server approves the lease with a DHCPACK (Acknowledgement) packet. The packet includes the lease duration and other configuration information.

A dedicated DHCP server is scalable and relatively easy to manage but it can be costly to have one at every location in a network. However, a Cisco router can be configured to provide DHCP services without the need for a dedicated server. As the network technician for your company, you have been assigned the task of configuring a Cisco router as a DHCP server. You are also required to configure the edge router as a DHCP client so that it receives an IP address from the ISP network.

For more details, please check the below reference.

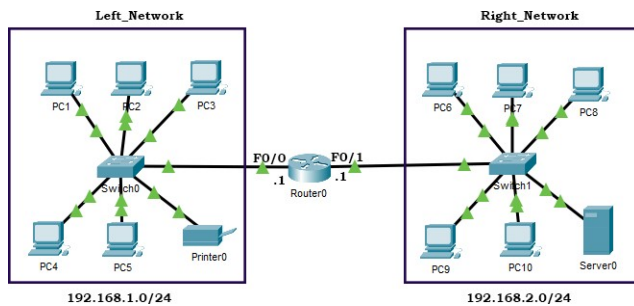
Reference

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- <https://geek-university.com/dynamic-host-configuration-protocol-dhcp/>
- <https://www.computernetworkingnotes.com/ccna-study-guide/how-to-configure-dhcp-server-on-cisco-routers.html>

1. Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>45</i>	<i>High</i>	<i>CLO-6</i>
<i>2</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>

Activity 1



In this lab, there are two networks: **Left_network** and **Right_Network**. Both networks are connected to the router's **Fast Ethernet 0/0** and **0/1** interfaces, respectively. Now our objective is to configure the following IP configuration on both networks.

Configuration	Left_Network	Right_Network
IP addresses	192.168.1.0 to 192.168.1.255	192.168.2.0 to 192.168.2.255
Available IP addresses for hosts	192.168.1.10 to 192.168.1.254	192.168.2.10 to 192.168.2.254
Subnet mask	255.255.255.0	255.255.255.0
Default Gateway	192.168.1.1	192.168.2.1
DNS Server	192.168.1.2	192.168.2.2
TFTP Server	192.168.1.3	192.168.2.3
Reserved	192.168.1.4 to 192.168.1.10	192.168.2.4 to 192.168.2.10

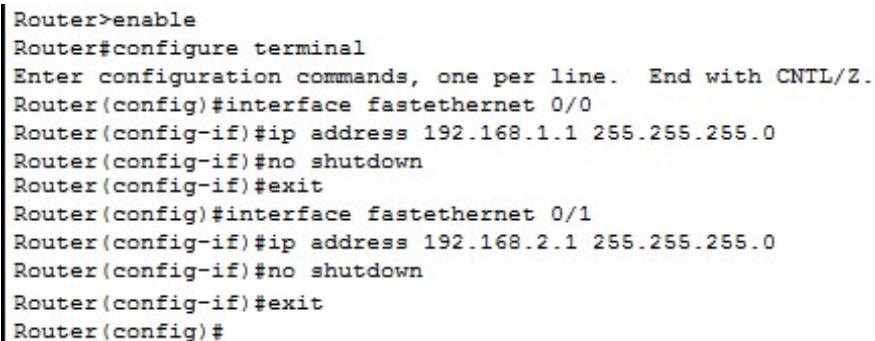
Configuring IP configuration on the router

A router connects different networks. If a router is connected to a network, hosts of the network use the router as the default gateway to reach the host of other networks. In our example, since the **Left_Network** and **Right_Network** are respectively connected to the **Fast Ethernet 0/0** and **0/1**

interfaces of the router, both networks will use the IP addresses of their respective interfaces as the default gateway IPs. In simple terms, **Fast Ethernet 0/0** and **Fast Ethernet 0/1** of the router are the default gateways of the **Left_Network** and **Right_Network** respectively. Before configuring the router to act as a DHCP server, we have to configure and enable these interfaces. To configure and enable these interfaces, access the command prompt of the router, and execute the following commands.

```
Router>enable
Router# configure terminal
Router(config)# interface FastEthernet 0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# interface FastEthernet 0/1
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
```

The following image shows the above commands on the packet tracer.



```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastethernet 0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface fastethernet 0/1
Router(config-if)#ip address 192.168.2.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

Step 1: Configuring DHCP server on the router

For each network that will obtain IP configuration from the DHCP server, we have to create and configure a DHCP pool on the router. In our example, we have two networks, so we have to create two DHCP pools, one for each network.

Use the following commands to create and configure a DHCP pool for the **Left_Network**.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip dhcp excluded-address 192.168.1.0 192.168.1.10
Router(config)#ip dhcp pool Left_Network
Router(dhcp-config)#default-router 192.168.1.1
Router(dhcp-config)#dns-server 192.168.1.2
Router(dhcp-config)#option 150 ip 192.168.1.3
Router(dhcp-config)#network 192.168.1.0 255.255.255.0
Router(dhcp-config)#exit
```

The following table describes the above commands.

Command	Description
<code>ip dhcp excluded-address 192.168.1.0 192.168.1.10</code>	This command tells the DHCP server not to assign the addresses from 192.168.1.0 to 192.168.1.10 to DHCP clients.
<code>ip dhcp pool Left_Network</code>	This command creates a DHCP pool named, Left_Network and changes command mode to DHCP pool configuration mode.
<code>default-router 192.168.1.1</code>	This command assigns the default gateway to clients of this DHCP pool.
<code>dns-server 192.168.1.2</code>	This command sets a primary DNS server for the clients.
<code>option 150 ip 192.168.1.3</code>	This command provides the IP address of the TFTP server to the clients.
<code>network 192.168.1.0 255.255.255.0</code>	This command specifies the range of IP addresses for the pool.
<code>exit</code>	This command exits DHCP pool configuration mode.

Create and configure a DHCP pool for the **Right_Network** using the same commands as shown below.

```
Router(config)#ip dhcp excluded-address 192.168.2.0 192.168.2.10
Router(config)#ip dhcp pool Right_Network
Router(dhcp-config)#default-router 192.168.2.1
```

```
Router(dhcp-config)#dns-server 192.168.2.2
Router(dhcp-config)#option 150 ip 192.168.2.3
Router(dhcp-config)#network 192.168.2.0 255.255.255.0
Router(dhcp-config)#exit
Router(config)#
```

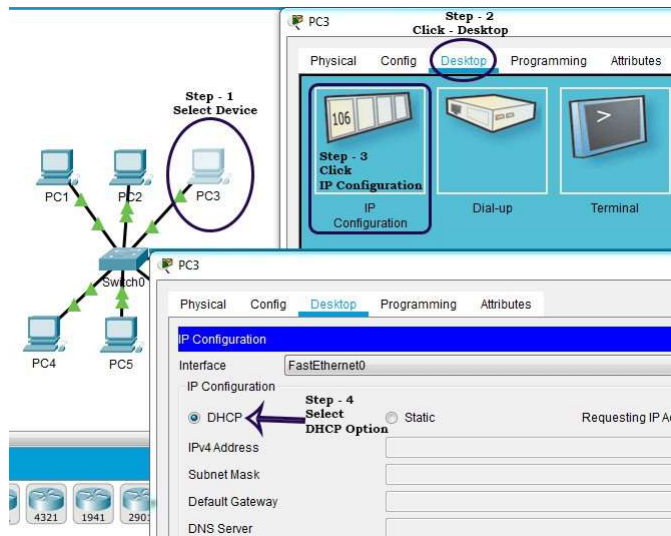
The following image shows how to execute the above commands on the router.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip dhcp excluded-address 192.168.1.0 192.168.1.10
Router(config)#ip dhcp pool Left_Network
Router(dhcp-config)#default-router 192.168.1.1
Router(dhcp-config)#dns-server 192.168.1.2
Router(dhcp-config)#option 150 ip 192.168.1.3
Router(dhcp-config)#network 192.168.1.0 255.255.255.0
Router(dhcp-config)#exit
Router(config)#ip dhcp excluded-address 192.168.2.0 192.168.2.10
Router(config)#ip dhcp pool Right_Network
Router(dhcp-config)#default-router 192.168.2.1
Router(dhcp-config)#dns-server 192.168.2.2
Router(dhcp-config)#option 150 ip 192.168.2.3
Router(dhcp-config)#network 192.168.2.0 255.255.255.0
Router(dhcp-config)#exit
Router(config)#
```

Configuring DHCP clients

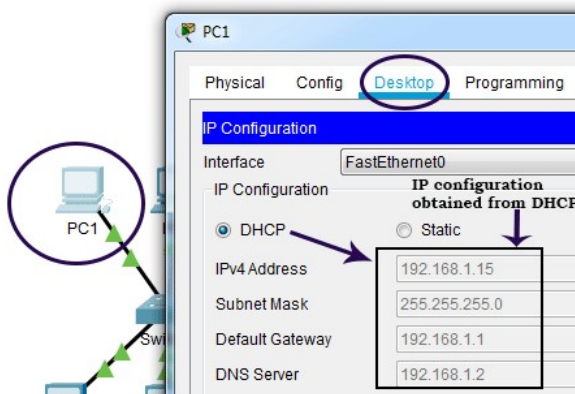
To configure a device as a DHCP client, change its IP configuration option to **DHCP**. To do this, click the device. In opened Windows, click the **IP configuration option** from the **Desktop** menu and set the **IP configuration option** to **DHCP**.

The following image shows the above procedure.

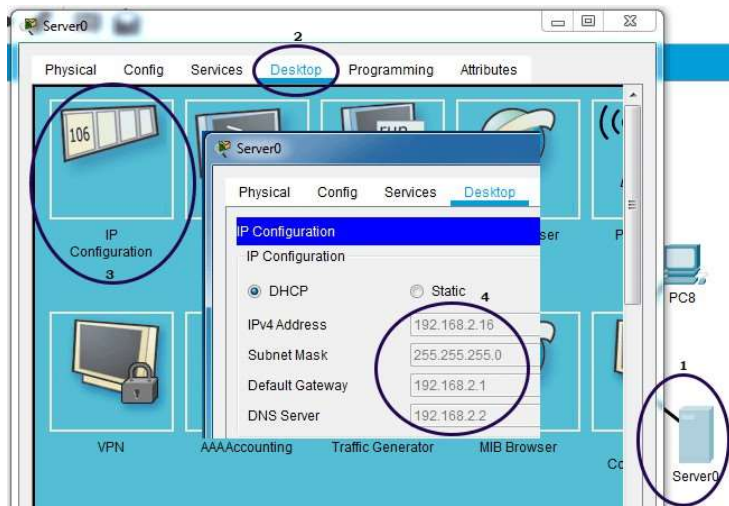


Verifying DHCP clients

To verify that the client has obtained IP configuration from the DHCP server, you can check the IP configuration option of the client again. For example, the following image shows how to verify this on a host of the **Left_Network**.



The following image shows how to verify this on a host of the **Right_Network**.



Step 1: Verifying the DHCP Server

To verify that the DHCP server is working properly and to see the IP addresses that are provided by the DHCP server, run the following command in **privileged-exec mode**.

```
#ip dhcp binding
```

The following image shows the output of this command.

```

Router#show ip dhcp binding
IP address      Client-ID/      Lease expiration    Type
                Hardware address
192.168.1.11    0002.4A7B.44EA  --                  Automatic
192.168.1.12    00D0.BC7C.BAB3  --                  Automatic
192.168.1.13    00E0.F76A.90C1  --                  Automatic
192.168.1.14    00D0.5884.22DC  --                  Automatic
192.168.1.15    00D0.BC31.67C3  --                  Automatic
192.168.1.16    0090.2B27.E90B  --                  Automatic
192.168.2.11    000C.8564.9677  --                  Automatic
192.168.2.12    0010.1171.4322  --                  Automatic
192.168.2.13    0004.9A93.BAA7  --                  Automatic
192.168.2.15    0030.F282.C8E7  --                  Automatic
192.168.2.14    0001.63D3.74B4  --                  Automatic
192.168.2.16    0000.0C8A.1542  --                  Automatic
Router#

```

To view detailed information about a specific DHCP pool, use the following command.

```
#show ip dhcp pool [pool-name]
```

For example, the following commands list the detailed information about the DHCP pools:

Left_Network and **Right_Network**, respectively.

```
#show ip dhcp pool Left_Network
#show ip dhcp pool Right_Network
```

The following image shows the output of the above commands.

```
Router#show ip dhcp pool Left_Network
Pool Left_Network :
Utilization mark (high/low)      : 100 / 0
Subnet size (first/next)          : 0 / 0
Total addresses                   : 254
Leased addresses                  : 6
Excluded addresses                : 2
Pending event                    : none

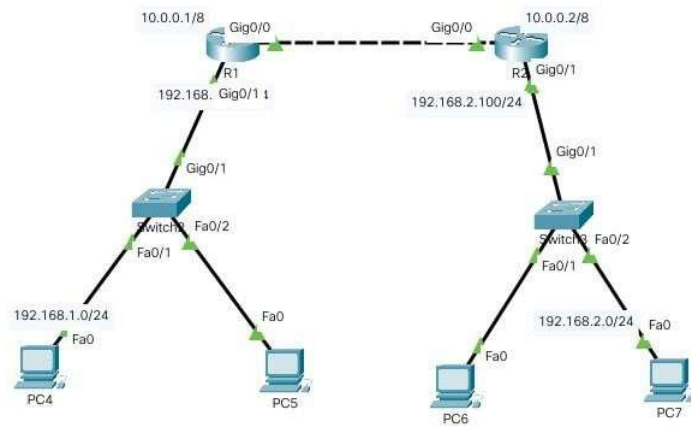
1 subnet is currently in the pool
Current index      IP address range      Leased/Excluded/Total
192.168.1.1        192.168.1.1 - 192.168.1.254    6 / 2 / 254
Router#show ip dhcp pool Right_Network
Pool Right_Network :
Utilization mark (high/low)      : 100 / 0
Subnet size (first/next)          : 0 / 0
Total addresses                   : 254
Leased addresses                  : 6
Excluded addresses                : 2
Pending event                    : none

1 subnet is currently in the pool
Current index      IP address range      Leased/Excluded/Total
192.168.2.1        192.168.2.1 - 192.168.2.254    6 / 2 / 254
Router#
```

End of document

3.Graded Lab Tasks

Configure the DHCP server on the two routers. In our network, we must exclude the IP addresses on the R1 and R2 interfaces, together with the other 9 addresses to be used for static assignment to other devices like servers and device management interfaces. Create two pools POOL-A on router R1 and POOL-2 on router R2. This includes configuring the network addresses, default gateway, and the IP address of the DNS server.



LAB 10

Subnet an IPv4 Network

Objective:

The objective of this lab is that the students get acquainted with Subnetting concepts in IPv4 Network.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Design an IPv4 Network Subnetting Scheme
- Configure the Devices
- Test and Troubleshoot the Network

Useful Concepts

An IP address is an address used in order to uniquely identify a device on an IP network. The address is made up of 32 binary bits, which can be divisible into a network portion and host portion with the help of a subnet mask. The 32 binary bits are broken into four octets (1 octet = 8 bits). Each octet is converted to decimal and separated by a period (dot). For this reason, an IP address is said to be expressed in dotted decimal format (for example, 172.16.81.100). The value in each octet ranges from 0 to 255 decimal, or 00000000 - 11111111 binary.

Here is how binary octets convert to decimal: The right most bit, or least significant bit, of an octet holds a value of 2^0 . The bit just to the left of that holds a value of 2^1 . This continues until the left-most bit, or most significant bit, which holds a value of 2^7 . So if all binary bits are a one, the decimal equivalent would be 255 as shown here:

1	1	1	1	1	1	1	1
128	64	32	16	8	4	2	1

(128+64+32+16+8+4+2+1=255)

Here is a sample octet conversion when not all of the bits are set to 1.

```
0 1 0 0 0 0 0 1
0 64 0 0 0 0 0 1 (0+64+0+0+0+0+0+1=65)
```

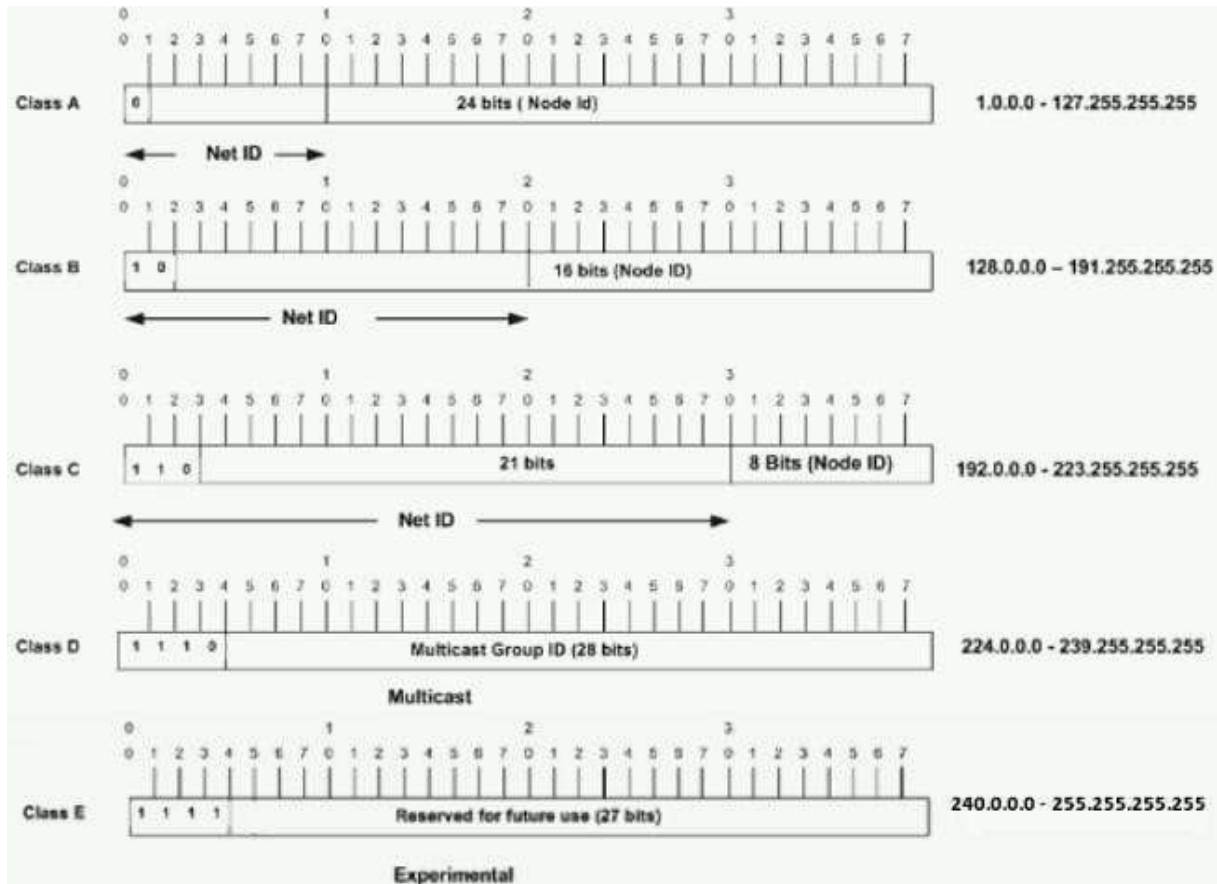
And this sample shows an IP address represented in both binary and decimal.

```
10.      1.      23.      19 (decimal)
00001010.00000001.00010111.00010011 (binary)
```

These octets are broken down to provide an addressing scheme that can accommodate large and small networks. There are five different classes of networks, A to E. This document focuses on classes A to C, since classes D and E are reserved and discussion of them is beyond the scope of this document.

Given an IP address, its class can be determined from the three high-order bits (the three left-most bits in the first octet). [Figure 1](#) shows the significance in the three high order bits and the range of addresses that fall into each class. For informational purposes, Class D and Class E

addresses are also shown.



A network mask helps you know which portion of the address identifies the network and which portion of the address identifies the node. Class A, B, and C networks have default masks, also known as natural masks, as shown here:

Class A: 255.0.0.0
Class B: 255.255.0.0
Class C: 255.255.255.0

An IP address on a Class A network that has not been subnetted would have an address/mask pair similar to: 8.20.15.1 255.0.0.0. In order to see how the mask helps you identify the network and node parts of the address, convert the address and mask to binary numbers.

```
8.20.15.1 = 00001000.00010100.00001111.00000001
255.0.0.0 = 11111111.00000000.00000000.00000000
```

Once you have the address and the mask represented in binary, then identification of the network and host ID is easier. Any address bits which have corresponding mask bits set to 1 represent the network ID. Any address bits that have corresponding mask bits set to 0 represent the node ID.

```
8.20.15.1 = 00001000.00010100.00001111.00000001
255.0.0.0 = 11111111.00000000.00000000.00000000
-----
          net id |          host id
```

```
netid = 00001000 = 8
hostid = 00010100.00001111.00000001 = 20.15.1
```

Subnetting allows you to create multiple logical networks that exist within a single Class A, B, or C network. If you do not subnet, you are only able to use one network from your Class A, B, or C network, which is unrealistic.

Each data link on a network must have a unique network ID, with every node on that link being a member of the same network. If you break a major network (Class A, B, or C) into smaller subnetworks, it allows you to create a network of interconnecting subnetworks. Each data link on this network would then have a unique network/subnetwork ID. Any device, or gateway, that connects n networks/subnetworks has n distinct IP addresses, one for each network / subnetwork that it interconnects.

In order to subnet a network, extend the natural mask with some of the bits from the host ID portion of the address in order to create a subnetwork ID. For example, given a Class C network of 204.17.5.0 which has a natural mask of 255.255.255.0, you can create subnets in this manner:

```
204.17.5.0 -      11001100.00010001.00000101.00000000
255.255.255.224 - 11111111.11111111.11111111.11100000
```

----- | sub | -----

By extending the mask to be 255.255.255.224, you have taken three bits (indicated by "sub") from the original host portion of the address and used them to make subnets. With these three bits, it is possible to create eight subnets. With the remaining five host ID bits, each subnet can have up to 32 host addresses, 30 of which can actually be assigned to a device *since host ids of all zeros or all ones are not allowed* (it is very important to remember this). So, with this in mind, these subnets have been created.

204.17.5.0	255.255.255.224	host address range 1 to 30
204.17.5.32	255.255.255.224	host address range 33 to 62
204.17.5.64	255.255.255.224	host address range 65 to 94
204.17.5.96	255.255.255.224	host address range 97 to 126
204.17.5.128	255.255.255.224	host address range 129 to 158
204.17.5.160	255.255.255.224	host address range 161 to 190
204.17.5.192	255.255.255.224	host address range 193 to 222
204.17.5.224	255.255.255.224	host address range 225 to 254

For more details, please check the below reference.

Reference:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- <https://www.studocu.com/en-ca/document/western-community-college/network/1155-packet-tracer-subnet-an-ipv4-network/21586427>
- <https://www.ciscopress.com/store/ip-subnetting-from-beginning-to-mastery-livelessons-9780135497777>
- <https://ipcisco.com/lesson/ip-subnetting-and-subnetting-examples/>
- <https://www.computernetworkingnotes.com/ccna-study-guide/subnetting-tutorial-subnetting-explained-with-examples.html>

1. Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>20</i>	<i>Low</i>	<i>CLO-6</i>

<i>2</i>	<i>25</i>	<i>Low</i>	<i>CLO-6</i>
<i>3</i>	<i>20</i>	<i>Medium</i>	<i>CLO-6</i>

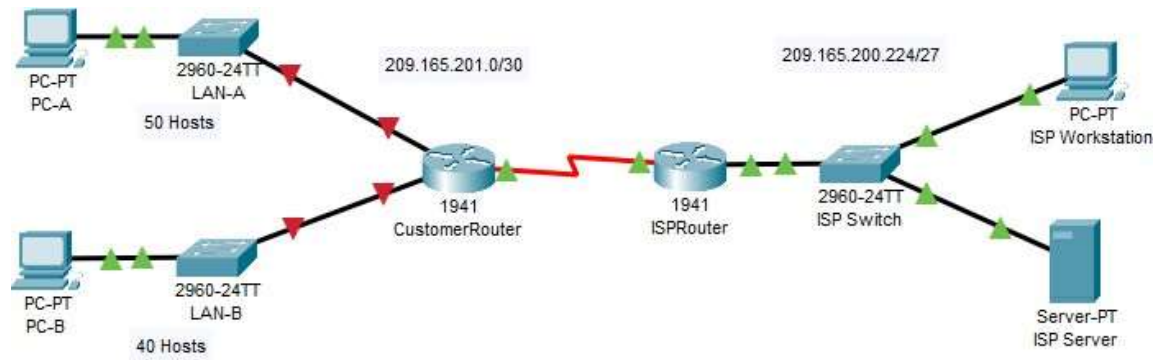
Activity 1

In this activity, you will subnet the Customer network into multiple subnets. The subnet scheme should be based on the number of host computers required in each subnet, as well as other network considerations, like future network host expansion.

After you have created a subnetting scheme and completed the table by filling in the missing host and interface IP addresses, you will configure the host PCs, switches and router interfaces.

After the network devices and host PCs have been configured, you will use the **ping** command to test for network connectivity.

Topology



Addressing Table

Device	Interface	IP Address	Subnet Mask	Default Gateway
Customer Router-PT CustomerRouter	G0/0	192.168.0.1	255.255.255.192	N/A Nil
	G0/1	192.168.0.65	255.255.255.192	
	S0/1/0	209.165.201.2	255.255.255.252	
LAN-A Switch	VLAN1	192.168.0.2	255.255.255.192	192.168.0.1
LAN-B Switch	VLAN1	192.168.0.66	255.255.255.192	192.168.0.65
PC-A	NIC	192.168.0.62	255.255.255.192	192.168.0.1
PC-B	NIC	192.168.0.126	255.255.255.192	192.168.0.65
ISPRouter ISPRouter	G0/0	209.165.200.225	255.255.255.224	N/A Nil
	S0/1/0	209.165.201.1	255.255.255.252	
ISPSwitch	VLAN1	209.165.200.226	255.255.255.224	209.165.200.225
ISP Workstation	NIC	209.165.200.235	255.255.255.224	209.165.200.225
ISP Server	NIC	209.165.200.240	255.255.255.224	209.165.200.225

Blank Line: No additional information

Activity 1: Subnet the Assigned Network

Create a subnetting scheme that meets the required number of subnets and required number of host addresses.

In this scenario, you are a network technician assigned to install a new network for a customer. You must create multiple subnets out of the 192.168.0.0/24 network address space to meet the following requirements:

The first subnet is the LAN-A network. You need a minimum of 50 host IP addresses.

The second subnet is the LAN-B network. You need a minimum of 40 host IP addresses.

You also need at least two additional unused subnets for future network expansion.

Note: Variable length subnet masks will not be used. All of the device subnet masks should be the same length.

Answer the following questions to help create a subnetting scheme that meets the stated network requirements:

Questions:

How many host addresses are needed in the largest required subnet?

Type your answers here.

50

What is the minimum number of subnets required?

Type your answers here.

The requirements stated above specify two company networks plus two additional networks for future expansion. So, the answer is a minimum of four networks.

The network that you are tasked to subnet is 192.168.0.0/24. What is the /24 subnet mask in binary?

Type your answers here.

11111111.11111111.11111111.00000000

The subnet mask is made up of two portions, the network portion, and the host portion. This is represented in the binary by the ones and the zeros in the subnet mask.

Questions:

In the network mask, what do the ones represent?

Type your answers here.

The ones represent the network portion.

In the network mask, what do the zeros represent?

Type your answers here.

The zeroes represent the host portion.

To subnet a network, bits from the host portion of the original network mask are changed into subnet bits. The number of subnet bits defines the number of subnets.

Question:

Given each of the possible subnet masks depicted in the following binary format, how many subnets and how many hosts are created in each example?

Hint: Remember that the number of host bits (to the power of 2) defines the number of hosts per subnet (minus 2), and the number of subnet bits (to the power of two) defines the number of subnets. The subnet bits (shown in bold) are the bits that have been borrowed beyond the original network mask of /24. The /24 is the prefix notation and corresponds to a dotted decimal mask of 255.255.255.0.

(/25) 11111111.11111111.11111111.**10000000**

Dotted decimal subnet mask equivalent:

Type your answers here.

255.255.255.128

Number of subnets? Number of hosts?

Type your answers here.

Two subnets (2^1) and 128 hosts (2^7) – 2 = 126 hosts per subnet

(/26) 11111111.11111111.11111111.**11000000**

Dotted decimal subnet mask equivalent:

Type your answers here.

255.255.255.192

Number of subnets? Number of hosts?

Type your answers here.

Four subnets (2^2) and 64 hosts (2^6) – 2 = 62 hosts per subnet

(/27) 11111111.11111111.11111111.**11100000**

Dotted decimal subnet mask equivalent:

Type your answers here.

255.255.255.224

Number of subnets? Number of hosts?

Type your answers here.

Eight subnets (2^3) and 32 hosts (2^5) – 2 = 30 hosts per subnet

(/28) 11111111.11111111.11111111.11110000

Dotted decimal subnet mask equivalent:

Type your answers here.

255.255.255.240

Number of subnets? Number of hosts?

Type your answers here.

Sixteen subnets (2^4) and 16 hosts (2^4) – 2 = 14 hosts per subnet

(/29) 11111111.11111111.11111111.11111000

Dotted decimal subnet mask equivalent:

Type your answers here.

255.255.255.248

Number of subnets? Number of hosts?

Type your answers here.

Thirty two subnets (2^5) and 8 hosts (2^3) – 2 = 6 hosts per subnet

(/30) 11111111.11111111.11111111.11111100

Dotted decimal subnet mask equivalent:

Type your answers here.

255.255.255.252

Number of subnets? Number of hosts?

Type your answers here.

Sixty four subnets (2^6) and 4 hosts (2^2) – 2 = 2 hosts per subnet

Considering your answers above, which subnet masks meet the required number of minimum host addresses?

Type your answers here.

/25, /26

Considering your answers above, which subnet masks meets the minimum number of subnets required?

Type your answers here.

/26, /27, /28, /29, /30 will give the required number of subnets.

Considering your answers above, which subnet mask meets both the required minimum number of hosts and the minimum number of subnets required?

Type your answers here.

/26 will give you the four subnets that are required, and 62 hosts per subnet, which is greater than the 50 hosts required for the first subnet.

When you have determined which subnet mask meets all of the stated network requirements, derive each of the subnets. List the subnets from first to last in the table. Remember that the first subnet is 192.168.0.0 with the chosen subnet mask.

Subnet Address	Prefix	Subnet Mask
192.168.0.0	/26	255.255.255.192
192.168.0.64	/26	255.255.255.192
192.168.0.128	/26	255.255.255.192
192.168.0.192	/26	255.255.255.192

Blank Line. No additional information

Fill in the missing IP addresses in the Addressing Table

Assign IP addresses based on the following criteria: Use the ISP Network settings as an example.

Assign the first subnet to LAN-A.

Use the first host address for the CustomerRouter interface connected to LAN-A switch.

Use the second host address for the LAN-A switch. Make sure to assign a default gateway address for the switch.

Use the last host address for PC-A. Make sure to assign a default gateway address for the PC.

Assign the second subnet to LAN-B.

Use the first host address for the CustomerRouter interface connected to LAN-B switch.

Use the second host address for the LAN-B switch. Make sure to assign a default gateway address for the switch.

Use the last host address for PC-B. Make sure to assign a default gateway address for the PC.

Activity 2: Configure the Devices

Configure basic settings on the PCs, switches, and router. Refer to the Addressing Table for device names and address information.

Configure CustomerRouter.

Set the enable secret password on CustomerRouter to **Class123**

Set the console login password to **Cisco123**.

Configure **CustomerRouter** as the hostname for the router.

Configure the G0/0 and G0/1 interfaces with IP addresses and subnet masks, and then enable them.

Save the running configuration to the startup configuration file.

Configure the two customer LAN switches.

Configure the IP addresses on interface VLAN 1 on the two customer LAN switches. Make sure to configure the correct default gateway on each switch.

Configure the PC interfaces.

Configure the IP address, subnet mask, and default gateway settings on **PC-A** and **PC-B**.

Activity 3: Test and Troubleshoot the Network

In Part 3, you will use the **ping** command to test network connectivity.

Determine if PC-A can communicate with its default gateway. Do you get a reply?

Type your answers here.

If you answered no, then you should go back and check your IP address and subnet mask configurations, and ensure that the default gateways have been correctly configured on PC-A and PC-B.

Determine if PC-B can communicate with its default gateway. Do you get a reply?

Type your answers here.

If you answered no, then you should go back and check your IP address and subnet mask configurations, and ensure that the default gateways have been correctly configured on PC-A and PC-B.

Determine if PC-A can communicate with PC-B. Do you get a reply?

Type your answers here.

If you answered no, then you should go back and check your IP address and subnet mask configurations, and ensure that the default gateways have been correctly configured on PC-A and PC-B.

Device Configs

CustomerRouter

Router-PT#

```
enable
configure terminal
hostname CustomerRouter
enable secret Class123
line con 0
  password Cisco123
  login
interface GigabitEthernet0/0
  ip address 192.168.0.1 255.255.255.192
  no shutdown
interface GigabitEthernet0/1
  ip address 192.168.0.65 255.255.255.192
  no shutdown
interface Serial0/1/0
  ip address 209.165.201.2 255.255.255.252
  no shutdown
end
```

LAN-A

Switch#

```
enable
configure terminal
interface Vlan1
  ip address 192.168.0.2 255.255.255.192
  no shutdown
ip default-gateway 192.168.0.1
end
```

LAN-B

Switch#

```
enable
configure terminal
interface Vlan1
  ip address 192.168.0.66 255.255.255.192
  no shutdown
ip default-gateway 192.168.0.65
end
```

PC-A

IP address: 192.168.0.63 /26
Default gateway: 192.168.0.1

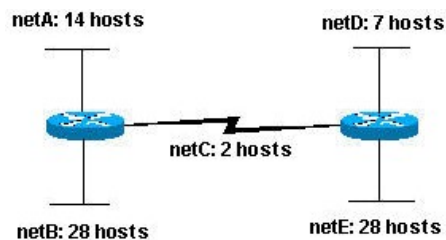
PC-B

IP address: 192.168.0.126 /26
Default gateway: 192.168.0.65

2. Graded Lab Tasks

Given the Class C network of 204.15.5.0/24, subnet the network in order to create the network in [Figure 3](#) with the host requirements shown.

Figure 3



Looking at the network shown in [Figure 3](#), you can see that you are required to create five subnets. The largest subnet must support 28 host addresses. Is this possible with a Class C network? And if so, then how?

You can start by looking at the subnet requirement. In order to create the five needed subnets you would need to use three bits from the Class C host bits. Two bits would only allow you four subnets (2^2).

Since you need three subnet bits, that leaves you with five bits for the host portion of the address. How many hosts does this support? $2^5 = 32$ (30 usable). This meets the requirement.

Therefore you have determined that it is possible to create this network with a Class C network.
An example of how you might assign the subnetworks is:

netA: 204.15.5.0/27	host address range 1 to 30
netB: 204.15.5.32/27	host address range 33 to 62
netC: 204.15.5.64/27	host address range 65 to 94
netD: 204.15.5.96/27	host address range 97 to 126
netE: 204.15.5.128/27	host address range 129 to 158

Static Routing

Objective:

The objective of this lab is that the students get acquainted with configuring static routing on routers. Learn how to create and manage static routes on routers through a packet tracer example.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Creating the Network Topology
- Configure PC Hosts, switches, routers
- Static route configuration

Useful Concepts

The static or manual method to add entries to the routing table is known as **static or manual routing**. In static or manual routing, the administrator manually adds entries to the routing table. The administrator creates a virtual map of all routes and manually adds them to each router's routing table. Routes that are manually added by an administrator to the routing table are known as **static routes**. In other words, a **static route** is a route that you manually add to the router's routing table.

Static routing allows the administrator to save money. In static routing, the router does not use CPU and RAM to learn the routes and calculate the best route to each destination. Since static routing does not put overhead on the router's CPU and RAM, the administrator can use a cheaper router. In static routing, routers do not exchange routing information. Since routers do not exchange routing information, they save the network bandwidth. If in a network, routers are connected through a paid WAN link, static routing can reduce the bill amount that the network pays for WAN connectivity. Static routing is the safest method of routing. The administrator manually adds routes for authorized networks. Since the administrator manually decides which network can reach which network, a network can only access the authorized network. In static routing, since the administrator adds and manages all routes, the administrator must have in-depth

knowledge of the internetwork. To add all routes correctly, the administrator has to learn how each router is connected to the network.

The process of adding each route on each router is tedious. If the administrator changes the location of a router in the network, the administrator has to update routing information on all routers manually. If a link goes up or down, the administrator has to manually update this information on all routers. On a flipping link, this will cause a huge problem. If you have a backup route, the router doesn't automatically switch to the backup route if the main route fails. The administrator must have to reconfigure the router to use the backup route.

For more details, please check the below reference.

Reference:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- <https://www.computernetworkingnotes.com/ccna-study-guide/static-routing-configuration-guide-with-examples.html>.
- <https://ipcisco.com/lesson/static-route-configuration-on-cisco-routers-ccna/>
- <https://www.computernetworkingnotes.com/ccna-study-guide/static-routes-and-static-routing-explained.html>
- <https://www.ciscopress.com/articles/article.asp?p=2180209&seqNum=4>

1. Solved Lab Activities

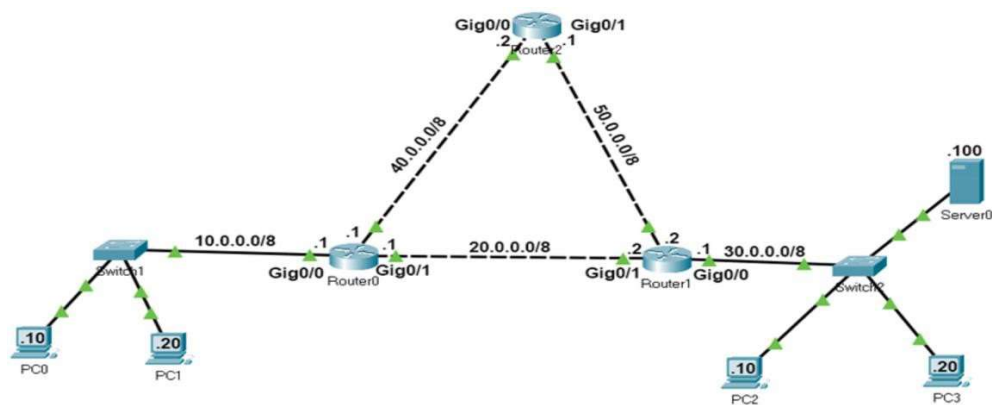
<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>10</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>15</i>	<i>Low</i>	<i>CLO-6</i>

Activity 1

Create a packet tracer lab or download it from here [Packet Tracer Lab with Initial IP Configuration \(https://www.computernetworkingnotes.com/ccna-study-guide/static-routing-configuration-guide-with-examples.html\)](https://www.computernetworkingnotes.com/ccna-study-guide/static-routing-configuration-guide-with-examples.html).

In this activity, you will use various **show** commands to display the current state of the router. You will then use the Addressing Table to configure router Ethernet interfaces. Finally, you will use commands to verify and test your configurations.

Topology



In this lab, each network has two routes to reach. We will configure one route as the main route and another route as the backup route. If the link bandwidth of all routes is the same, we use the route that has the least number of routers as the main route. If the link bandwidth and the number of routers are the same, we can use any route as the main route and another route as the backup route. If we specify two routes for the same destination, the router automatically selects the best route for the destination and adds the route to the routing table. If you manually want to select a route that the router should add to the routing table, you have to set the AD value of the route lower than other routes. For example, if you use the following commands to create two static routes for network 30.0.0.0/8, the route will place the first route to the routing table.

```
#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
```

```
#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
```

If the first route fails, the router automatically adds the second route to the routing table.

Creating, adding, verifying static routes

Routers automatically learn their connected networks. We only need to add routes for the networks that are not available on the router's interfaces. For example, network 10.0.0.0/8, 20.0.0.0/8 and 40.0.0.0/8 are directly connected to Router 0. Thus, we don't need to configure routes for these networks. Network 30.0.0.0/8 and network 50.0.0.0/8 are not available on Router0. We have to create and add routes only for these networks.

The following table lists the connected networks of each router.

Router	Available networks on local interfaces	Networks available on other routers' interfaces
Router0	10.0.0.0/8, 20.0.0.0/8, 40.0.0.0/8	30.0.0.0/8, 50.0.0.0/8
Router1	20.0.0.0/8, 30.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 40.0.0.0/8
Router2	40.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 20.0.0.0/8, 30.0.0.0/8

Let's create static routes on each router for networks that are not available on the router

- Create two routes for network 30.0.0.0/8 and configure the first route (via -Router1) as the main route and the second route (via-Router2) as a backup route.
- Create two routes for the host 30.0.0.100/8 and configure the first route (via -Router2) as the main route and the second route (via-Router1) as a backup route.
- Create two routes for network 50.0.0.0/8 and configure the first route (via -Router2) as the main route and the second route (via-Router1) as a backup route.
- Verify the router adds only main routes to the routing table.

Access the CLI prompt of Router0 and run the following commands.

```

Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20
Router(config)#exit
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [10/0] via 40.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
Router#

```

Router0

Physical Config CLI Attributes

IOS Command Line Interface

```

Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10 Primary route
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20 Backup route
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10 Primary route
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20 Backup route
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10 Primary route
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20 Backup route
Router(config)#exit
Router#show ip route static
    30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S      30.0.0.0/8 [10/0] via 20.0.0.2
S      30.0.0.100/32 [10/0] via 40.0.0.2
S      50.0.0.0/8 [10/0] via 40.0.0.2
Router#

```

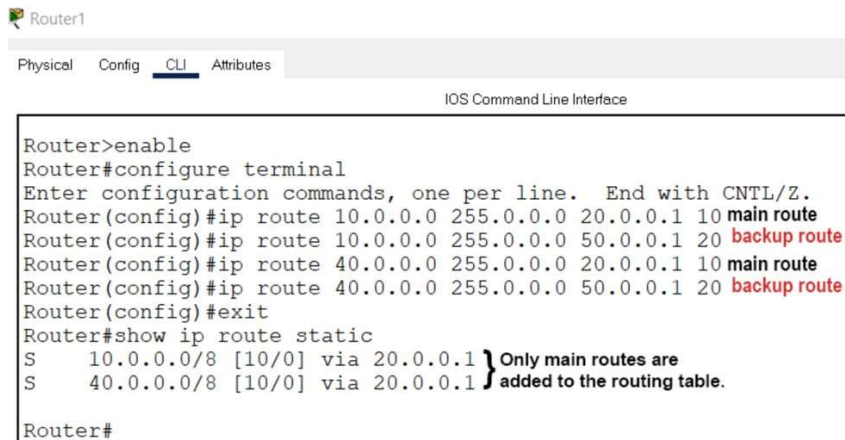
Router adds only primary routes to the routing table.

Router1 requirements

- Create two routes for network 10.0.0.0/8 and configure the first route (via -Router0) as the main route and the second route (via-Router1) as a backup route.
- Create two routes for network 40.0.0.0/8 and configure the first route (via -Router0) as the main route and the second route (via-Router2) as a backup route.
- Verify the router adds only main routes to the routing table.

Router1 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 10.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#ip route 40.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 40.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [10/0] via 20.0.0.1
S 40.0.0.0/8 [10/0] via 20.0.0.1
Router#
```



```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 20.0.0.1 10 main route
Router(config)#ip route 10.0.0.0 255.0.0.0 50.0.0.1 20 backup route
Router(config)#ip route 40.0.0.0 255.0.0.0 20.0.0.1 10 main route
Router(config)#ip route 40.0.0.0 255.0.0.0 50.0.0.1 20 backup route
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [10/0] via 20.0.0.1 } Only main routes are
S 40.0.0.0/8 [10/0] via 20.0.0.1 } added to the routing table.
Router#
```

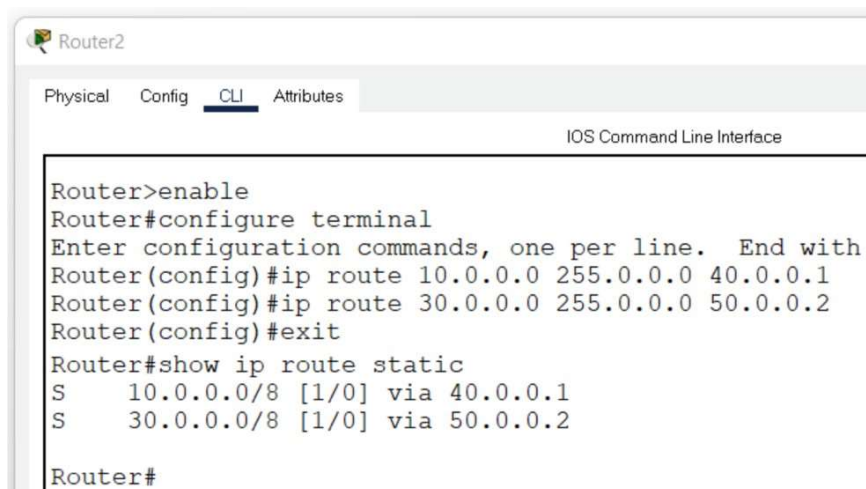
Router2 requirements

Router Requirements

Create static routes for network 10.0.0.0/8 and network 30.0.0.0/8 and verify the router adds both routes to the routing table.

Router2 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 40.0.0.1
Router(config)#ip route 30.0.0.0 255.0.0.0 50.0.0.2
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [1/0] via 40.0.0.1
S 30.0.0.0/8 [1/0] via 50.0.0.2
Router#
```

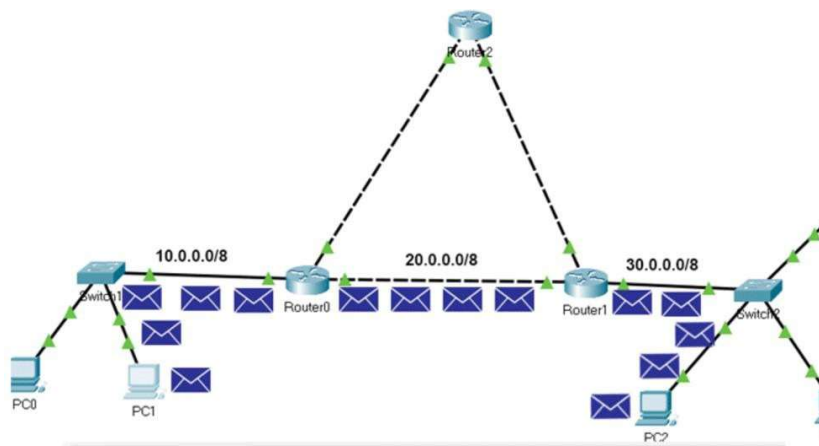


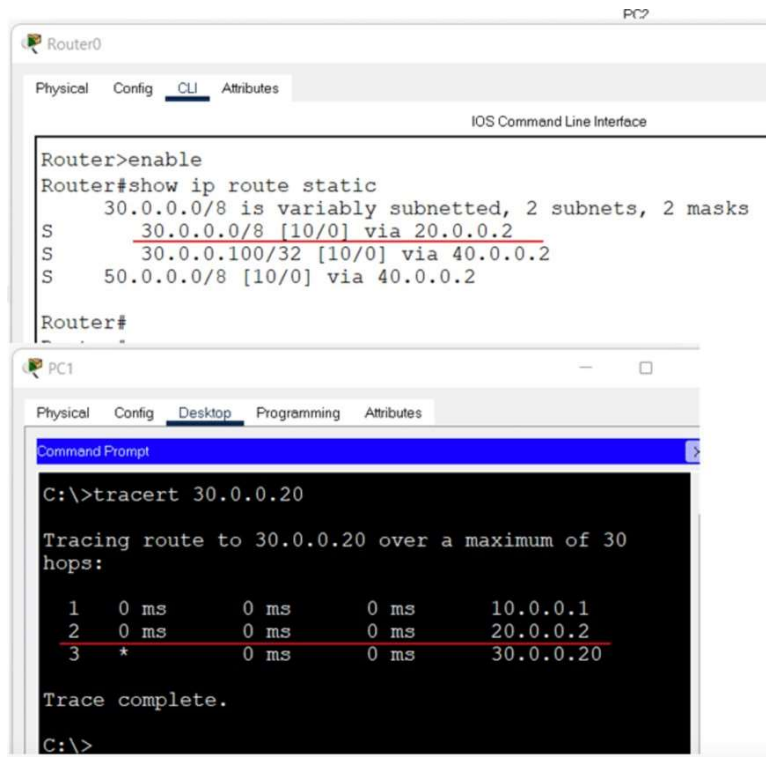
Step 1: Verifying static routing

On Router0, we configured two routes for network 30.0.0.0/8. These routes are via Router1 and via Router2. We set the first route (via-Router1) as the main route and the second route as the backup route. We can verify this configuration in two ways. By sending ping requests to a PC of network 30.0.0.0/8 and tracing the path they take to reach the network 30.0.0.0/8. For this, you can use '**tracert**' command on a PC of network 10.0.0.0/8. The '**tracert**' command sends ping requests to the destination host and tracks the path they take to reach the destination. By listing the routing table entries on Router0. Since a router uses the routing table to forward data packets, you can

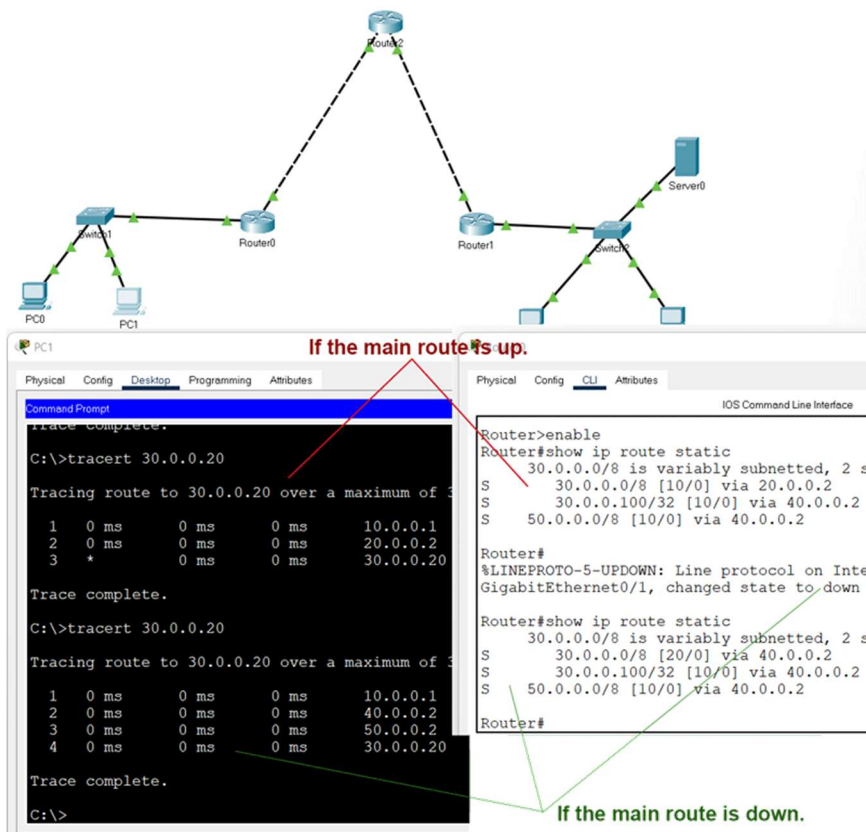
check the routing table to figure out the route the router uses to forward data packets for each destination.

The following image shows the above testing.





We also configured a backup route for network 30.0.0.0/8. The router must put the backup route to the routing table and use it to forward data packets to network 30.0.0.0/8 when the main route fails. To verify this, we have to simulate the failure of the main route. To simulate the failure of the main route, you can delete the link between Router0 and Router1. After deleting the link, do the same testing again for the network 30.0.0.0/8.



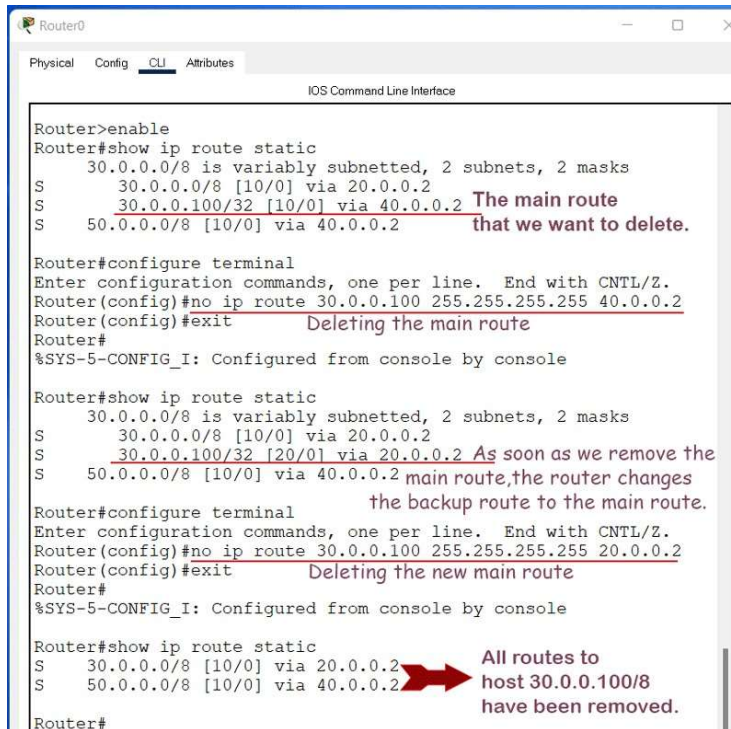
Deleting a static route

To delete a static route, use the following steps.

- Use the '**show ip route static**' command to print all static routes.
- Note down the route you want to delete.
- Use the '**no ip route**' command to delete the route.

If you have a backup route, the backup route becomes the main route when you delete the main route.

In our example, we have a backup route and a main route for the host 30.0.0.100/8. The following image shows how to delete both routes.



```
Router>enable
Router#show ip route static
  30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2 The main route
S       50.0.0.0/8 [10/0] via 40.0.0.2 that we want to delete.

Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 40.0.0.2
Router(config)#exit Deleting the main route
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show ip route static
  30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [20/0] via 20.0.0.2 As soon as we remove the
S       50.0.0.0/8 [10/0] via 40.0.0.2 main route, the router changes
                                         the backup route to the main route.

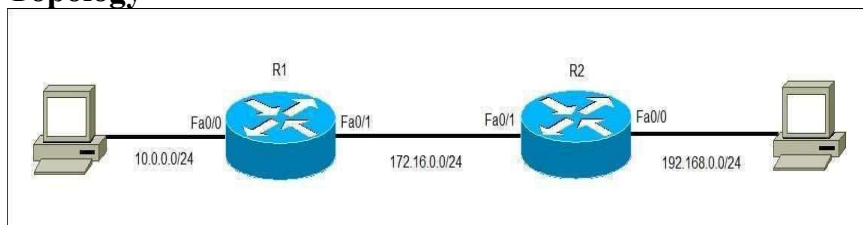
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 20.0.0.2
Router(config)#exit Deleting the new main route
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show ip route static
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       50.0.0.0/8 [10/0] via 40.0.0.2
Router#
```

All routes to host 30.0.0.100/8 have been removed.

3. Graded Lab Tasks

Topology



Note that R1 knows about the two directly connected networks: 10.0.0.0/24 and 172.16.0.0/24. What it doesn't know about, however, is the 192.168.0.0/24 network directly attached to R2. Configure static routes to ping a host in that network:

LAB 11

Network Address Translation (NAT) Configuration

Objective:

The objective of this lab is that the students would learn how to configure static NAT, map address, debug and verify Static NAT translation step by step with practical examples in packet tracer.

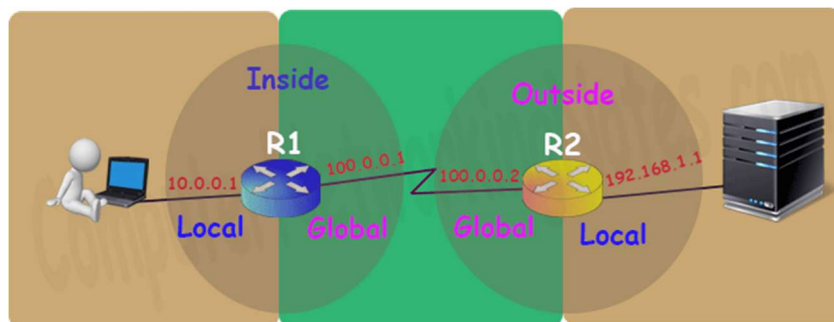
Activity Outcomes:

The activities provide hands - on practice with the following topics

- Configure static NAT
- Map IP addresses
- Debug and verify Static NAT translation

Useful Concepts

There are several situations where we need address translation such as, a network which do not have sufficient public IP addresses want to connect with the Internet, two networks which have same IP addresses want to merge or due to security reason a network want to hide its internal IP structure from the external world. NAT (Network Address Translation) is the process which translates IP address. NAT can be performed at firewall, server and router. In this lab we will understand how it is performed at router. Let's understand these terms with an example. Suppose a user is browsing a website from his home computer. The network which connects his computer with internet is considered as a local network for him. Same as the network which connects the webserver where the website is located with internet is considered as a local network for webserver. The network which connects both networks on internet is considered as a global network.



On router the interface which is connected with local network will be configured with inside local IP address and the interface which is connected with global network will be configured with inside global IP address. Inside and outside depend on where we are standing right now. For example in above network for user router R1 is inside and router R2 is outside.

For more details, please check the below reference.

Reference:

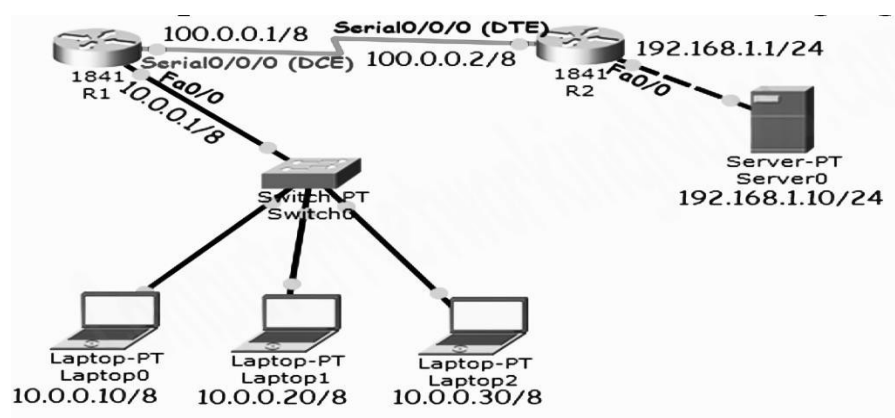
- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- <https://www.computernetworkingnotes.com/ccna-study-guide/how-to-configure-static-nat-in-cisco-router.html>
- <https://www.computernetworkingnotes.com/ccna-study-guide/basic-concepts-of-nat-explained-in-easy-language.html>
- <https://ipccisco.com/lesson/nat-network-address-translation/>
- https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/ipaddr_nat/configuration/xr-16/nat-xr-16-book/iadnat-addr-consv.html
- <https://www.ciscopress.com/articles/article.asp?p=1725268>

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>25</i>	<i>Low</i>	<i>CLO-6</i>
<i>2</i>	<i>20</i>	<i>Low</i>	<i>CLO-6</i>

3	20	Medium	CLO-6
---	----	--------	-------

Activity 1: Create a practice lab as shown in following figure



Initial IP Configuration

Device / Interface	IP Address	Connected With
Laptop0	10.0.0.10/8	Fa0/0 of R0
Laptop1	10.0.0.20/8	Fa0/0 of R0
Laptop2	10.0.0.30/8	Fa0/0 of R0
Server0	192.168.1.10/24	Fa0/0 of R1
Serial 0/0/0 of R1	100.0.0.1/8	Serial 0/0/0 of R2
Serial 0/0/0 of R2	100.0.0.2/8	Serial 0/0/0 of R2

To assign IP address in Laptop click **Laptop** and click **Desktop** and **IP configuration** and Select **Static** and **set IP address** as given in above table.

Following same way configure IP address in Server.

To configure IP address in Router1 click **Router1** and select **CLI** and press **Enter key**.

Two interfaces of Router1 are used in topology; FastEthernet0/0 and Serial 0/0/0.

By default interfaces on router are remain administratively down during the start up. We need to configure IP address and other parameters on interfaces before we could actually use them for routing. Interface mode is used to assign the IP address and other parameters. Interface mode can be accessed from global configuration mode. Following commands are used to access the global configuration mode.

```
Router-PT>enable
Router-PT# configure terminal
Enter configuration commands, one per line.
Router-PT(config)#
```

Before we configure IP address in interfaces let's assign a unique descriptive name to router.

```
Router-PT(config)#hostname R1
```

Now execute the following commands to set IP address in FastEthernet 0/0 interface.

```
R1(config)#interface FastEthernet0/0
R1(config-if)#ip address 10.0.0.1 255.0.0.0
R1(config-if)#no shutdown
R1(config-if)#exit
```

interface FastEthernet 0/0 command is used to enter in interface mode.

ip address 10.0.0.1 255.0.0.0 command assigns IP address to interface.

no shutdown command is used to bring the interface up.

exit command is used to return in global configuration mode.

Serial interface needs two additional parameters clock rate and bandwidth. Every serial cable has two ends DTE and DCE. These parameters are always configured at DCE end.

We can use show controllers interface command from privilege mode to check the cable's end.

```
R1(config)#exit
R1#show controllers serial 0/0/0
Interface Serial0/0/0
Hardware is PowerQUICC MPC860
DCE V.35, clock rate 2000000
[Output omitted]
```

Fourth line of output confirms that DCE end of serial cable is attached. If you see DTE here instead of DCE skip these parameters.

Now we have necessary information let's assign IP address to serial interface.

```
R1#configure terminal
R1(config)#interface Serial0/0/0
R1(config-if)#ip address 100.0.0.1 255.0.0.0
R1(config-if)#clock rate 64000
R1(config-if)#bandwidth 64
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#
```

Router#configure terminal Command is used to enter in global configuration mode.

Router(config)#interface serial 0/0/0 Command is used to enter in interface mode.

Router(config-if)#ip address 100.0.0.1 255.0.0.0 Command assigns IP address to interface.

Router(config-if)#clock rate 64000

In real life environment this parameter controls the data flow between serial links and need to be set at service provider's end. In lab environment we need not to worry about this value. We can use any valid rate here.

Router(config-if)#bandwidth 64

Bandwidth works as an influencer. It is used to influence the metric calculation of EIGRP or any other routing protocol which uses bandwidth parameter in route selection process.

Router(config-if)#no shutdown Command brings interface up.

Router(config-if)#exit Command is used to return in global configuration mode.

We will use same commands to assign IP addresses on interfaces of Router2. We need to provided clock rate and bandwidth only on DCE side of serial interface. Following command will assign IP addresses on interface of Router2.

Initial IP configuration in R2

```
Router>enable
Router#configure terminal
Router(config)#hostname R2
R2(config)#interface FastEthernet0/0
R2(config-if)#ip address 192.168.1.1 255.255.255.0
R2(config-if)#no shutdown
R2(config-if)#exit
R2(config)#interface Serial0/0/0
R2(config-if)#ip address 100.0.0.2 255.0.0.0
R2(config-if)#no shutdown
R2(config-if)#exit
R2(config)#
```

That's all initial IP configuration we need. Now this topology is ready for the practice of static nat.

Activity 2: Configure Static NAT

Static NAT configuration requires three steps: -

1. Define IP address mapping
2. Define inside local interface

3. Define inside global interface

Since static NAT use manual translation, we have to map each inside local IP address (which needs a translation) with inside global IP address. Following command is used to map the inside local IP address with inside global IP address.

```
Router(config)#ip nat inside source static [inside local ip address] [inside global IP address]
```

For example in our lab Laptop1 is configured with IP address 10.0.0.10. To map it with 50.0.0.10 IP address we will use following command

```
Router(config)#ip nat inside source static 10.0.0.10 50.0.0.10
```

In second step we have to define which interface is connected with local the network. On both routers interface Fa0/0 is connected with the local network which need IP translation.

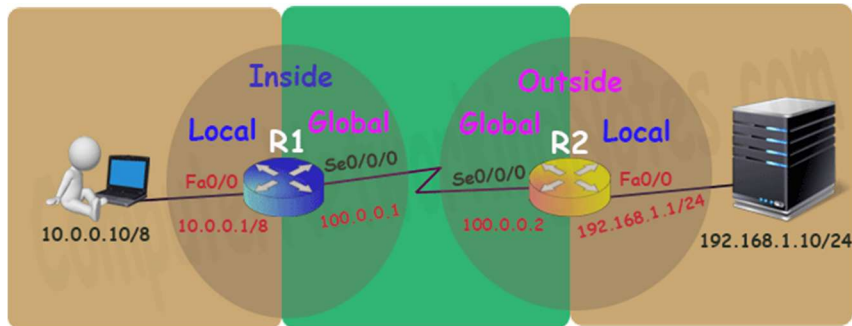
Following command will define interface Fa0/0 as inside local.

```
Router(config-if)#ip nat inside
```

In third step we have to define which interface is connected with the global network. On both routers serial 0/0/0 interface is connected with the global network. Following command will define interface Serial0/0/0 as inside global.

```
Router(config-if)#ip nat outside
```

Following figure illustrates these terms.



Let's implement all these commands together and configure the static NAT.

R1 Static NAT Configuration

```
R1(config)#ip nat inside source static 10.0.0.10 50.0.0.10
R1(config)#interface FastEthernet 0/0
R1(config-if)#ip nat inside
R1(config-if)#exit
R1(config)#
R1(config)#interface Serial 0/0/0
R1(config-if)#ip nat outside
R1(config-if)#exit
```

For testing purpose I configured only one static translation. You may use following commands to configure the translation for remaining address.

```
R1(config)#ip nat inside source static 10.0.0.20 50.0.0.20
R1(config)#ip nat inside source static 10.0.0.30 50.0.0.30
```

R2 Static NAT Configuration

```
R2(config)#ip nat inside source static 192.168.1.10 200.0.0.10
R2(config)#interface FastEthernet 0/0
R2(config-if)#ip nat inside
R2(config-if)#exit
R2(config)#
R2(config)#interface Serial 0/0/0
R2(config-if)#ip nat outside
```

```
R2(config-if)#exit
```

Before we test this lab we need to configure the IP routing. IP routing is the process which allows router to route the packet between different networks.

Configure static routing in R1

```
R1(config)#ip route 200.0.0.0 255.255.255.0 100.0.0.2
```

Configure static routing in R2

```
R2(config)#ip route 50.0.0.0 255.0.0.0 100.0.0.1
```

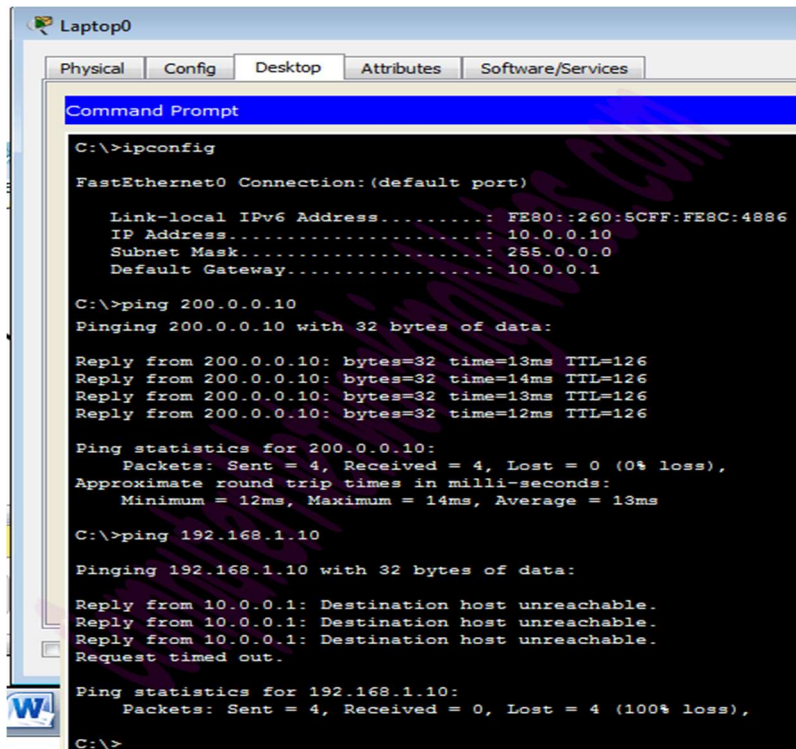
Testing Static NAT Configuration

In this lab we configured static NAT on R1 and R2. On R1 we mapped inside local IP address 10.0.0.10 with inside global address 50.0.0.10 while on R2 we mapped inside local IP address 192.168.1.10 with inside global IP address 200.0.0.10.

Device	Inside Local IP Address	Inside Global IP Address
Laptop0	10.0.0.10	50.0.0.10
Server	192.168.1.10	200.0.0.10

To test this setup click Laptop0 and Desktop and click Command Prompt.

- Run **ipconfig** command.
- Run **ping 200.0.0.10** command.
- Run **ping 192.168.1.10** command.



```
C:\>ipconfig

FastEthernet0 Connection: (default port)

    Link-local IPv6 Address . . . . . : FE80::260:SCFF:FE8C:4886
    IP Address . . . . . : 10.0.0.10
    Subnet Mask . . . . . : 255.0.0.0
    Default Gateway . . . . . : 10.0.0.1

C:\>ping 200.0.0.10

Pinging 200.0.0.10 with 32 bytes of data:

Reply from 200.0.0.10: bytes=32 time=13ms TTL=126
Reply from 200.0.0.10: bytes=32 time=14ms TTL=126
Reply from 200.0.0.10: bytes=32 time=13ms TTL=126
Reply from 200.0.0.10: bytes=32 time=12ms TTL=126

Ping statistics for 200.0.0.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 12ms, Maximum = 14ms, Average = 13ms

C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Reply from 10.0.0.1: Destination host unreachable.
Reply from 10.0.0.1: Destination host unreachable.
Reply from 10.0.0.1: Destination host unreachable.
Request timed out.

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>
```

First command verifies that we are testing from correct NAT device.

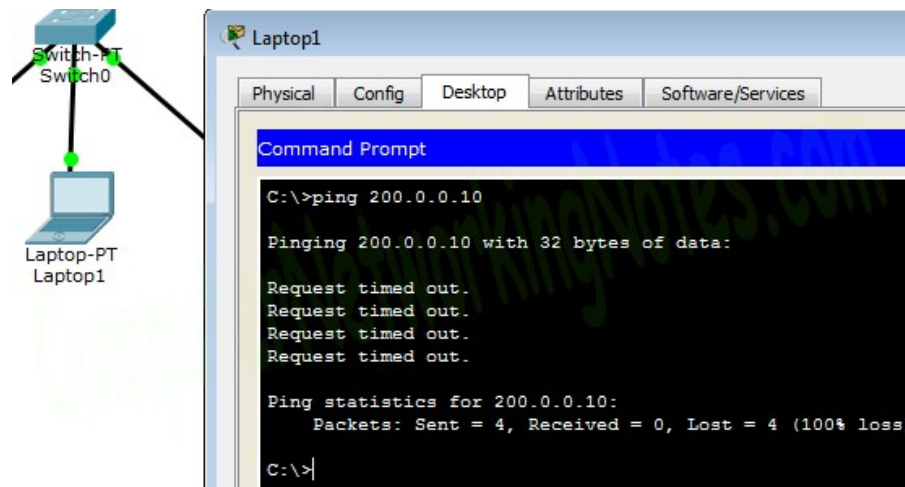
Second command checks whether we are able to access the remote device or not. A ping reply confirms that we are able to connect with remote device on this IP address.

Third command checks whether we are able to access the remote device on its actual IP address or not. A ping error confirms that we are not able to connect with remote device on this IP address.

Let's do one more testing. Click **Laptop0** and click **Desktop** and click **Web Browser** and access 200.0.0.10.

Above figure confirms that host 10.0.0.10 is able to access the 200.0.0.10.

Now run **ping 200.0.0.10** command from Laptop1.



Why we are not able to connect with the remote device from this host?

Because we configured NAT only for one host (Laptop0) which IP address is 10.0.0.10. So only the host 10.0.0.10 will be able to access the remote device.

To confirm it again, let's try to access web service from this host.

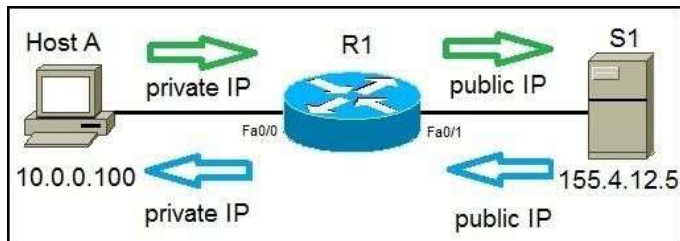
We can also verify this translation on router with ***show ip nat translation*** command.

Pay a little bit extra attention on outside local address field. Have you noticed one interesting feature of NAT in above output? Why actual outside local IP address is not listed in this field?

The actual IP address is not listed here because router is receiving packets after the translation. From R1's point of view remote device's IP address is 200.0.0.10 while from R2's point of view end device's IP address is 50.0.0.10. This way if NAT is enabled we would not be able to trace the actual end device.

3. Graded Lab Tasks

Consider the following example:



Host A requests a web resource from **S1**. Host A uses its private IP address when sending the request to the router R1, which serves as a default gateway. Router R1 receives the request, changes the private IP address to the public one and sends the request to S1. S1 responds to R1. R1 receives the response, looks up in its NAT table and changes the destination IP address to the private IP address of Host A. Configure static NAT in the example above.

LAB 12

RIP Protocol configuration

Objective:

This tutorial explains how to configure RIP Routing protocol step by step in detail. RIP is the simplest and one of the oldest Distance Vector routing protocol. Learn how to enable and configure RIP routing in router with practical example in packet tracer.

Activity Outcomes:

The activities provide hands - on practice to configure RIP Routing protocol on router.

Useful Concepts

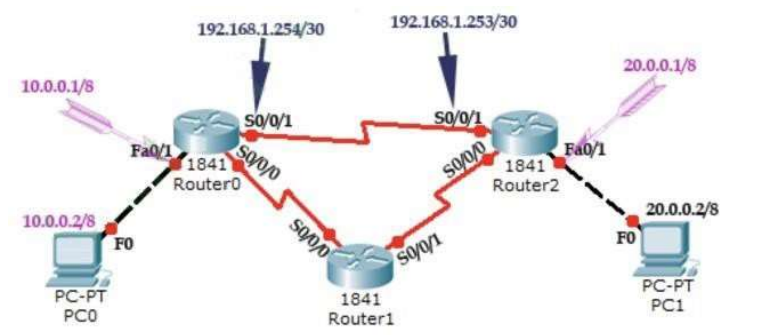
RIP routing protocol uses the concept of distance-vector routing. Routers running RIP protocol learn about the destination networks from neighboring routers through broadcast messages. Routers running RIP protocol periodically broadcast the configured networks from all ports. Routers also receive broadcasts (routing updates) on their active interfaces. Routers compare their routing tables with routing updates to learn about new IP subnets. RIP uses distance to select the best route for each destination subnet. Distance is calculated in the term of **hops**. Each instance where a packet goes through a router is called a **hop**, and the route with the least number of hops to the destination subnet is selected as the best route for that destination subnet.

For more details, please check the below reference.

Reference:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- <https://www.computernetworkingnotes.com/ccna-study-guide/rip-protocol-configuration-guide-with-examples.html>
- <https://ipccisco.com/lesson/routing-information-protocol-rip/>
- <https://www.ciscopress.com/articles/article.asp?p=2262897&seqNum=3>

- https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst9300/software/release/16-8/configuration_guide/rtnng/b_168_rtnng_9300_cg/b_168_rtnng_9300_cg_chapter_0100.pdf



1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>35</i>	<i>Medium</i>	<i>CLO-6</i>
<i>2</i>	<i>30</i>	<i>High</i>	<i>CLO-6</i>

Activity 1: Initial IP configuration

Device	Interface	IP Configuration	Connected with
PC0	Fast Ethernet	10.0.0.2/8	Router0's Fa0/1
Router0	Fa0/1	10.0.0.1/8	PC0's Fast Ethernet
Router0	S0/0/1	192.168.1.254/30	Router2's S0/0/1
Router0	S0/0/0	192.168.1.249/30	Router1's S0/0/0
Router1	S0/0/0	192.168.1.250/30	Router0's S0/0/0

Router1 S0/0/1	192.168.1.246/30	Router2's S0/0/0
Router2 S0/0/0	192.168.1.245/30	Router1's S0/0/1
Router2 S0/0/1	192.168.1.253/30	Router0's S0/0/1
Router2 Fa0/1	20.0.0.1/30	PC1's Fast Ethernet
PC1 Fast Ethernet	20.0.0.2/30	Router2's Fa0/1

Assign IP address to PCs

Double click **PC0** and click **Desktop** menu item and click **IP Configuration**. Assign IP address 10.0.0.2/8 to PC0.

Assign IP address to interfaces of routers

Double click **Router0** and click **CLI** and press **Enter** key to access the command prompt of **Router0**. Three interfaces *FastEthernet0/0*, *Serial0/0/0* and *Serial0/0/1* of **Router0** are used in this topology. By default interfaces on router are remain administratively down during the start up.

We need to configure IP address and other parameters on interfaces before we could actually use them for routing. Interface mode is used to assign IP address and other parameters. Interface mode can be accessed from global configuration mode. Following commands are used to access the global configuration mode.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#
```

From global configuration mode we can enter in interface mode. From there we can configure the interface. Following commands will assign IP address on FastEthernet0/0.

```
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 10.0.0.1 255.0.0.0
Router(config-if)#no shutdown
```

```
Router(config-if)#exit
Router(config)#
```

interface *fastEthernet 0/0* command is used to enter in interface mode.

ip address 10.0.0.1 255.0.0.0 command will assign IP address to interface.

no shutdown command will bring the interface up.

exit command is used to return in global configuration mode.

Serial interface needs two additional parameters **clock rate** and **bandwidth**. Every serial cable has two ends DTE and DCE. These parameters are always configured at DCE end.

We can use **show controllers interface** command from privilege mode to check the cable's end.

```
Router#show controllers serial 0/0/0
Interface Serial0/0/0
Hardware is PowerQUICC MPC860
DCE V.35, clock rate 2000000
[Output omitted]
```

Fourth line of output confirms that DCE end of serial cable is attached. If you see DTE here instead of DCE skip these parameters.

Now we have necessary information let's assign IP address to serial interface.

```
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.249 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.254 255.255.255.252
```

```
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

Router#configure terminal Command is used to enter in global configuration mode.

Router(config)#interface serial 0/0/0 Command is used to enter in interface mode.

Router(config-if)#ip address 192.168.1.249 255.255.255.252 Command assigns IP address to interface. For serial link we usually use IP address from /30 subnet.

Router(config-if)#clock rate 64000 And **Router(config-if)#bandwidth 64** In real life environment these parameters control the data flow between serial links and need to be set at service providers end. In lab environment we need not to worry about these values. We can use these values.

Router(config-if)#no shutdown Command brings interface up.

Router(config-if)#exit Command is used to return in global configuration mode.

We will use same commands to assign IP addresses on interfaces of remaining routers. We need to provided clock rate and bandwidth only on DCE side of serial interface. Following command will assign IP addresses on interface of Router1.

```
Router1
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.250 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.246 255.255.255.252
Router(config-if)#clock rate 64000
```

```
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
```

Use same commands to assign IP addresses on interfaces of Router2.

```
Router2
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 20.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.245 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.253 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
```

Now routers have information about the networks that they have on their own interfaces. Routers will not exchange this information between them on their own. We need to implement RIP routing protocol that will insist them to share this information.

Configure RIP routing protocol

Configuration of RIP protocol is much easier than you think. It requires only two steps to configure the RIP routing.

- Enable RIP routing protocol from global configuration mode.
- Tell RIP routing protocol which networks you want to advertise.

Let's configure it in Router0

Router0

```
Router0(config)#router rip
Router0(config-router)# network 10.0.0.0
Router0(config-router)# network 192.168.1.252
Router0(config-router)# network 192.168.1.248
```

router rip command tell router to enable the RIP routing protocol.

network command allows us to specify the networks which we want to advertise. We only need to specify the networks which are directly connected with the router.

That's all we need to configure the RIP. Follow same steps on remaining routers.

Router1

```
Router1(config)#router rip
Router1(config-router)# network 192.168.1.244
Router1(config-router)# network 192.168.1.248
```

Router2

```
Router2(config)#router rip
Router2(config-router)# network 20.0.0.0
Router2(config-router)# network 192.168.1.252
Router2(config-router)# network 192.168.1.244
```

That's it. Our network is ready to take the advantage of RIP routing. To verify the setup we will use ping command. ping command is used to test the connectivity between two devices.

Access the command prompt of **PC1** and use *ping* command to test the connectivity from **PC0**.

RIP protocol automatically manage all routes for us. If one route goes down, it automatically switches to another available. To explain this process more clearly we have added one more route in our network.

Currently there are two routes between PC0 and PC1.

Route 1

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/1 – 192.168.1.254] <==> Router2 [Serial 0/0/1 – 192.168.1.253] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination /source – 20.0.0.2]

Route 2

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/0 – 192.168.1.249] <==> Router1 [Serial 0/0/0 – 192.168.1.250] <==> Router1 [Serial 0/0/1 – 192.168.1.246] <==> Router2 [Serial 0/0/0 – 192.168.1.245] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination /source – 20.0.0.2]

By default RIP will use the route that has low hops counts between source and destination. In our network route1 has low hops counts, so it will be selected. We can use *tracert* command to verify it.

Now suppose route1 is down. We can simulate this situation by removing the cable attached between **Router0 [s0/0/1]** and **Router2 [s0/0/1]**.

Okay our primary route went down. What will be happen now?

So far we are running RIP routing protocol and have another route to destination, there is no need to worry. RIP will automatically reroute the traffic. Use *tracert* command again to see the magic of dynamic routing.

3. Graded Lab Tasks

Build the network topology and configure RIP in packet tracer.



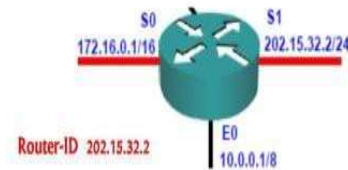
LAB 13

Open Shortest Path First (OSPF)

OSPF stand for Open Shortest path first It's a link state protocol
Standard protocol
It uses SPF (shortest path first) or dijkistra algorithm
Unlimited hop count
Metric is cost (cost= $10^8/B.W.$)
Administrative distance is 110
It is a classless routing protocol (carry subnet-mask information & supports VLSM)
Supports equal cost load balancing.

OSPF router-ID

used to identify the router inside the OSPF database.
OSPF identify using same ID in all directions.
Default uses highest IP address of active physical interface.



OSPF tables

Neighbor table

- ° Contains list of directly connected routers
- ° # show ip ospf neighbor

Routing table

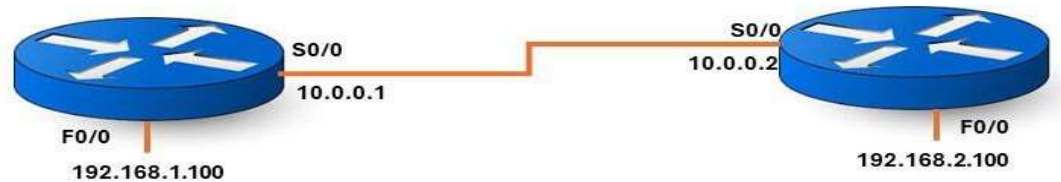
- The best route to the destination
- ° # show ip route

Database Table

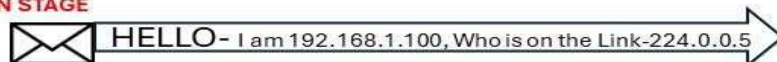
referred to as LSDB (link state database)
Contains information about all the possible routes to the networks with in the area
show ip ospf database

All the routers must have the same database

Basic OSPF process



DOWN STAGE



INIT STAGE



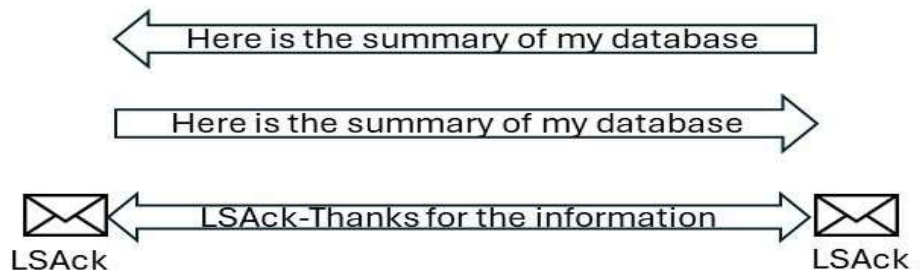
TWO WAY STAGE

They become Neighbor and build **NEIGHBOR TABLE**

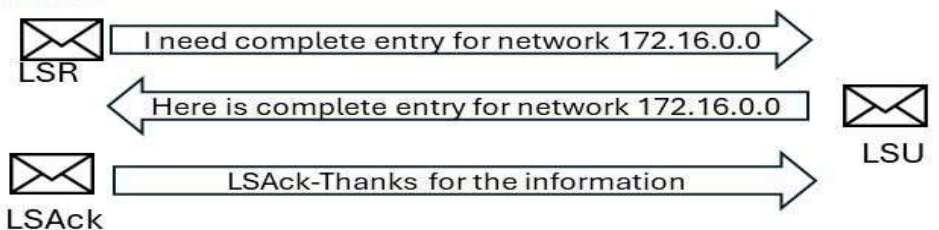
EXSTART STAGE



EXCHANGE STAGE



LOADING STAGE



FULL STAGE

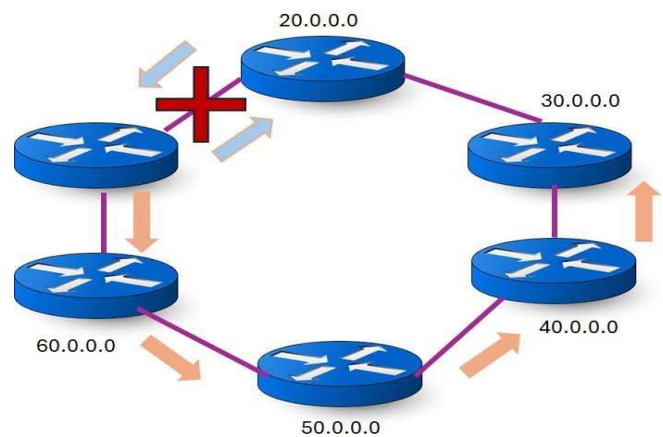
- Routers have synchronized database
- Calculate the best route (Reference BW say 100Mbps/Actual BW on the link say 2 Mbps)
- Cost will be $100/2=50$ in this case

Seven stages

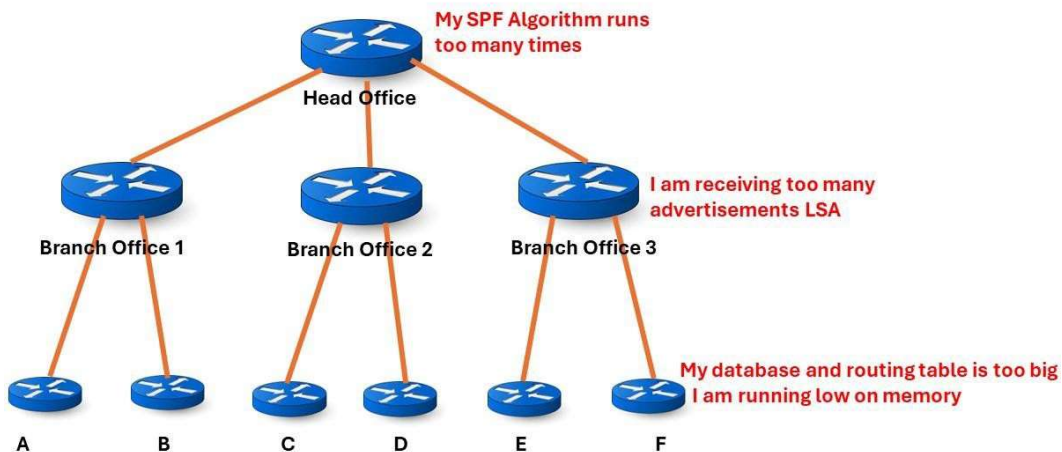
1. Down stage
2. Init stage
3. 2 way stage
4. Exstart stage
5. Exchange stage
6. Loading stage
7. Full stage

OSPF Convergence

Incremental updates
Periodically send hello packets are sent every 10 seconds (dead - 40 sec)
Convergence rate is fast (40 sec)



Issues with maintaining a Large OSPF Network



- All the routers must have the same database
- Any change in the database advertised to all routers

Concept of areas

Area is logical grouping of Routers.

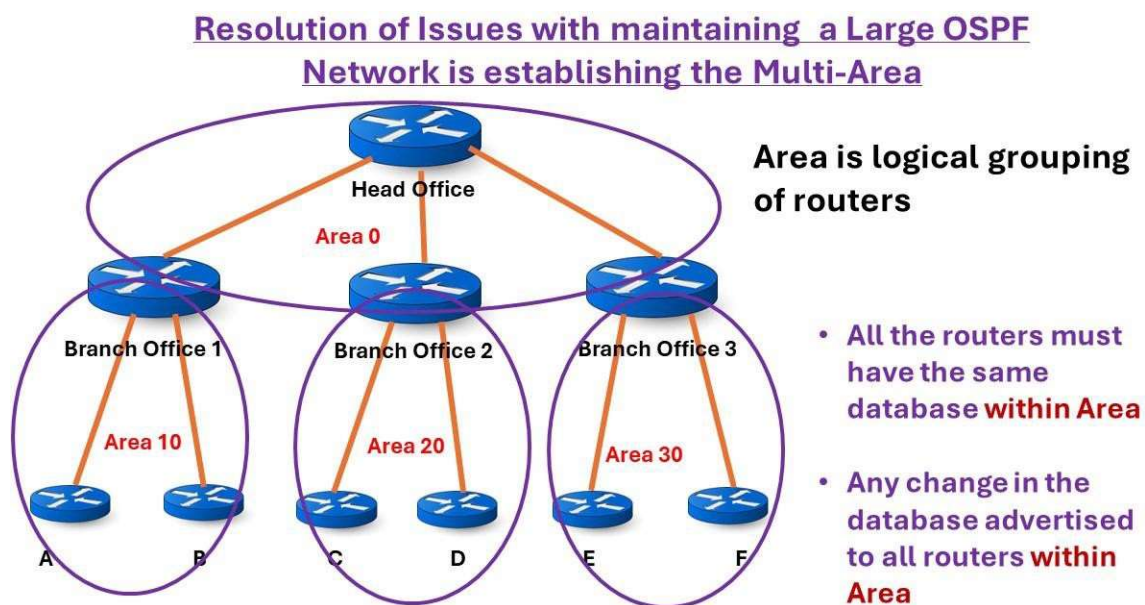
All the routers maintain same database with in the same Area.

Any change impact all the routers with the same area.

Minimizes size of database

Restrict any changes with in that area, (not flood outside area)

Routers with in the same area participate in Algorithm



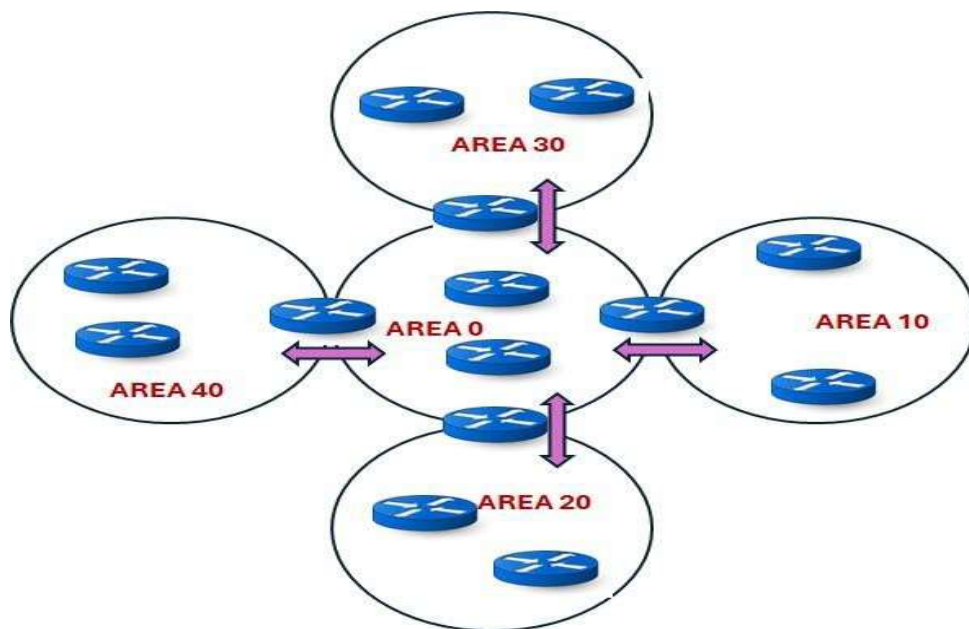
OSPF area design Rules

Must have one area called as area 0 (its backbone area)

All the non-backbone areas must connect to area 0. (Area 0 must be transit area)

At least one Area Border Router (connecting two or more areas) .

Interfaces of both routers facing must be in the same Area.



Configuring OSPF

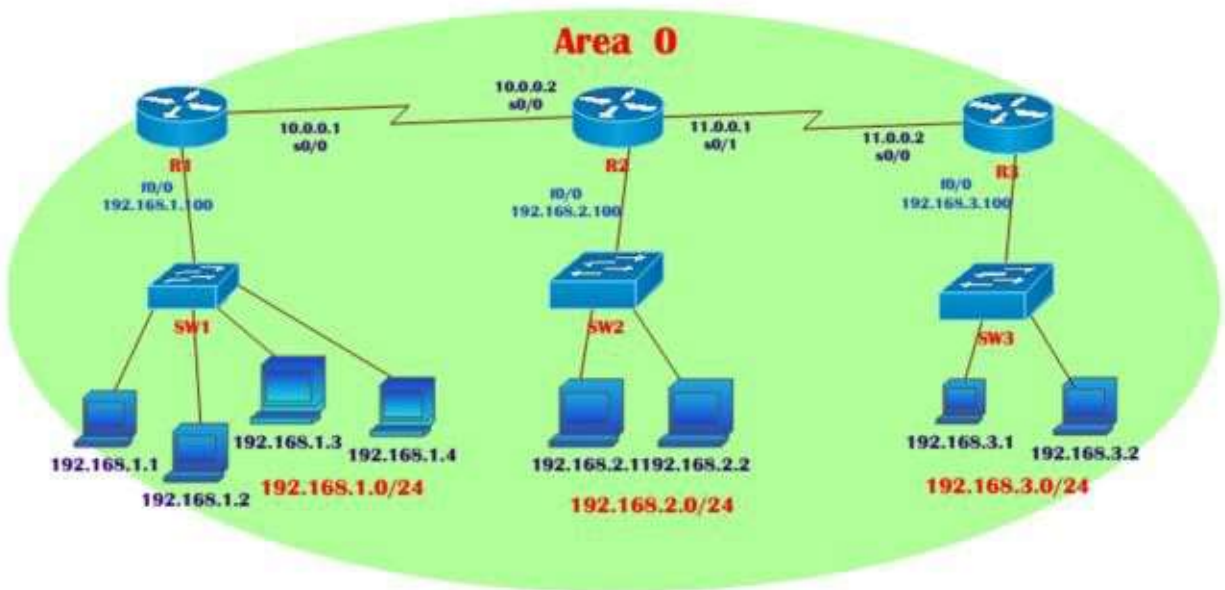
```
(config)# router ospf <process ID>
```

```
(config-router)# network <Network ID> <wildcard mask> area <area id>
```

Process ID is a number used to identify an OSPF routing process on the router.

- Multiple OSPF processes can be started on the same router.
- The number can be any value between 1 and 65,535.

LAB: Dynamic Routing using OSPF in Single Area



Pre-requirement for LAB (check previous labs)

- Design the topology (connectivity)
- Assign the IP address according to diagram
- Make sure that interfaces used should be in UP UP state

TASK

- Configure Dynamic routing using OSPF single area as per the diagram
- Verify Routing table and reachability between the LAN's (using PING and TRACE commands)

R-1#sh ip route

Gateway of last resort is not set

C 10.0.0.0/8 is directly connected, Serial0/0

C 192.168.1.0/24 is directly connected, FastEthernet0/0

R-2#sh ip route

Gateway of last resort is not set

C 10.0.0.0/8 is directly connected, Serial0/0

C 11.0.0.0/8 is directly connected, Serial0/1

C 192.168.2.0/24 is directly connected, FastEthernet0/0

R-3#sh ip route

Gateway of last resort is not set

C 11.0.0.0/8 is directly connected, Serial0/0

C 192.168.3.0/24 is directly connected, FastEthernet0/0

Router- 1

R-1(config)#router ospf 1

R-1(config-router)#network 192.168.1.0 0.0.0.255 area 0

R-1(config-router)#network 10.0.0.0 0.255.255.255 area 0

Router - 2

R-2(config)#router ospf 1

R-2(config-router)#network 192.168.2.0 0.0.0.255 area 0

R-2(config-router)#network 11.0.0.0 0.255.255.255 area 0

R-2(config-router)#network 10.0.0.0 0.255.255.255 area 0

06:14:49: %OSPF-5-ADJCHG: Process 1,Nbr 192.168.1.100 on Serial 0/0 from
LOADING to FULL, Loading Done

Router - 3

R-3(config)#router ospf 1

R-3(config-router)#network 192.168.3.0 0.0.0.255 area 0

R-3(config-router)#net 11.0.0.0 0.255.255.255 area 0

06:15:46: %OSPF-5-ADJCFG: Process 1,Nbr 192.168.2.100 on Serial o/o from
LOADING to FULL, Loading Done

R-2#show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
192.168.1.100	0	FULL/	-00:00:35	10.0.0.1	Serial0/0
192.168.3.100	0	FULL/	-00:00:37	11.0.0.2	Serial0/1

R-1#show ip route

Gateway of last resort is not set

C 10.0.0.0/8 is directly connected, Serial0/0

O 11.0.0.0/8 [no/128] via 10.0.0.2, 00:04:21, Serial0/0

C 192.168.1.0/24 is directly connected, FastEthernet0/0

O 192.168.2.0/24 [110/65] via 10.0.0.2, 00:04:21, Serial0/0

O 192.168.3.0/24 [110/129] via 10.0.0.2, 00:03:19, Serial0/0

R-1#sh ip route ospf

O 11.0.0.0 [110/28] via 10.0.0.2, 00:04:25, Serial0/0

O 192.168.2.0 [110/65] via 10.0.0.2, 00:04:25, Serial 0/0

O 192.168.3.0 [110/129] via 10.0.0.2, 00:03:23, Serial 0/0

R-2#show ip route ospf

O 192.168.1.0 [110/65] via 10.0.0.1, 00:05:09, Serial0/0

O 192.168.3.0 [110/65] via n.o.0.2, 00:04:14, Serial0/1

R-3#show ip route ospf

O 10.0.0.0 [110/128] via 11.0.0.1, 00:04:49, Serial0/0

O 192.168.1.0 [110/129] via 11.0.0.1, 00:04:49,

O 192.168.2.0 [110/65] via 11.0.0.1, 00:04:49, Serial0/0

R-l#show ip protocols

Routing Protocol is "ospf 1"

Outgoing update filter list for all interfaces is not set

Incoming update filter list for all interfaces is not set

Router ID 192.168.1.100

Number of areas in this router is 1.1 normal o stub o nssa

Maximum path: 4

Routing for Networks:

192.0.0.0 0.0.0.255 area 0

10.0.0.0 0.255.255.255 area 0

Routing Information Sources:

Gateway Distance Last Update

10.0.0.2 110 00:05:46

Distance: (default is no)

PC>ipconflg

IP Address.	:: 192.168.1.1
Subnet Mask	...:
Default Gateway.	255.255.255.0
PC>ping 192.168.2.1	:
	192.168.1.100

Pinging 192.168.2.1 with 32 bytes of data:

Request timed out.

Reply from 192.168.2.1: bytes=32 time=i9ms TTL=126

Reply from 192.168.2.1: bytes=32 time=20ms TTL=126
Reply from 192.168.2.1: bytes=32 time=14ms TTL=126

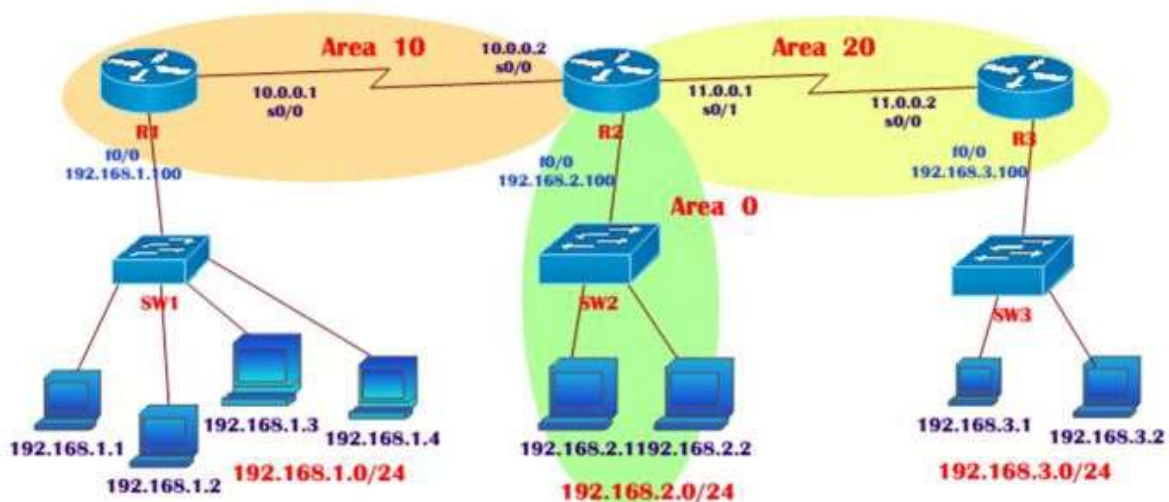
PC>ping 192.168.3.1
Pinging 192.168.3.1 with 32 bytes of data:
Request timed out.
Reply from 192.168.3.1: bytes=32 time=27ms TTL=125
Reply from 192.168.3.1: bytes=32 time=22ms TTL=125
Reply from 192.168.3.1: bytes=32 time=25ms TTL=125

PC>tracert 192.168.3.1
Tracing route to 192.168.3.1 over a maximum of 30 hops:
1 5 ms 8 ms 8 ms 192.168.1.100
2 12 ms 9 ms 8 ms 10.0.0.2
3 17 ms 6 ms 12 ms 11.0.0.2
4 24 ms 27 ms 25 ms 192.168.3.1
Trace complete.

R-1#ping 192.168.3.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.3.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 9/16/31 ms
R-3#ping 192.168.1.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.1.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 10/15/18 ms

LAB: DYNAMIC ROUTING USING OSPF MULTIPLE AREA



Pre-requirement for LAB (check previous labs)

Design the topology (connectivity)

- Assign the IP address according to diagram
- Make sure that interfaces used should be in UP state

TASK:

- Dynamic routing using OSPF multiple area
- Verify Routing table and reachability between the LAN's (using PING and TRACE commands)

R-l#sh ip route

Gateway of last resort is not set

C 10.0.0.0/8 is directly connected, Serial0/0
C 192.168.1.0/24 is directly connected, FastEthernet0/0

R-2#sh ip route
Gateway of last resort is not set
C 10.0.0.0/8 is directly connected, Serial0/0
C 11.0.0.0/8 is directly connected, Serial0/1
C 192.168.2.0/24 is directly connected, FastEthernet0/0

R-3#sh ip route
Gateway of last resort is not set
C 11.0.0.0/8 is directly connected, Serial0/0
C 192.168.3.0/24 is directly connected, FastEthernet0/0

R-1(config-router)#network 192.168.1.0 0.0.0.255 area 10
R-1(config-router)#network 10.0.0.0 0.255.255.255 area 10

Router - 2
R-2(config)#router ospf 1
R-2(config-router)#network 192.168.2.0 0.0.0.255 area 0
R-2(config-router)#network n.0.0.0 0.255.255.255 area 20
R-2(config-router)#network 10.0.0.0 0.255.255.255 area 10
06:14:49: %OSPF-5-ADJCHG: Process 1,Nbr 192.168.1.100 on Serial0/0 from
LOADING to FULL, Loading Done

Router - 3
R-3(config)#router ospf 1
R-3(config-router)#network 192.168.3.0 0.0.0.255 area 20
R-3(config-router)#network n.0.0.0 0.255.255.255 area 20
06:15:46: %OSPF-5-ADJCFG: Process 1,Nbr 192.168.2.100 on Serial0/0 from
LOADING to FULL, Loading Done

Testing and verification

R-2#show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
192.168.3.100	0	FULL/	-00:00:39	11.0.0.2	Serial0/1
192.168.1.100	0	FULL/	-00:00:39	10.0.0.1	Serial0/0

R-1#show ip route

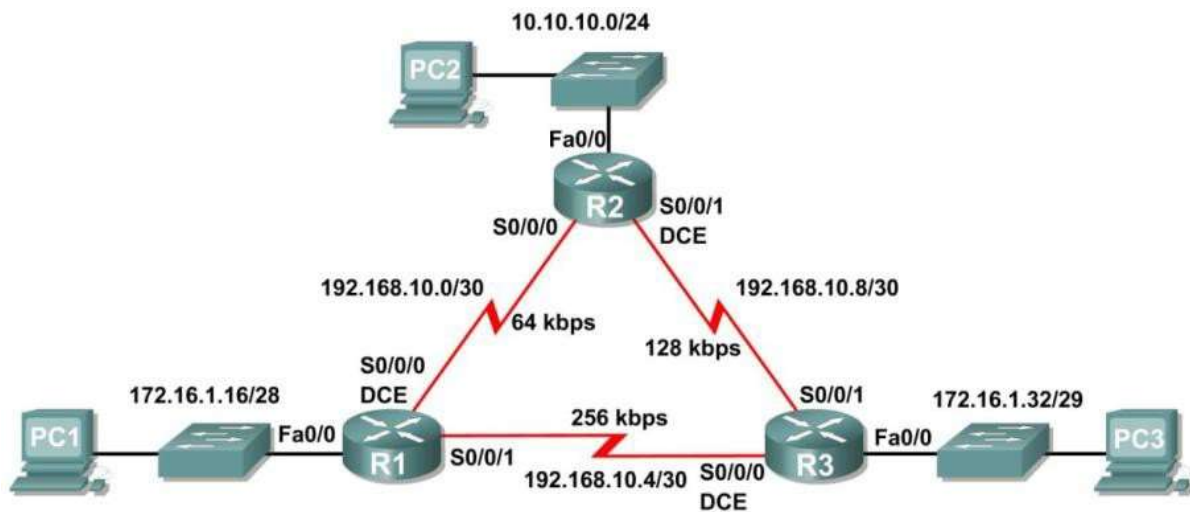
R-1#show ip route ospf

R-2#show ip route ospf

R-3#show ip route ospf

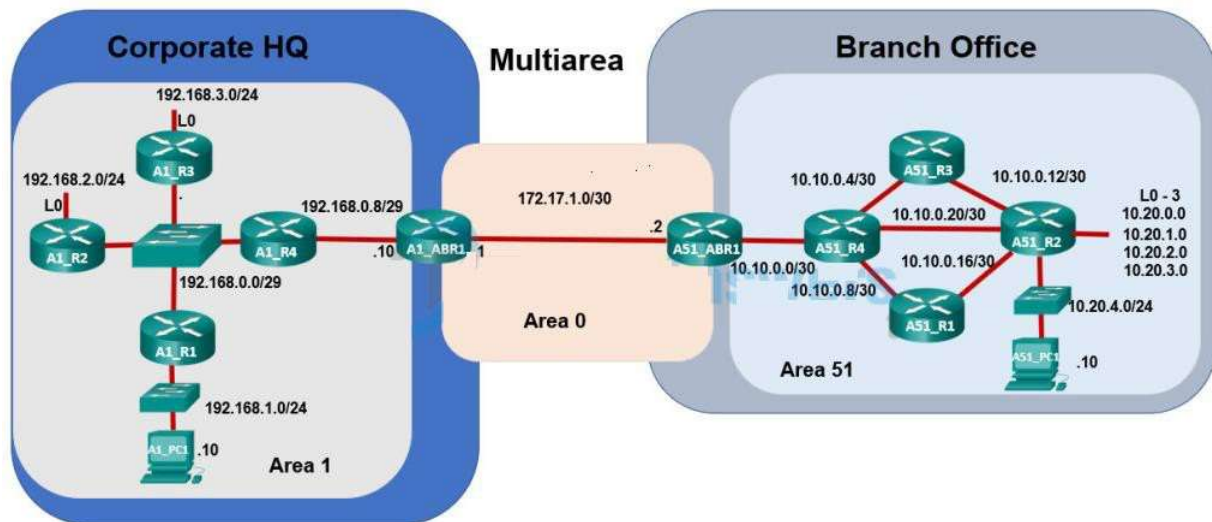
Practice Lab 1

Perform exercise in ospf single area according to the shown topology



Practice Lab 2

Perform exercise in ospf multi-area according to the shown topology



LAB 14

EIGRP

EIGRP-Enhanced Interior Gateway Routing Protocol

EIGRP is a Cisco proprietary routing protocol. It is one of the most popular and widely used routing protocols. It is a complex routing protocol, but it provides many benefits and supports all sizes and types of networks.

Advanced distance vector (Hybrid protocol)

Standard protocol (initially was cisco proprietary)

Classless routing protocol (carry subnet-mask, support subnets/VLSM)

Max Hop count is 255 (100 by default)

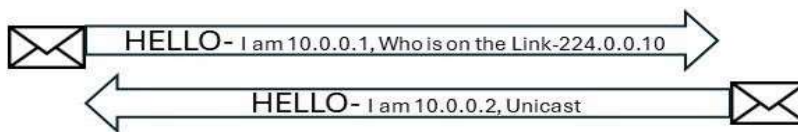
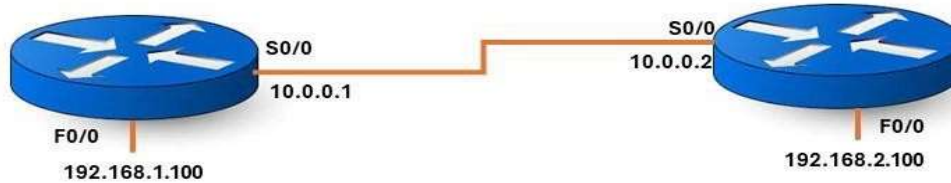
Administrative distance is 90 (Administrative distance is Trust worthiness)

Easy and Flexible network design, (unlike OSPF)

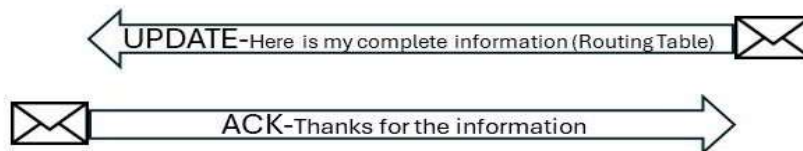
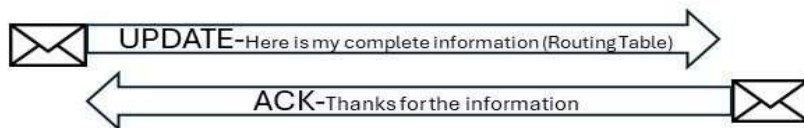
Uses Multicast (224.0.0.10) and unicast for initial neighbor discovery process

EIGRP step by step initial process

EIGRP step by step initial process



They become Neighbor and build **NEIGHBOR TABLE**



Converged, Calculate the best route builds routing table

EIGRP Tables

Neighbor table

Contains list of directly connected routers

show ip eigrp neighbor

Topology table

List of all the best routes learned from each neighbor

Show ip eigrp topology

Routing table

The best route to the destination

show ip route

EIGRP Convergence

Incremental updates

Periodically send hello packets are sent every 5 seconds
(dead - 15 sec)

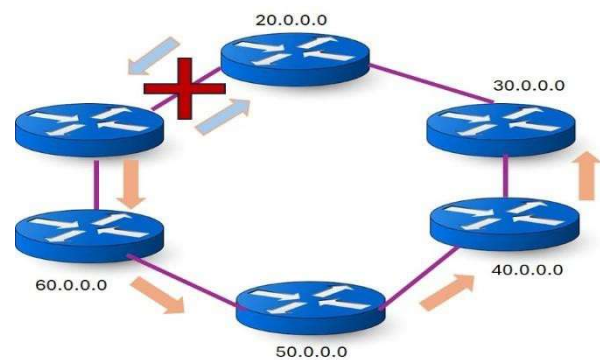
Convergence rate is fast (15 sec)

Also pre-calculates second best route

° **best route = successor**

° **Second best route = feasible successor**

EIGRP Metric - parameters

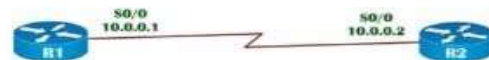


	Bandwidth	Delay	load	Reliability	MTU
Serial	1544 Kbps	20,000 Us	1 – 255	1 – 255	1500 bytes
Ethernet	10 Mbps	200 Us	1- less	1- less	
Fasteth	100 Mbps	100 Us	255 more	255 more	
gigeth	1000 Mbps	10 Us			

```
R-1(config)#interface serial 0/0
R-1(config-if)#bandwidth 1000
```

R-1#sh interfaces s0/0

```
Serial0/0 is up, line protocol is up (connected)
Hardware is HD64570
Internet address is 10.0.0.1/8
MTU 1500 bytes, BW 1000 Kbit, DLY 20000 usec,
reliability 255/255, txload 1/255, rxload 1/255
```



R-1#sh interfaces s0/0

```
Serial0/0 is up, line protocol is up (connected)
Hardware is HD64570
Internet address is 10.0.0.1/8
MTU 1500 bytes, BW 1544 Kbit, DLY 20000 usec,
reliability 255/255, txload 1/255, rxload 1/255
```

EIGRP Metric Calculation

BW	Delay	load	Reliability	MTU
$K1 = 1$	$K3 = 1$	$K2 = 0$	$K4 = 0$	$K5 = 0$

The EIGRP metric calculation formula is as below:

$$metric = \left[\left(K1 \times \frac{10^7}{BW_{min}} + \frac{K2 \times BW_{min}}{256 - load} + K3 \times \sum delays \right) \times \frac{K5}{K4 + reliability} \right] \times 256$$

By default uses BW and Delay in the metric calculation.

Formula with default K values ($K1 = 1$, $K2 = 0$, $K3 = 1$, $K4 = 0$, $K5 = 0$):

$$Metric = [K1 * BW + ((K2 * BW) / (256 - load)) + K3 * delay]$$

EIGRP Configuration

```
Router(config)# router eigrp <AS_number>
```

```
Router(config-router)# network network_address [wildcard_mask]
```

The above command needs two parameters: a network address and wildcard mask of the network. The network address is the network ID of the subnet that you want to include in the EIGRP operation. The wildcard mask is the range of addresses that you want to include in the operation.

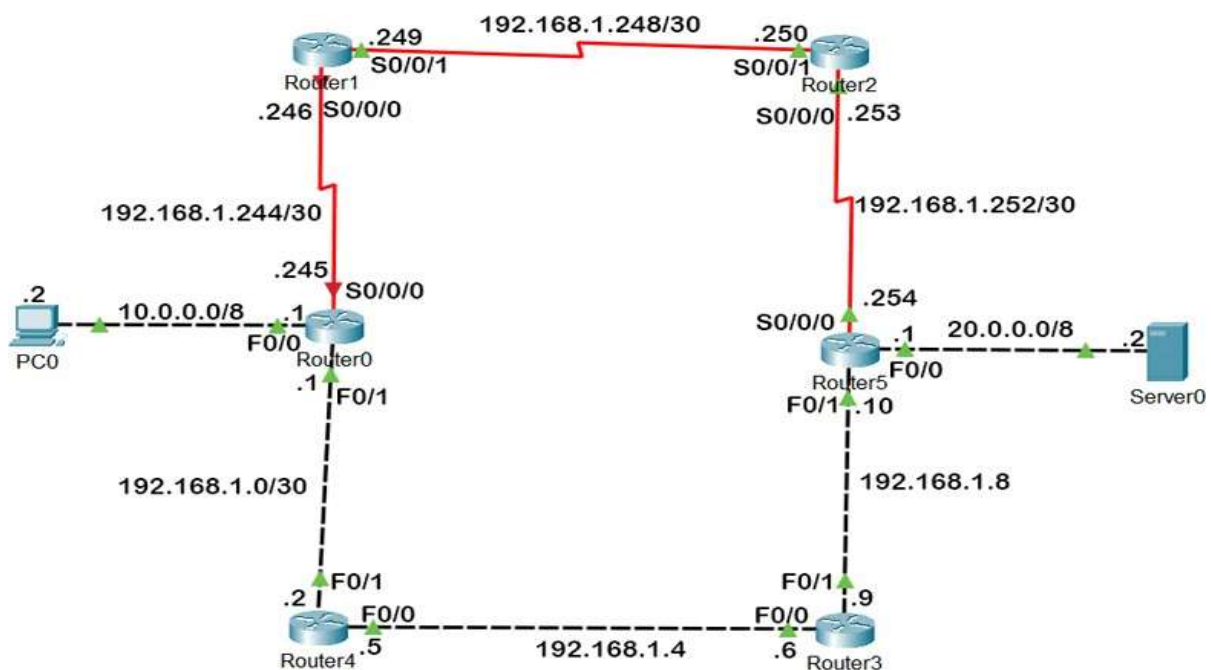
To calculate the wildcard mask, subtract the subnet mask from **255.255.255.255** (also called **universal subnet mask**).

To calculate the wildcard mask, subtract the subnet mask from **255.255.255.255**.

Subtract from	255.255.255.255	255.255.255.255	255.255.255.255	255.255.255.255
Subnet mask	255.255.0.0	255.255.255.252	255.128.0.0	255.0.0.0
Wildcard mask	0.0.255.255	0.0.0.3	0.127.255.255	0.255.255.255

Packet Tracer Activity

Take AS no. say 20 (can be any number from 1-65536 but same on all routers otherwise neighborship will be established)



Router0

```
Router-PT(config)#router eigrp 20
Router-PT(config-router)#network 10.0.0.0 0.0.0.255
Router-PT(config-router)#network 192.168.1.244 0.0.0.3
Router-PT(config-router)#network 192.168.1.0 0.0.0.3
```

Router1

```
Router(config)#router eigrp 20
Router(config-router)#network 192.168.1.244 0.0.0.3
Router(config-router)#network 192.168.1.248 0.0.0.3
Router(config-router)#
```

Router2

```
Router(config)#router eigrp 20
Router(config-router)#network 192.168.1.248 0.0.0.3
Router(config-router)#network 192.168.1.252 0.0.0.3
Router(config-router)#
```

Router3

```
Router(config)#router eigrp 20
Router(config-router)#network 192.168.1.8 0.0.0.3
Router(config-router)#network 192.168.1.4 0.0.0.3
Router(config-router)#
```

Router4

```
Router(config)#router eigrp 20
Router(config-router)#network 192.168.1.4 0.0.0.3
Router(config-router)#network 192.168.1.0 0.0.0.3
Router(config-router)#
```

Router5

```
Router(config)#router eigrp 20
Router(config-router)#network 20.0.0.0 0.255.255.255
Router(config-router)#network 192.168.1.252 0.0.0.3
```

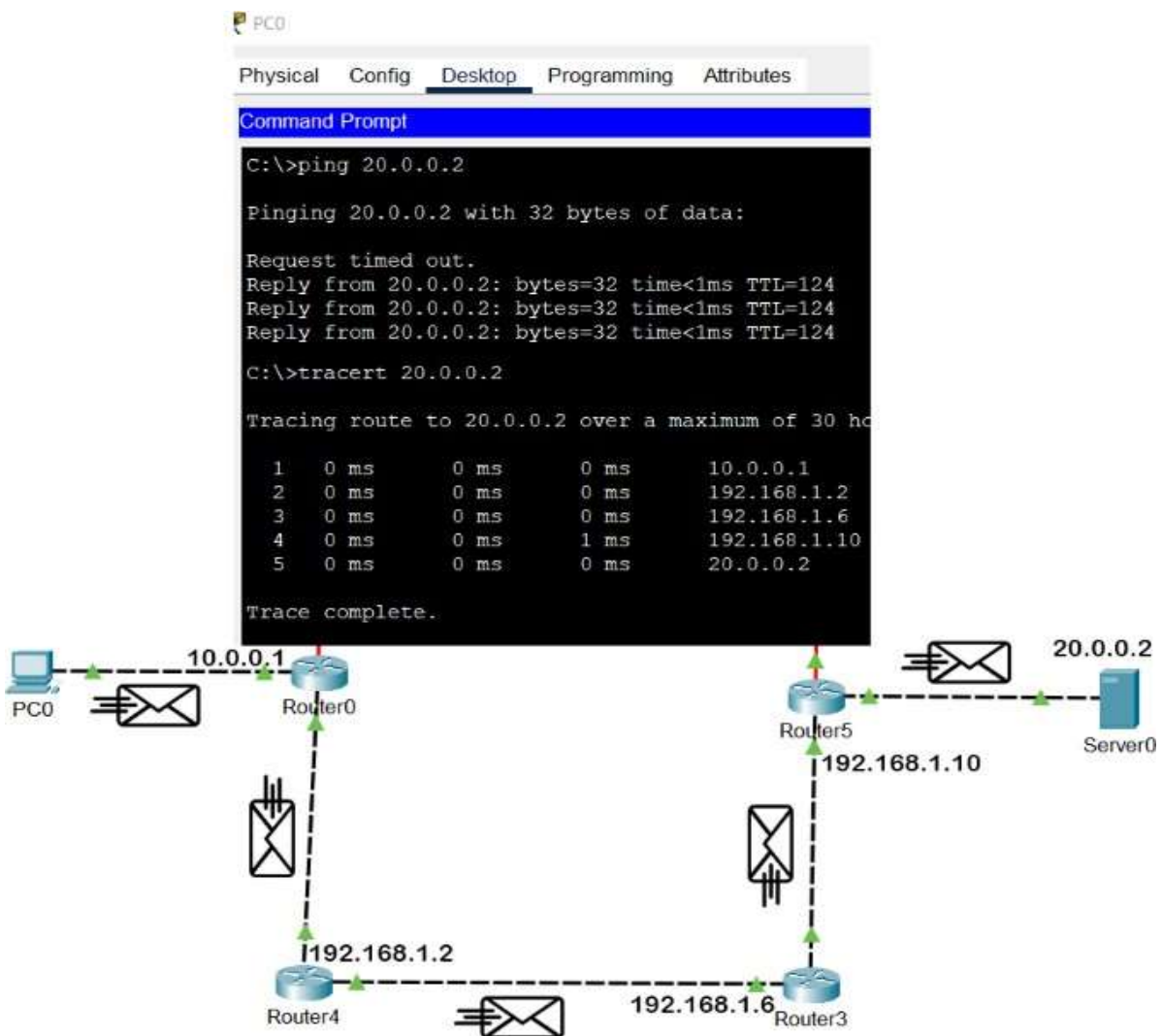
```
Router(config-router)#network 192.168.1.8 0.0.0.3
Router(config-router)#
```

Testing end-to-end connectivity

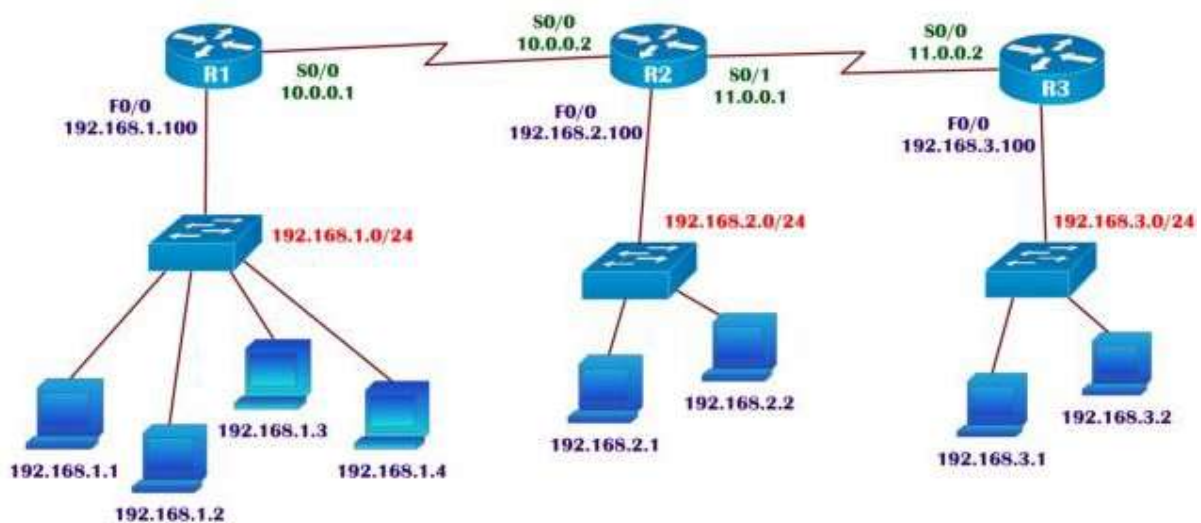
To test connectivity between two devices, we use the **ping** command. This command sends dummy data packets to the destination device and tracks the response. If it receives replies from the destination device, it lists the replies on the console. A successful reply verifies that the source and the destination have proper connectivity.

We can also use the **tracert** command. This command also sends dummy data packets to the destination device, but instead of tracking the response, it monitors the path the data packets take to reach the destination device.

In our example network, we have two end devices: **PC0** and **Server0**. The following image shows how to test connectivity between both devices.



Practice LAB 1: DYNAMIC ROUTING USING EIGRP



Pre-requirement for LAB (check previous labs)

Take AS no. say 100 (can be any number from 1-65536 but same on all routers otherwise neighborship will be established)

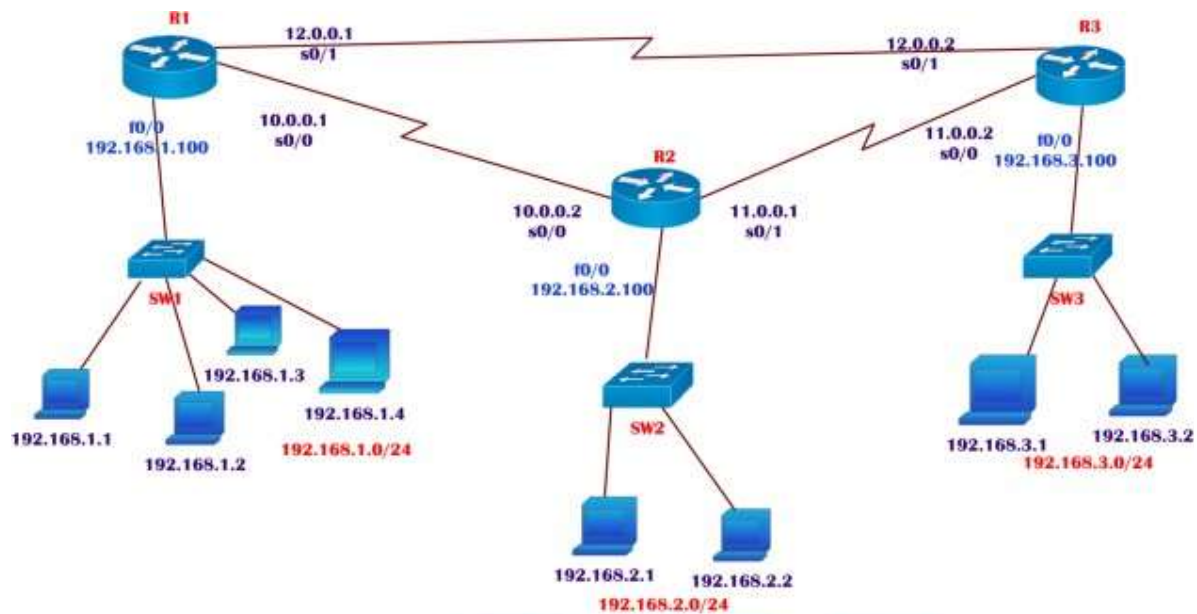
- Design the topology (connectivity)

-
- Assign the IP address according to diagram
 - Make sure that interfaces used should be in UP UP state

TASK

- Configure Dynamic routing using EIGRP 100
- Verify Routing table and reachability between the LAN's (using PING and TRACE commands)

Practice LAB 2: Dynamic Routing using EIGRP - using Redundant links



STEPS:

Pre-requirement for LAB (check previous labs)

- Design the topology (connectivity)
- Assign the IP address according to diagram
- Make sure that interfaces used should be in UP UP state

What we do in this lab

- Dynamic routing using EIGRP for redundant links
- Verify Routing table and reachability between the LAN's (using PING and TRACE commands)
- Shutdown the interface and verify redundancy in Routing.

LAB 15

Configure a Wireless Network

Objective:

This lab explains how to configure and connect wireless network in packet tracer step by step with examples. Learn how to secure a wireless network with WAP key and enable static routing in router with packet tracer.

Activity Outcomes:

- Connect to a wireless router
- Configure the wireless router
- Connect a wired device to the wireless router
- Connect a wireless device to the wireless router
- Add an AP to the network to extend wireless coverage
- Update default router settings

The activities provide hands - on practice to configure RIP Routing protocol on router.

Useful Concepts

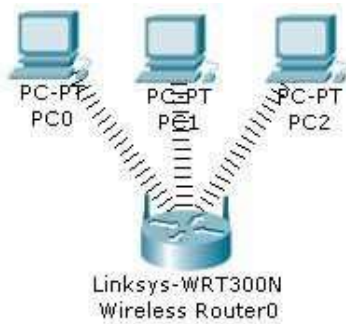
WLANs (Wireless LANs) are very common in today's World. Everywhere there are a lot of wireless networks. Even now, you are in many of these wireless signals. It is not an healthy life but they are in our lives. This lab explains how to configure and connect wireless network in packet tracer step by step with examples. Learn how to secure a wireless network with WAP key and enable static routing in router with packet trace.

For more details, please check the below reference.

Reference:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.
- <https://www.computernetworkingnotes.com/ccna-study-guide/how-to-configure-wireless-network-in-packet-tracer.html>

- <https://ipccisco.com/lesson/wlan-configuration-on-packet-tracer/>
- <https://www.cisco.com/c/en/us/support/docs/wireless-mobility/wireless-lan-wlan/68005-wlan-connect.html>



1. Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>45</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1: Create a practice lab as shown in following figure.

In this practice lab we have three pc connected from Linksys Wireless routers with following configuration.

- DHCP is configured and enabled on Wireless router
- IP pool for DHCP is 192.168.0.100 to 192.168.0.150
- PC are configured to receive IP from DHCP Server
- No security is configured
- Default SSID is configured to Default
- Topology is working on infrastructure mode
- Default user name and password is admin
- IP of wireless is set to 192.168.0.1

Now your task is to:-

-
- Configure Static IP on PC and Wireless Router
 - Change SSID to MotherNetwork
 - Change IP address of router to 10.0.0.1 and 10.0.0.2 of PC0 10.0.0.3 of PC1 10.0.0.4 of PC2
 - Secure your network by configuring WAP key on Router
 - Connect PC by using WAP key

Step 1: Initial IP Configuration

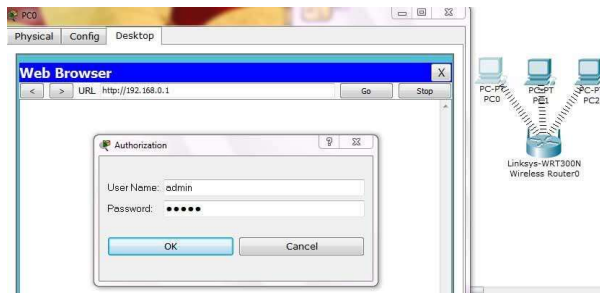
To complete these tasks follow these step by step instructions

As given in question our network is running on 192.168.0.0 network and all PC's are DHCP clients and functioning properly. So we will first connect to Wireless router to off DHCP.

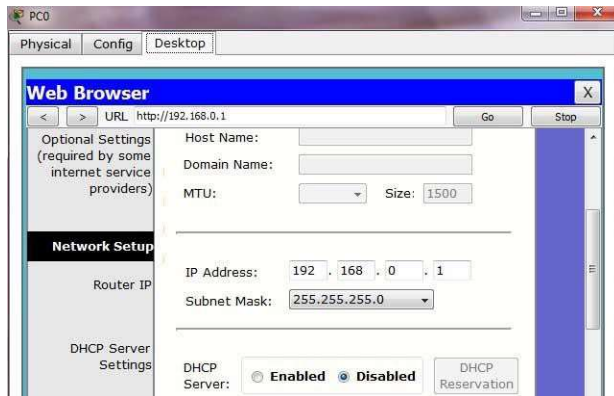
Adding Wireless NIC to PC

- i) Click PC
- ii) Click Physical
- iii) Add WMP300N (under Modules Tab) (you must turn OFF PC first before adding this module and turn back ON after adding)

Double click on PC and select Web Browser. As given in question IP of Wireless router is 192.168.0.1 so give it in Web browser and press enter, now it will ask for authentication which is also given in question. Give user name admin and Password to admin



This will bring GUI mode of Wireless router. Scroll down screen to Network Step and Select Disable DHCP



Go in end of page and click on Save setting this will save setting click on continue for further setting



Now select Administration from top Menu and change password to test and go in the end of page and Click on Save Setting

Again go in the end of page and Click on Save Setting

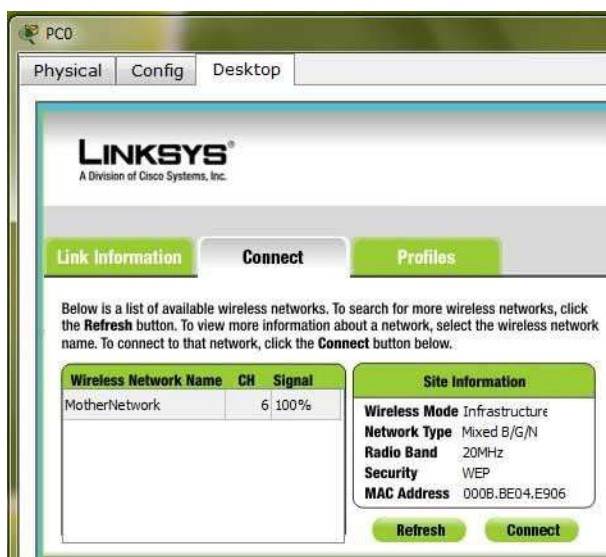
Now we have completed all given task on Wireless router. Now configure the static IP on all three PC's

Double click on pc select Desktop tab click on IP configuration select Static IP and set IP as given below

PC	IP	Subnet Mask	Default Gateway
PC0	192.168.0.2	255.255.255.0	192.168.0.1
PC1	192.168.0.3	255.255.255.0	192.168.0.1
PC2	192.168.0.4	255.255.255.0	192.168.0.1

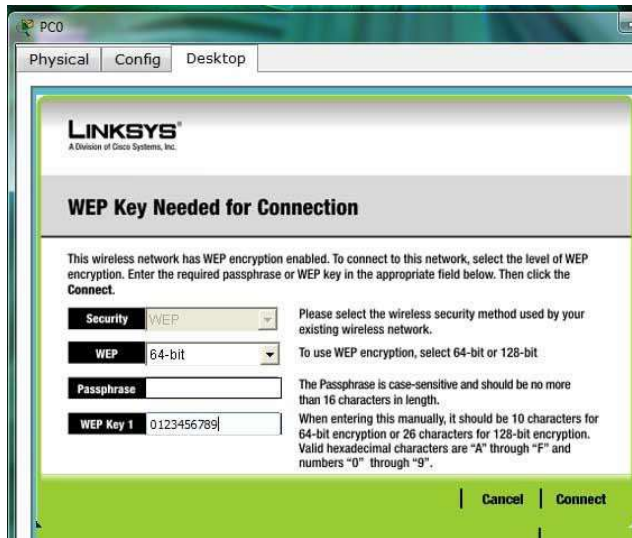
Now it's time to connect PC's from Wireless router. To do so click PC select Desktop click on PC Wireless

Click on connect tab and click on Refresh button



As you can see in image that Wireless device is accessing MotherNetwork on CH 6 and signal strength is 100%. In left side you can see that WEP security is configured in network. Click on connect button to connect MotherNetwork

It will ask for WAP key insert 0123456789 and click connect



It will connect you with wireless router.

As you can see in image below that system is connected. And PCI card is active.



Repeat same process on PC1 and PC2.

Click on continue for further setting. This time it will ask you to authenticate again give new password test this time

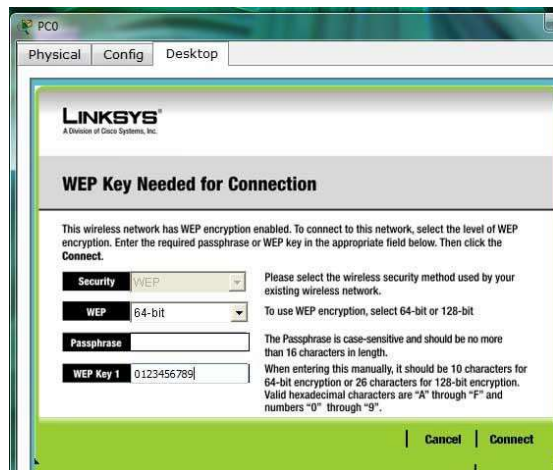
Now click on wireless tab and set default SSID to MotherNetwork



Now Select wireless security and change Security Mode to WEP



Set Key1 to 0123456789



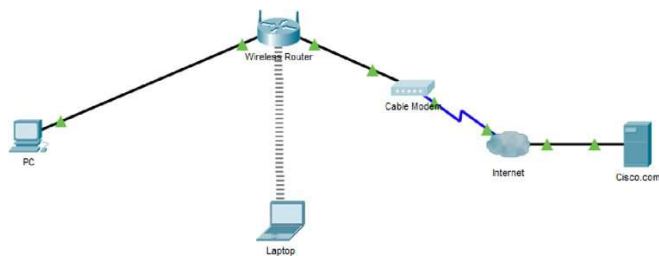
It will connect you with wireless router.
As you can see in image below that system is connected. And PCI card is active.



Repeat same process on PC1 and PC2

Home Task

Configure the wireless network



Addressing Table

Device	Interface	IP Address	Subnet Mask	Default Gateway
PC	Ethernet0	DHCP		192.168.0.1
Wireless Router	LAN	192.168.0.1	255.255.255.0	
Wireless Router	Internet	DHCP		
Cisco.com Server	Ethernet0	208.67.220.220	255.255.255.0	
Laptop	Wireless0	DHCP		

Task 1: Build a Simple Network in the Logical Topology Workspace

Task 2: Configure the Network Devices

Task 3: Test Connectivity between Network Devices

Task 4: Save the File and Close Packet Tracer

LAB 16

Configure IPv4 Access Control List (ACL)

Objective

This lab explains how to configure, view, edit, update and delete a standard named access control. Learn how to create and manage a standard access list through a packet tracer example.

Activity Outcome

- Plan an ACL Implementation
- Configure, Apply, and Verify a Standard ACL

1. Useful Concepts

An Access Control List (ACL) is a list of network traffic filters and correlated actions used to improve security. It blocks or allows users to access specific resources. ACLs are characterized by single or multiple permit/deny statements. The purpose is to filter inbound or outbound packets on a selected network interface. This activity focuses on defining filtering criteria, configuring standard ACLs, applying ACLs to router interfaces, and verifying and testing the ACL implementation. There are two types of IPv4 ACLs:

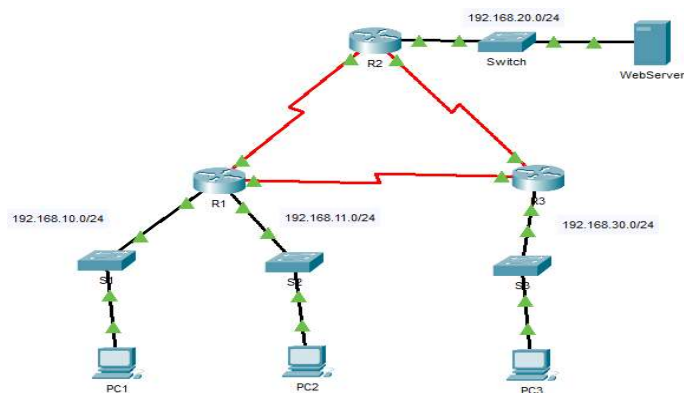
- *Standard ACLs*: These ACLs permit or deny packets based only on the source IPv4 address.
- *Extended ACLs*: These ACLs permit or deny packets based on the source IPv4 address and destination IPv4 address, protocol type, source and destination TCP or UDP ports, and more.

This lab will concentrate on standard ACL. For more details, please check the below reference.

Reference:

- Jim Kurose, Keith Ross, "Computer Networking: A Top-Down Approach," 8th edition, Pearson, 2020.

- <https://www.ciscopress.com/articles/article.asp?p=3089353&seqNum=7>
- <https://www.computernetworkingnotes.com/ccna-study-guide/configure-standard-access-control-list-step-by-step-guide.html>
- <https://ipccisco.com/lesson/access-control-lists/>



Addressing Table

Device	Interface	IP Address	Subnet Mask	Default Gateway
R1	G0/0	192.168.10.1	255.255.255.0	N/A
R1	G0/1	192.168.11.1	255.255.255.0	N/A
R1	S0/0/0	10.1.1.1	255.255.255.252	N/A
R1	S0/0/1	10.3.3.1	255.255.255.252	N/A
R2	G0/0	192.168.20.1	255.255.255.0	N/A
R2	S0/0/0	10.1.1.2	255.255.255.252	N/A
R2	S0/0/1	10.2.2.1	255.255.255.252	N/A
R3	G0/0	192.168.30.1	255.255.255.0	N/A
R3	S0/0/0	10.3.3.2	255.255.255.252	N/A
R3	S0/0/1	10.2.2.2	255.255.255.252	N/A
PC1	NIC	192.168.10.10	255.255.255.0	192.168.10.1
PC2	NIC	192.168.11.10	255.255.255.0	192.168.11.1

PC3	NIC	192.168.30.10	255.255.255.0	192.168.30.1
WebServer	NIC	192.168.20.254	255.255.255.0	192.168.20.1

1. Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>10</i>	<i>Medium</i>	<i>CLO-6</i>

Activity 1: Initial Configuration

Part 1: Plan an ACL Implementation

Step 1: Investigate the current network configuration.

Before applying any ACLs to a network, it is important to confirm that you have full connectivity. Verify that the network has full connectivity by choosing a PC and pinging other devices on the network. You should be able to successfully ping every device.

Step 2: Evaluate two network policies and plan ACL implementations.

a. The following network policies are implemented on **R2**:

- The 192.168.11.0/24 network is not allowed access to the **WebServer** on the 192.168.20.0/24 network.
- All other access is permitted.

To restrict access from the 192.168.11.0/24 network to the **WebServer** at 192.168.20.254 without interfering with other traffic, an ACL must be created on **R2**. The access list must be placed on the outbound interface to the **WebServer**. A second rule must be created on **R2** to permit all other traffic.

b. The following network policies are implemented on **R3**:

- The 192.168.10.0/24 network is not allowed to communicate with the 192.168.30.0/24 network.
- All other access is permitted.

To restrict access from the 192.168.10.0/24 network to the 192.168.30.0/24 network without interfering with other traffic, an access list will need to be created on **R3**. The ACL must be placed on the outbound interface to **PC3**. A second rule must be created on **R3** to permit all other traffic.

Part 2: Configure, Apply, and Verify a Standard ACL

Step 1: Configure and apply a numbered standard ACL on R2.

- a. Create an ACL using the number **1** on **R2** with a statement that denies access to the 192.168.20.0/24 network from the 192.168.11.0/24 network.

Open configuration window

```
R2(config)# access-list 1 deny 192.168.11.0 0.0.0.255
```

- b. By default, an access list denies all traffic that does not match any rules. To permit all other traffic, configure the following statement:

```
R2(config)# access-list 1 permit any
```

- c. Before applying an access list to an interface to filter traffic, it is a best practice to review the contents of the access list, in order to verify that it will filter traffic as expected.

```
R2# show access-lists
```

```
Standard IP access list 1
  10 deny 192.168.11.0 0.0.0.255
  20 permit any
```

- d. For the ACL to actually filter traffic, it must be applied to some router operation. Apply the ACL by placing it for outbound traffic on the GigabitEthernet 0/0 interface. Note: In an actual operational network, it is not a good practice to apply an untested access list to an active interface.

```
R2(config)# interface GigabitEthernet0/0
```

```
R2(config-if)# ip access-group 1 out
```

Step 2: Configure and apply a numbered standard ACL on R3.

- a. Create an ACL using the number **1** on **R3** with a statement that denies access to the 192.168.30.0/24 network from the **PC1** (192.168.10.0/24) network.

```
R3(config)# access-list 1 deny 192.168.10.0 0.0.0.255
```

- b. By default, an ACL denies all traffic that does not match any rules. To permit all other traffic, create a second rule for ACL 1.

```
R3(config)# access-list 1 permit any
```

- c. Verify that the access list is configured correctly.

```
R3# show access-lists
```

```
Standard IP access list 1
```

```
10 deny 192.168.10.0 0.0.0.255
20 permit any
```

- d. Apply the ACL by placing it for outbound traffic on the GigabitEthernet 0/0 interface.

```
R3(config)# interface GigabitEthernet0/0
R3(config-if)# ip access-group 1 out
```

Step 3: Verify ACL configuration and functionality.

- a. Enter the **show run** or **show ip interface gigabitethernet 0/0** command to verify the ACL placements.
- b. With the two ACLs in place, network traffic is restricted according to the policies detailed in Part 1. Use the following tests to verify the ACL implementations:
- A ping from 192.168.10.10 to 192.168.11.10 succeeds.
 - A ping from 192.168.10.10 to 192.168.20.254 succeeds.
 - A ping from 192.168.11.10 to 192.168.20.254 fails.
 - A ping from 192.168.10.10 to 192.168.30.10 fails.
 - A ping from 192.168.11.10 to 192.168.30.10 succeeds.
 - A ping from 192.168.30.10 to 192.168.20.254 succeeds.
- c. Issue the **show access-lists** command again on routers **R2** and **R3**. You should see output that indicates the number of packets that have matched each line of the access list. Note: The number of matches shown for your routers may be different, due to the number of pings that are sent and received.

```
R2# show access-lists
Standard IP access list 1
 10 deny 192.168.11.0 0.0.0.255 (4 match(es))
 20 permit any (8 match(es))
```

```
R3# show access-lists
Standard IP access list 1
 10 deny 192.168.10.0 0.0.0.255 (4 match(es))
 20 permit any (8 match(es))
```

Close configuration window

Answer Configurations

Router R2

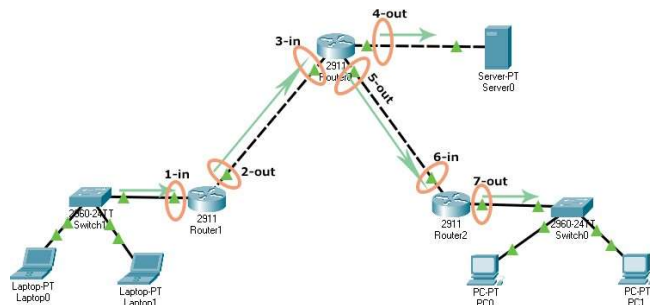
```
enable
configure terminal
interface GigabitEthernet0/0
 ip access-group 1 out
access-list 1 deny 192.168.11.0 0.0.0.255
access-list 1 permit any
end
```

Router R3

```
enable
configure terminal
interface GigabitEthernet0/0
 ip access-group 1 out
access-list 1 deny 192.168.10.0 0.0.0.255
access-list 1 permit any
end
End of document
```

Create and implement a standard access list that blocks the Students section from accessing the Server section. Students section uses IP subnet 10.0.0.0/8. All packets originating from this section have an IP address from this subnet. If we create a standard ACL with a deny statement for this subnet, all packets having an IP address from this subnet in their source address will be dropped.

In our network, we have seven locations where we can implement the ACL. The following image shows these locations and the direction in which they can be used to filter traffic.



The following table lists the above locations and the effect of the ACL on each location.

Location	Interface	Direction	Effect
1	Router1's Gig0/0	In	The Students section will not be able to access the Server and the Teachers section.
2	Router1's Gig0/2	Out	The Students section will not be able to access the Server and Teachers section.
3	Router0's Gig0/2	In	The Students section will not be able to access the Server and Teachers section.
4	Router0's Gig0/0	Out	The Students section will not be able to access the Server section but it will be able to access the Teachers section.
5	Router0's Gig0/1	Out	The Students section will not be able to access the Teachers section but it will be able to access the Server section.
6	Router1's Gig0/1	In	The Students section will not be able to access the Teachers section but it will be able to access the Server section.
7	Router1's Gig0/0	Out	The Students section will not be able to access the Teachers section but it will be able to access the Server section.

Firewall

(Allow Good Traffic and Deny Bad Traffic)

A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules. it is system that is designed to prevent unauthorized access from entering a private network. A firewall can be a Hardware or software or Combination of Hardware and software protect an entire Network.



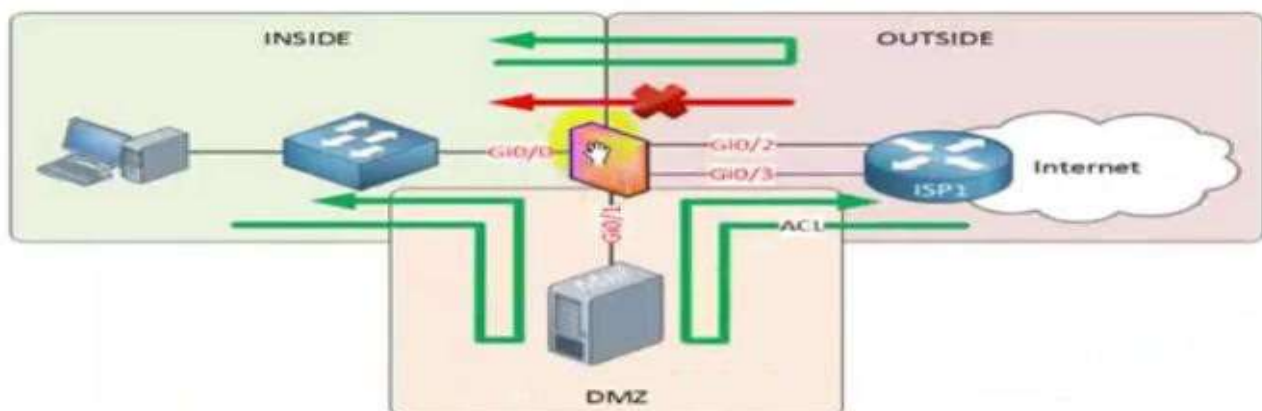
Firewall Zones

Inside Zone (Trusted or Private Zone) Generally LAN , Security level highest and # 100, Purely the area or Zone under your network control.

Outside Zone (Untrusted Zone) Generally WAN or Internet, least security # 0

DMZ (Demilitarized Zone) Area or zone under your control but outside traffic under certain rules allowed, Generally email servers, web servers, proxy servers. Security #1-99

Traffic from INSIDE to OUTSIDE, is permitted	Traffic from DMZ to INSIDE, is denied
Traffic from INSIDE to DMZ, is permitted	Traffic from OUTSIDE to DMZ, is denied
Traffic from DMZ to OUTSIDE, is permitted	Traffic from OUTSIDE to INSIDE, is denied



Firewalls determine the allowed traffic versus the disallowed traffic based on many characteristics of the packets, including their destination and source IP addresses and the TCP and UDP port numbers (which imply the application protocol).

Firewalls also examine the application layer headers.

The term firewall is taken from the world of building and architecture.

A firewall in a building has two basic requirements. It must be made of fire-resistant materials, and the architect limits the number of openings in the wall (doors, conduits for wires and plumbing), limiting the paths through which the fire can spread.

A network firewall must itself be hardened against security attacks.

It must disallow all packets unless the engineer has configured a firewall rule that allows the traffic—a process often called “opening a hole,” again with analogies to a firewall in a building.

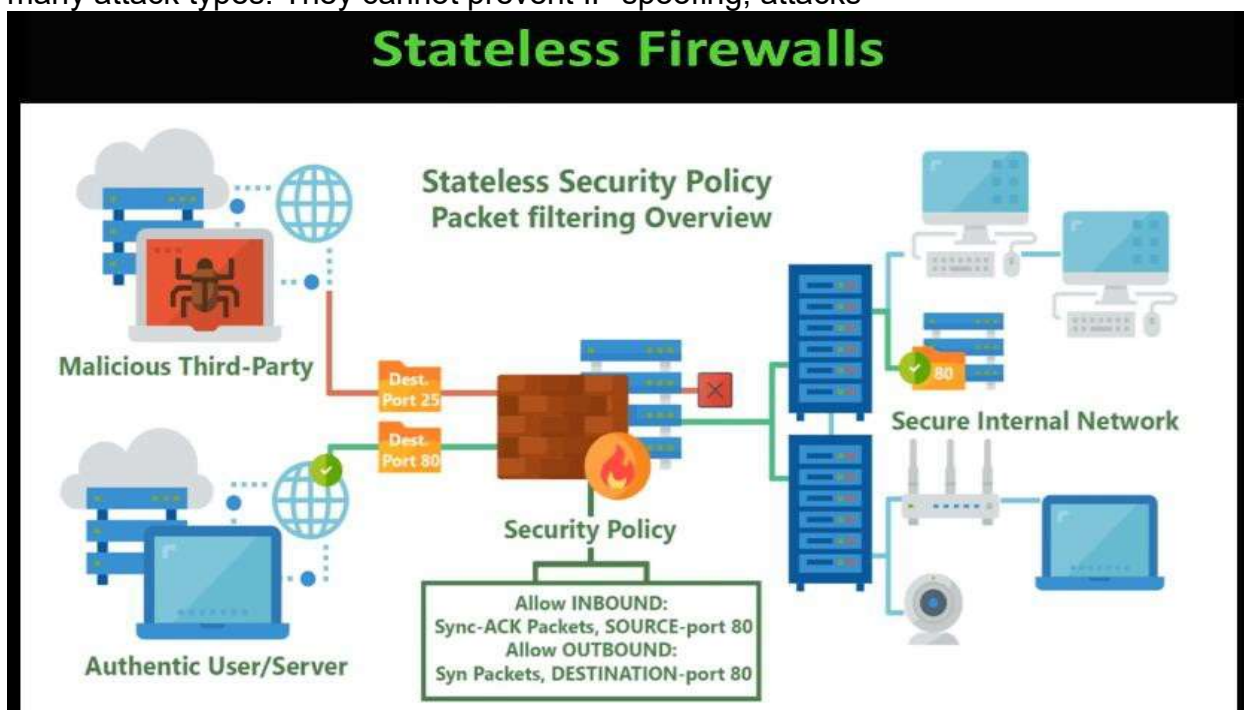
Firewalls sit in the packet-forwarding path between two networks, often with one LAN interface connecting to the secure local network, and one to the other, less-secure network (often the Internet).

Additionally, because some hosts in the Enterprise need to be accessible from the Internet—an inherently less secure practice—the firewall typically also has an interface connected to another small part of the Enterprise network, called the demilitarized zone (DMZ).

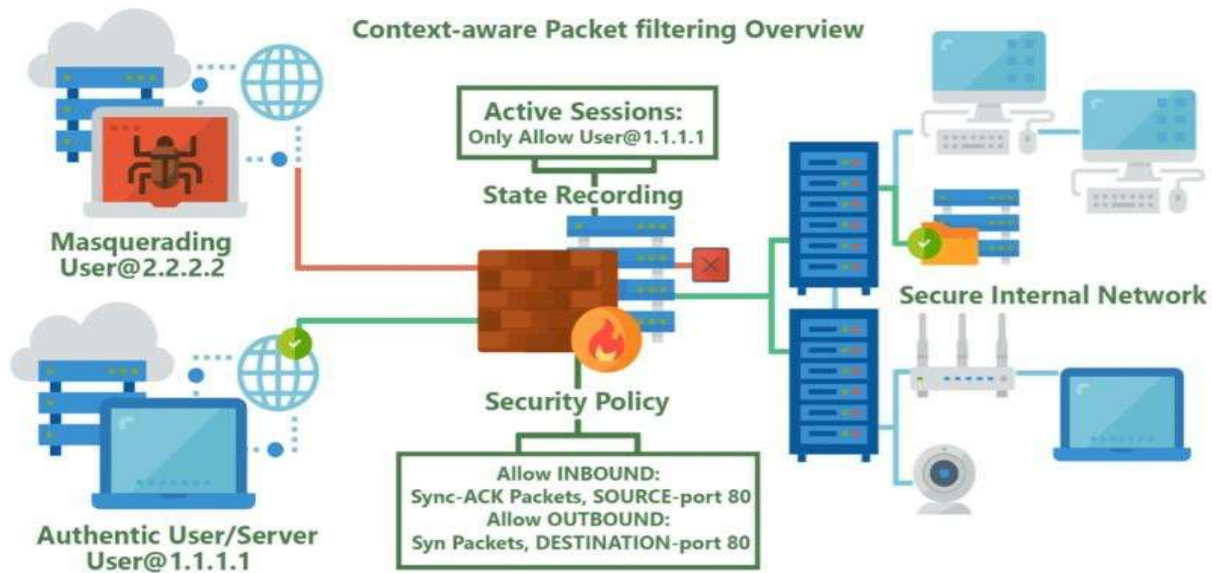
Firewall Techniques

- Stateless firewalls
- Stateful firewalls
- Next Generation Firewall
- UTM Firewall
- Personal Firewall
- Proxy Firewall
- Transparent Firewall
- Application Firewall
- Cloud firewalls
- Software firewalls
- Hardware firewalls
- Zone Base firewall
- Open-Source firewall

Packet filtering firewalls are the least detrimental to throughput because they only inspect the header of the packet for allowed IP addresses or port numbers. Although even performing this function will slow traffic, it involves only looking at the beginning of the packet and making a quick allow or disallow decision. Although packet filtering firewalls serve an important function, they cannot prevent many attack types. They cannot prevent IP spoofing, attacks

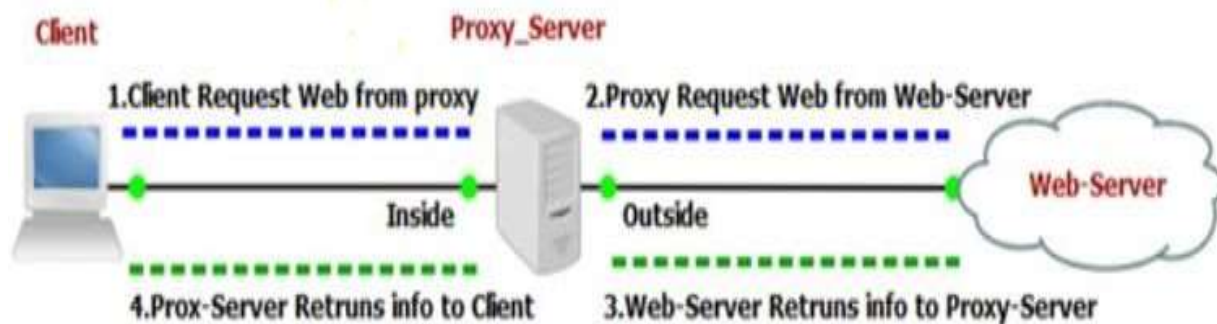


Stateful Firewalls



Proxy Firewall

stand between each connection from the outside to the inside and make the connection on behalf of the endpoints. Therefore, there is no direct connection. The proxy firewall acts as a relay between the two endpoints.



Configuring ASA (Cisco) Firewall

Adaptive Security Appliance (ASA) is a Cisco security appliance that combines classic firewall features with VPN, Intrusion Prevention, and antivirus capabilities. It has the capability to provide threat defense before the attacks spread into the networks.



It is like a router with multiple interfaces and has certain additional capabilities. Its every interface (to be utilized) is configured like IP Address is and interface name is configured as Inside ,outside and DMZ which automatically provides the defined characteristics to each interface working as Inside ,outside and DMZ.

As an administrator, we have to ensure protection against unauthorized access to our firewall. We can set login passwords, enable passwords for this. Also, we will discuss configuring an IP address on the ASA interface.

Administrative Configuration –

Bring up the interface and assign an IP address to ASA.

To configure an IP address on the interface of an ASA, we have to configure 4 things:

1. Bring up the interface –

After entering into global interface mode, use the command `no shut` to bring up the interface.

2. Assign an IP address to the interface of ASA –

After bringing up the interface, assign an IP address by the command
`IP address IP_address Subnet_Mask`

It's the same way by which we assign an IP address to the router's interface. But the difference is that we can assign an IP address to the ASA interface without the subnet mask also.

IP address IP_address

Now, if we don't give a subnet mask, it automatically takes a classful subnet mask. for example, if we assign 192.168.1.1 to the ASA interface it will automatically take 255.255.255.0 as a subnet mask.

3. Assign a nameif to the ASA interface –

In ASA, we also assign a name to the interface otherwise the interface will be down. The most common names are INSIDE OUTSIDE or DMZ. These names are used while applying a policy but have no role in forwarding the traffic.

We can assign a name to an ASA interface by the command:

nameif NAME

NAME is the name you want to give to an interface.

4. Assign a security level to the interface –

The security level is an integer value ranging from 0 to 100. It tells the trustworthiness of an interface i.e which interface is most trusted. 0 means less trusted while 100 means the most trusted.

If we provide the name INSIDE to an interface, it will automatically provide security level 100 to it and if we provide any other name like OUTSIDE or DMZ, it will assign automatically 0 to it but can be changed manually.

We can assign a security level to an interface by the command:

Security-level {value}

Here is an example where we will provide IP address 192.268.1.1 and subnet mask 255.255.255.0, name as INSIDE and security-level as 100.

```
asa(config)#int e0
```

```
asa(config-if)#no shut
```

```
asa(config-if)#ip address 192.168.1.1 255.255.255.0
```

```
asa(config-if)#nameif INSIDE
```

```
asa(config-if)#security level 100
```

Giving hostname to ASA –

It is used to set a name to a device stating an identity to a device. It is given by the same command that is used on the router:-

```
asa(config)#hostname ciscoasa
```

```
ciscoasa(config)#
```

After this primitive or very basic configuration of firewall, everything is blocked as per table below

Traffic from INSIDE to OUTSIDE, is permitted	Traffic from DMZ to INSIDE, is denied
Traffic from INSIDE to DMZ, is permitted	Traffic from OUTSIDE to DMZ, is denied
Traffic from DMZ to OUTSIDE, is permitted	Traffic from OUTSIDE to INSIDE, is denied

Then the policies are defined according to ACL-Access control List to perform the desired functionality of firewall by the network administrator. For example

Access control List				
Permission	IP Address	Protocol	Destination	port
Allow	192.168.1.10	TCP	ANY	80
Allow	192.168.1.12	TCP	172.33.33.10	443
Deny	192.168.0.2	TCP	ANY	80
Allow	ANY	TCP	ANY	80
Deny	ANY	UDP	ANY	69
Allow	10.0.0.10	TCP	103.10.12.20	21
Allow	ANY	TCP/UDP	ANY	53

Exercise Lab 1

Take ASA 5506 firewall having 8 gigabit ethernet interfaces. Configure 3 interfaces with inside, 1 outside and 2 as DMZ. Then test it for default behavior without any ACL policy implemented.

Exercise Lab 2

Take ASA 5505 firewall having 7 fast ethernet interfaces. Configure 2 interfaces with inside, 1 outside and 2 as DMZ capability. Then test it for default behavior without any ACL policy implemented. Again test it for ACL

1. One PC from first interface of inside be blocked for using internet (http traffic)

-
2. One PC from DMZ LAN be allowed to communicate with one inside PC connected to 2nd inside interface LAN.